

RWTH AACHEN UNIVERSITY

The present work was submitted to:

Chair of Process and Data Science
RWTH Aachen University

State-Based Process Monitoring in Traditional Event Logs for Concept Drift Detection

Bachelor Thesis

presented by
Bora Deren
438377

First Examiner: Prof. Dr. Wil M. P. van der Aalst
Second Examiner: Prof. Dr. Sander Leemans

Advisor: Dr. Alessandro Berti

Munich, October 25, 2026

Acknowledgments

I would like to thank my advisor Dr. Alessandro Berti for his supervision, guidance and feedback during the development of this thesis. I also thank Prof. Dr. Wil van der Aalst and Prof. Sander Leemans for their time as examiners. I further acknowledge the use AI-based tools, including ChatGPT and Claude to review writing and support language refinement. Claude was additionally used to assist coding tasks for the implementation. All content and resulst were reviewed and validated by the author.

Abstract

Process mining analyses event logs from information systems in order to understand and enhance real processes. These processes, however, change during an event log captures them. A such concept drift does not only concern the order of activites, but also the working principles of resources and the situation of all running cases together. Existing detectors mostly focus only on the control-flow and publicly available test data does not contain resource drifts.

This thesis introduces an unsupvervised, state-based framework that monitors a process through intra-case, resource and inter-case states. For each perspective feature extraction, dimensionality reduction and clustering form the states and a drift signal compares their distributions over time. Differently than control-flow-based detectors, the framework detects drifts in all three perspectives, and states describe what changed. Our generator Rheon generates synthetic event logs with injected drifts supporting nine drift types from three perspectives. The library kairo and a dashboard with an AI assistant implement the framework.

We evaluate the framework on 34 logs of the Padella benchmark with 29 logs where a drift was injected into a real log and on 45 Rheon logs with 103 drifts. With the hand-tunes configurations, the framework achieves F1-scores from 0.862 to 0.933, and with configurations from the AI agents only from 0.309 to 0.774. The difference is especially caused by the precision: Many false positives of the agent configurations are real changes what are unknown by the benchmark or reactions to different perspectives. The states are thereby a useful and explainable indicator for concept drifts as long as the appropriate configurations are found. The real difficult problem is, therefore, finding appropriate configurations, and rules of thumb for window sizes and thresholds offer a start point for that.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 2
- 1.2 Problem Statement . . . 5
- 1.3 Research Questions . . . 6
- 1.4 Research Goals . . . 6
- 1.5 Contributions . . . 7
- 1.6 Thesis Structure . . . 8

2 Related Work **9**
- 2.1 Drift Detection in Control-Flow . . . 9
- 2.2 Drift Detection Beyond Control-Flow . . . 10
- 2.3 Clustering-Based Drift Detection . . . 10
- 2.4 States in Process Mining . . . 10
- 2.5 Event Logs with Known Drifts . . . 11
- 2.6 Positioning of This Thesis . . . 12

3 Preliminaries **13**
- 3.1 Foundations . . . 13
  - 3.1.1 Mathematical Preliminaries . . . 13
  - 3.1.2 Event Logs, Cases and Traces . . . 15
  - 3.1.3 Process Trees . . . 17
  - 3.1.4 Calendar Windows . . . 17
- 3.2 Concept Drift . . . 19
- 3.3 Scaling and Principal Component Analysis . . . 21
- 3.4 Clustering . . . 24
  - 3.4.1 k-Means . . . 24
  - 3.4.2 Self-Organizing Maps . . . 25
  - 3.4.3 DBSCAN . . . 26
  - 3.4.4 Clustering Evaluation . . . 27
- 3.5 Divergences . . . 30

4 Implementation (Log Drift Generator) **33**
- 4.1 Generated Log Schema . . . 33
- 4.2 Generation of an Event Log . . . 34
- 4.3 Drifts . . . 36
- 4.4 Examples . . . 38
- 4.5 Limitations and Future Work . . . 41

5 State-Based Drift Detection **45**
5.1 Feature Extraction . . . 46
5.1.1 Intra-Case Perspective . . . 49
5.1.2 Resource Perspective . . . 50
5.1.3 Inter-Case Perspective . . . 53
5.2 Scaling and Dimensionality Reduction . . . 55
5.2.1 Intra-Case Perspective . . . 57
5.2.2 Resource Perspective . . . 58
5.2.3 Inter-Case Perspective . . . 59
5.3 Clustering into States . . . 59
5.3.1 Intra-Case Perspective . . . 61
5.3.2 Resource Perspective . . . 62
5.3.3 Inter-Case Perspective . . . 63
5.4 Drift Signals . . . 66
5.4.1 Intra-Case Perspective . . . 69
5.4.2 Resource Perspective . . . 70
5.4.3 Inter-Case Perspective . . . 71

6 Implementation (Drift Detector) **75**
6.1 Kairo . . . 75
6.1.1 Running the Pipeline in Python . . . 75
6.1.2 Agentic Analysis . . . 78
6.2 Dashboard . . . 78
6.2.1 Running the Pipeline in the Dashboard . . . 79
6.2.2 Kairo Copilot . . . 79

7 Evaluation **85**
7.1 Padella Benchmark . . . 86
7.2 Rheon Benchmark . . . 91
7.2.1 Single-Drift Logs . . . 93
7.2.2 Multi-Drift Logs . . . 94
7.2.3 Mixed-Drift Logs . . . 96

8 Discussion **99**
8.1 Research Questions . . . 99
8.2 Flexibility and Evaluation . . . 99
8.3 Choosing a Configuration . . . 101
8.4 LLM Support . . . 101
8.5 Runtime and Open Issues . . . 102

9 Conclusion **103**

List of Figures

1.1 A waiting-time drift that control-flow does not show 4
1.2 Drifts on two different time scales 6

2.1 Transition system with states for remaining time prediction 11

3.1 Example process tree 18
3.2 Example event log with calendar windows 19
3.3 The four drift modes 21
3.4 PCA for two correlated features 23
3.5 Three clustering algorithms in comparison 26

4.1 Workflow of the generation of an event log 36
4.2 Python call that generates the event log L_2 38
4.3 Process trees of the example log with a control-flow drift 39
4.4 Example log with a control-flow drift 40
4.5 Example log with a reassignment drift 42
4.6 Example log with an arrival rate drift 43

5.1 Overview of the state-based framework 46
5.2 Feature windows and signal windows 48
5.3 Intra-case feature groups and the drift types they respond to 51
5.4 Resource feature groups and the drift types they respond to 53
5.5 Inter-case feature groups and the drift types they respond to 56
5.6 PCA of the intra-case features of the running example 57
5.7 PCA of the resource features of the running example 58
5.8 Intra-case clustering of the running example 62
5.9 Intra-case states of the running example 63
5.10 Resource clustering of the running example 64
5.11 Inter-case clustering of the running example 65
5.12 Example of a drift signal 68
5.13 Intra-case drift signals of the running example 70
5.14 Resource drift signals of the running example 71
5.15 Inter-case drift signals of the running example 72

6.1 Two uses of an LLM on top of the framework 76
6.2 Workflow of kairo 77
6.3 Text abstraction of a drift signal 78
6.4 States page of the dashboard 80

6.5 Trajectory of the log in the dashboard 81
6.6 Drift signal page of the dashboard 82
6.7 Copilot page of the dashboard 83

7.1 Example of the evaluation of a drift signal 87
7.2 Clustering algorithms in all configurations 88
7.3 Padella drift types and our perspectives 89
7.4 Confusion matrices on the Padella benchmark 89
7.5 Detections around the turn of the year 2017/18 91
7.6 Case arrivals in the Bank account closure log 92
7.7 Confusion matrices on the Rheon logs 93

8.1 F1-scores of both runs on all sets of logs 100

List of Tables

3.1 Example event log table 16
3.2 Evaluation of the example clusterings 30
3.3 Divergences of the example distributions 31

4.1 Schema of a Rheon log 34
4.2 Excerpt of a Rheon log 34
4.3 Parameters of Rheon 35
4.4 The nine drift types of Rheon 37
4.5 Example logs per perspective 39

5.1 Drifts of the running example 47
5.2 Excerpt of the running example 47
5.3 Intra-case feature vectors of the running example 50
5.4 Resource feature vectors of the running example 52
5.5 Inter-case feature vectors of the running example 55
5.6 Evaluation of the chosen clusterings of the running example 61
5.7 Configuration of the running example 73

7.1 Padella base logs 88
7.2 Results on the Padella benchmark 90
7.3 Results on the single-drift Rheon logs 94
7.4 Results on the multi-drift Rheon logs 96
7.5 Results on the mixed-drift Rheon logs 98

1 Introduction

Process mining uses event logs that information systems record during process execution to discover, monitor and improve real processes [28]. In a traditional event log, each event belongs to exactly one case, and carries an activity and timestamp, usually also resource and case attributes. Such an event log captures a process over months or years, in which the processes change.

If a process changes while it is being analyzed, we call it a concept drift (van der Aalst [28]). A concept drift does not only concern the order of activities of a case but also the resources and the situation of all running cases. Process models and Key Performance Indicators (KPIs) over the entire event logs then mix up multiple process versions. Most detectors focus only on the control-flow (Sato et al. [26]) and miss drifts in other perspectives as in Figure 1.1. Moreover, in publicly available test data there is a lack of resource drifts and arrival rate drifts (Section 2.5).

In this thesis, we introduce a framework for monitoring a process through states in three perspectives. The intra-case states describe the history of a single case, the resource states the workload of the resources and the inter-case states the situation of all running cases. Our framework adapts the execution states of Berti et al. [7] from objects to cases and extends them to resource and inter-case perspectives through feature windows. For each perspective, our framework runs a pipeline that extracts features, reduces their dimensionality, clusters them into states and compares the state distributions over time in a drift signal. The input is an event log and the output the states as well as a drift signal per perspective. In comparison to detectors with control-flow focus, the framework is also capable of detecting drifts from resource and inter-case perspectives. The states are able to summarize multiple features and show what changed. Moreover, we developed the synthetic event log generator Rheon, the Python library kairo and a dashboard with an AI-assistant around our library.

We evaluate the framework based on two sets of logs with known drift points. The benchmark of Padella and Vinci [24] comprises 34 logs with 29 control-flow and time drifts that are injected into five real logs. The 45 Rheon logs contain 103 drifts from all three perspectives. With hand-tuned configurations from Run 2, the framework achieves an F1-score of 0.862 on the Padella benchmark and F1-scores from 0.907 to 0.933 in the three subsets of Rheon logs. These configurations let us show the potential of the framework in detecting drifts. The configurations of Run 1 by agents with our skill `kairo-drift-analyst` achieve, however, only F1-scores from 0.309 to 0.774 (Chapter 7). These configurations let us show the limitations of the framework (Section 8.2). Thus, the states are a useful and explainable indicator for concept drift, but their results strongly depend on the configurations.

In the remainder of this chapter, Section 1.1 motivates drift detection beyond the

control-flow and Section 1.2 describes the problem statement. Sections 1.3 and 1.4 name the research questions and research goals of this thesis, Sections 1.5 and 1.6 state the contributions and structure of the thesis.

1.1 Motivation

Played-out event logs do not change, but real processes do. Following van der Aalst [28], the situation in which a process is changing while being analyzed is called a concept drift. For example, two activities are concurrent at the start of an event log but then become sequential. According to van der Aalst [28], typical triggers of concept drift are seasonal effects, such as higher demand in December, and changed market conditions. Classical process mining techniques mostly assume that the process at the start and at the end of the log is the same [10]. If a drift occurs, process models or KPIs computed over the entire log mix two different processes. The process owner must therefore detect whether and when the process changed, and what changed.

A drift does not restrict itself to the order of activities. Processes run in a context, but process analysis sometimes ignores it. As described by van der Aalst [28], process context covers how cases influence each other like competing for the same resources. A single case is therefore not sufficient to explain the observed behavior alone. To explain the observed behavior and its changes, we analyze dependencies within cases, between cases, and with resources. For these dependencies, Senderovich et al. [27] introduce the terms intra-case and inter-case dependencies. This thesis adopts this terminology and considers a process from three perspectives:

- *Intra-case perspective:* The history of a single case up to a specific time, covering which activities happened in which order and how often. Drift example: A new trace variant dominates.
- *Resource perspective:* The working style of resources, such as their workload, service times, handovers, and the activities they perform. Drift example: A resource no longer exists and other resources take over the work.
- *Inter-case perspective:* The current situation of all running cases of the process, covering arrivals, waiting times, and case attributes. Drift example: The mean waiting time between consecutive events of a case increases.

From all three perspectives concept drifts can take place, but they can also influence each other. For example, a new trace variant with a frequent activity becomes dominant (intra-case perspective). The resource responsible for that activity then has more work to do (resource perspective), which affects the waiting times in the entire process (inter-case perspective).

We describe each drift by three properties:

- *Drift type:* What changes, e.g., the control-flow, the reassignment of an activity to another resource, or the waiting time between consecutive events.

- *Drift perspective:* The perspective to which the drift type belongs. For example, control-flow drifts belong to the intra-case perspective, reassignment drifts to the resource perspective, and waiting time drifts to the inter-case perspective.
- *Drift mode:* How the change takes effect over time, e.g., as a sudden, gradual, or recurring change.

In the literature, the terms drift type, drift mode, and drift perspective are used interchangeably. We reserve the term drift type for what changes, the term drift mode for how it changes, and the term drift perspective for the group a drift type belongs to.

Existing concept drift detection approaches cover this picture only partially:

- *Control-flow focus:* Most of the work on drift detection focuses on control-flow. Out of the 38 drift detection papers in the survey of Sato et al. [26], only six focus on the time perspective, three on the data perspective, and none on the resource perspective. In our framework, waiting times and case attributes fall under the inter-case perspective, and service times under the resource perspective. A concept drift that only changes the time or resource perspective stays undetected with control-flow approaches.
- *Single KPIs:* Approaches beyond control-flow often track single KPIs. Adams et al. [3] turn each KPI chosen by the user into a time series over time windows and detect drifts in each series separately. A drift therefore shows up as separate change points in several KPIs. This approach does not deliver a compact description of the overall process situation. For steady-state detection, Kraus et al. [18] show that combining complementary process characteristics is more accurate than relying on a single one.
- *Drift modes:* How a change takes effect over time is also important information for the process owner. Most of the existing approaches focus on sudden drifts only [26].
- *Missing test data:* Sato et al. [26] argue that real-world logs are unsuitable for measuring detection accuracy, since their drift points are unknown. They find only three public synthetic datasets with injected drifts. These datasets cover only the control-flow and time perspectives and do not mix different drift modes. None of them contains a resource drift or a change in case arrivals. A recent benchmark by Padella and Vinci [24] injects drifts into five real logs, but again only into the control-flow and activity durations.

Figure 1.1 shows this gap using a synthetic event log generated by Rheon. Rheon is our generator for synthetic event logs with injected concept drifts in all three perspectives (Chapter 4). This log has 300,299 cases with 1,946,540 events, seven activities, and seven resources. Its cases arrive between January 1 and December 31, 2020. On April 19, 2020, the mean waiting time between consecutive events of a case jumps from 14.2 minutes to 45 minutes. The activities, the control-flow, the resources, and the dominant

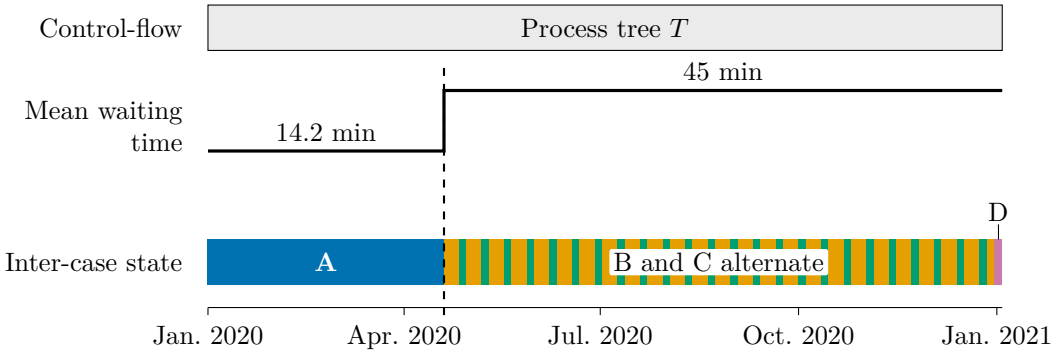


Figure 1.1: Visualization of the synthetic Rheon log generated with a mean waiting time drift. At the drift point (dashed), the mean waiting time between consecutive events of a case rises from 14.2 to 45 min, while all cases still follow process tree T. The inter-case state of each 12-hour window switches exactly there from A to an alternation of B and C. State D marks the run-out of the log after the last case arrivals. The alternation of B and C is drawn schematically.

resource of each activity remain unchanged. A detector that only compares relations between activities sees the same control-flow before and after the drift point and misses the change.

The state-based approach of this thesis aims to detect drifts in all three perspectives and to show what changed. Instead of separate change points per KPI, our states combine many features into one compact description. Rheon provides test logs with known drift points in all three perspectives. In Berti et al. [7], execution states from object-centric event logs are computed for objects. We adapt the approach from objects to cases in our case-centric framework and define our intra-case states. As an extension to their work, our approach also takes into account resource and inter-case perspectives and adapts the idea to them.

Overall, we compute states from an event log by extracting features, compressing, and clustering. The input of our approach is an event log, and the output consists of intra-case, resource, and inter-case states whose changes over time indicate drifts. In Figure 1.1 one can visually see the general idea of our approach for the inter-case perspective. We describe each 12-hour window by features such as the number of active cases, arrivals, completions, and the mean waiting time between consecutive events of a case. Following the execution states of Berti et al., a Principal Component Analysis (PCA) compresses these features. A 2×2 Self-Organizing Map (SOM) is trained on these compressed feature vectors. Each of its four cells is one state, and each window gets the state of its best-matching cell.

The pipeline is unsupervised and does not see the drift point or the drift type. As seen in Figure 1.1, all 219 windows before the drift point fall into state A. From the drift point until December 31, the windows alternate between states B and C. The last seven windows, in which the log runs out, form state D. The first principal component explains

51% of the variance and is dominated by the mean waiting time between consecutive events of a case. It therefore also shows what changed: after the drift point, more time passes between the events of a case.

1.2 Problem Statement

An event log records a process over a long period, and the process often changes within this period. Such a concept drift affects the intra-case, resource, or inter-case perspective, or several of them at once. A detector that only captures control-flow drifts therefore misses drifts that only affect the resource or inter-case perspective.

Van der Aalst [28] names three main challenges in dealing with concept drifts:

1. *Change point detection*: Did the process change? If yes, when did it change?
2. *Change localization and characterization*: What has changed?
3. *Change process discovery*: How to capture and predict second-order dynamics?

According to van der Aalst, a process is not static even when there is no drift (first-order dynamics). This makes it even harder to detect changes of the process itself (second-order dynamics). Most approaches compare sliding windows of events with statistical methods. To localize drift points exactly and detect them at different time scales, the window length must vary. Moreover, drifts at different time scales can overlap, as in Figure 1.2.

This thesis focuses on challenges 1 and 2. To characterize a drift, our framework determines its drift perspective, its drift mode, and what exactly changed.

In conclusion, this thesis focuses on the following problem:

- *Given:* An event log in which each event belongs to exactly one case. Each event carries a case identifier, an activity, and a timestamp, and optionally a resource and case attributes. Drift points, drift types, drift perspectives, and drift modes are unknown.
- *Objective:* Compute intra-case, resource, and inter-case states whose changes over time indicate drifts. From these changes, detect the drift points and determine for each drift its perspective, its mode, and what changed.
- *Evaluation:* We compare the detected drift points with known drift points in two sets of logs. First, the benchmark of Padella and Vinci [24] injects sudden and gradual control-flow and duration drifts into five real logs. Second, synthetic Rheon logs contain drifts in all three perspectives.
- *Scope of the Padella benchmark:* We did not use the recurring logs of the Padella benchmark and ignored them in this thesis.

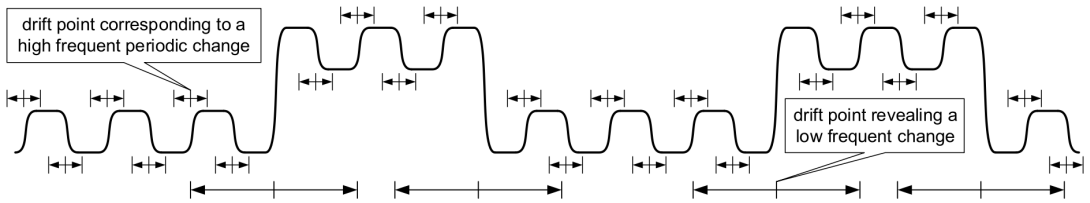


Figure 1.2: A periodically changing process with drifts at two different time scales, taken from van der Aalst [28]. Shorter sliding windows are used to detect short-term drifts. The left half of each sliding window is compared with the right half to spot statistically relevant changes. The larger sliding windows at the bottom are used for long-term drifts.

1.3 Research Questions

The main research question of this thesis is:

> *Can a process be monitored as a combination of intra-case, resource and inter-case states, and is this combination a useful indicator for a concept drift in traditional case-centric event logs?*

We divide our main research question into four further Research Questions (RQs):

RQ1 *States:* How can we compute execution states from a traditional event log in the intra-case, resource and inter-case perspectives, and how interpretable are these states in explaining concept drifts?

RQ2 *Detection:* Do the changes over time in those states signal drift points even if an event log has drifts in multiple perspectives?

RQ3 *Characterization:* Do the states indicate the perspective and the mode of a drift, and which features suit which drift types?

RQ4 *Applicability:* In which processes and event logs is the state-based drift detection approach more reliable and when does it deliver deceptive results?

RQ1 addresses the first part of the main research question, monitoring a process through states. RQ2 to RQ4 cover the second part of the main research question, the usage of the states as an indicator for concept drift detection.

1.4 Research Goals

In order to answer the research questions, we follow seven Research Goals (RGs). Each goal lists the research questions it addresses in parentheses.

RG1 *Terminology:* To clearly define drift type, drift mode and drift perspective (RQ3).

RG2 *Test data:* Develop a generator for synthetic event logs that injects concept drifts of various drift types in two drift modes (sudden and gradual). Along with each log, the generator writes a metadata file that lists the injected drifts (RQ2, RQ3, RQ4).

RG3 *Framework:* Develop a state-based framework for concept drift detection (RQ1, RQ2). It adapts execution states from Berti et al. [7] for objects to cases and extends them to resource and inter-case states.

RG4 *Library:* Develop a Python library with artificial intelligence features that implements our framework (RQ1). The library offers each result as a plot or text for Large Language Models (LLMs) and also provides LLM connections.

RG5 *Dashboard:* Develop a user-friendly dashboard around the library (RQ1, RQ3). In the dashboard, users compute and analyze results like the states and try different pipelines and configurations with the help of an LLM-based assistant, the Copilot.

RG6 *Evaluation:* Evaluate the robustness and flexibility of the framework against different processes, event logs and timespans (RQ2, RQ4)

RG7 *Limitations:* Reveal the limitations of the approach and potential misleading results especially when using the Copilot (RQ4)

1.5 Contributions

This thesis has six main contributions. Each contribution lists its research goals and chapters in parentheses.

- *Terminology:* A terminology defining and clearly separating drift types (what changes), drift modes (how it changes over time), and drift perspectives (which perspective the drift type belongs to) (RG1).

- *Rheon:* (RG2, Chapter 4). A library that generates synthetic event logs and injects nine drift types from all three perspectives, each suddenly or gradually. For each generated log, Rheon creates a metadata file comprehensively describing how the log was generated and a list of drifts together with their drift points, types and metadata.

- *State-based framework:* A framework that computes the intra-case, resource and inter-case states (RG3, Chapter 5). The framework consists of four steps: feature extraction, scaling and dimensionality reduction with PCA, clustering into states, and drift signals. For every signal window, the drift signal compares the distribution of the states with a reference using a divergence, which is the drift score of the signal window.

- *kairo:* A Python library that implements the framework (RG4, Section 6.1). With the help of its user-friendly documentation, users create a pipeline configuration in kairo and run it. The library supports various features addressing different drift types, various scaling techniques, PCA, and SOM, k-means and Density-Based Spatial Clustering of Applications with Noise (DBSCAN) for state clustering. It also computes state distributions for signal windows of different lengths, offers multiple divergences for the drift signal, and provides various plotting options for the different pipeline steps.
- *Dashboard:* A dashboard built around kairo (RG5, Section 6.2). In a user-friendly User Interface (UI), users can upload an event log, run the pipeline steps for each perspective and investigate the states in detail. With the LLM-based Copilot, users ask questions, gain insights about the results and request further steps.
- *Evaluation:* An evaluation on the five real event logs of Padella and Vinci [24] and on synthetic Rheon logs with different drift combinations from all perspectives (RG6, RG7, Chapters 7 and 8). The evaluation encompasses comprehensive comparisons between different hyperparameters like scaling techniques, feature and signal window lengths and feature selections.

1.6 Thesis Structure

The remainder of the thesis is structured as follows. Chapter 2 reviews the literature on concept drift detection and state-based process mining approaches and places this thesis within the literature. Chapter 3 introduces the notation for event logs, foundations of concept drifts, PCA, clustering and divergences. Chapter 4 presents the implementation of our log drift generator Rheon, which generates the synthetic event logs for the evaluation.

Chapter 5 defines intra-case, resource and inter-case states and answers RQ1. Each of its sections covers one step of the framework, i.e., feature extraction, scaling and dimensionality reduction, clustering into states, and drift signals, for all three perspectives. Chapter 6 describes the implementation of our drift detector, i.e., kairo and the dashboard tool, and introduces the AI-assisted analysis of the results with LLMs.

Chapter 7 answers RQ2 to RQ4 using the benchmark logs of Padella and Vinci [24] and the Rheon logs. Chapter 8 discusses the design decisions and limitations of the framework. Chapter 9 summarizes the results and discusses future directions and potential extensions of the topic.

2 Related Work

This chapter gives an overview of the literature on different concept drift detection approaches, state-based process mining and positions the thesis in the literature. Section 2.1 presents detectors that focus on detecting control-flow drifts, and Section 2.2 focuses on the works on detectors beyond control-flow. Section 2.3 covers clustering-based detection approaches. Section 2.4 describes the role of states in process mining and reviews the state-based literature. In Section 2.5 we focus on publicly available event logs with known drifts as test data for the evaluation. Finally, Section 2.6 positions this thesis in the literature by explaining similarities and differences of our framework.

2.1 Drift Detection in Control-Flow

The most common strategy is a statistical test that compares the behavior of neighboring windows. In the survey of Sato et al. [26] 11 out of 36 papers use a statistical test approach. Bose et al. [10] apply statistical tests on follows relationships features between activities. The tool ProDrift developed by Maaradji et al. [22] uses a χ^2 test on the distribution of partially ordered runs, changes the window size adaptively and detects gradual drifts alongside sudden drifts. ProDrift receives higher F1-scores and detects drifts earlier than the approach of Bose et al. [10].

Other detectors represent the control-flow differently. In Tsinghua Process Concept Drift Detection (TPCDD) Zheng et al. [29] derive candidate drift points from a matrix of traces and relations and cluster them together using Density-Based Spatial Clustering of Applications with Noise (DBSCAN). With an Error Tolerance (ET) of 10 traces around the actual drift point, TPCDD achieves an F1-score of 1.0, whereas ProDrift needs an ET of about 100 traces for a comparable score on the same test data. CV4CDD-4D [19] learns to detect concept drifts using event logs with known sudden, gradual, incremental, and recurring drifts in an image of window similarities. CV4CDD-4D achieves an F1-score of 0.86 and finds the drift points more granular than the other approaches evaluated. It is also more robust against noise.

Beyond drift detection, Kraus and van der Aa [20] distinguish drift localization (what changed) from drift characterization (how it changed over time) and address the latter. In our terms, these correspond to drift types and drift modes.

All the detectors consider only the control-flow and therefore just the intra-case perspective of this thesis. Drifts that only change the waiting times, such as the one in Figure 1.1, stay hidden.

2.2 Drift Detection Beyond Control-Flow

Several detectors cover further perspectives. Brockhoff et al. [11] compare the trace distribution of two windows using the Earth Mover's Distance (EMD) with regard to processing and sojourn times.

Three detectors detect drift points using the Pruned Exact Linear Time (PELT) algorithm on time series of features. Adams et al. [3] use control-flow, performance, data, resource, object features and test pairs of detected drifts for Granger causality. Klijn et al. [17] use features on resources and control-flow using an Event Knowledge Graph (EKG). CVDrift by Khraiwesh et al. [16] chooses the window size of each time series from its Coefficient of Variation (CV), with series for control-flow, time, and resources. On event logs with activity duration drifts CVDrift achieves F1-scores from 0.77 to 0.97.

PELT-based detectors observe a drift in many features but capture the inter-case perspective only through individual features such as arrival time or processing time.

2.3 Clustering-Based Drift Detection

Methodologically, the clustering-based detectors are the most similar to this thesis. Clustering-based detectors observe how the composition of clusters changes over time and newly emerging or vanishing clusters are a drift signal (Sato et al. [26]). Concept-Drift in Event Stream Framework (CDESF) developed by Barbon Junior et al. [6] clusters cases of an event stream using DBSCAN with respect to distances in control-flow and time. CDESF signals a drift when outlier cases become dense enough to form a new cluster. Version Clustering (Abarca Zúñiga et al. [1]) clusters windows according to their directly-follows frequencies into process versions using Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN). It signals drift points at the transitions between these versions. Using a tolerance of 5% of the cases, Version Clustering achieves F1-scores of 0.84 and 0.96 on two benchmarks.

Both approaches apply clustering in a feature space describing mostly the control-flow. Version Clustering (Abarca Zúñiga et al. [1]) applies clustering on full windows similar to resource and inter-case perspectives of this thesis, but in our thesis the states do not represent a process version.

2.4 States in Process Mining

Classically a state describes a running case for making predictions. van der Aalst [28] predicts remaining time of cases using annotated transition systems with states (Figure 2.1). Senderovich et al. [27] encode running cases for prediction in a state space from intra-case and inter-case dependencies.

Berti et al. [7] learn execution states of objects using sliding windows, Principal Component Analysis (PCA) and a Self-Organizing Map (SOM). In this thesis, we adapt the approach to cases and extend it to resource and inter-case perspectives beyond the

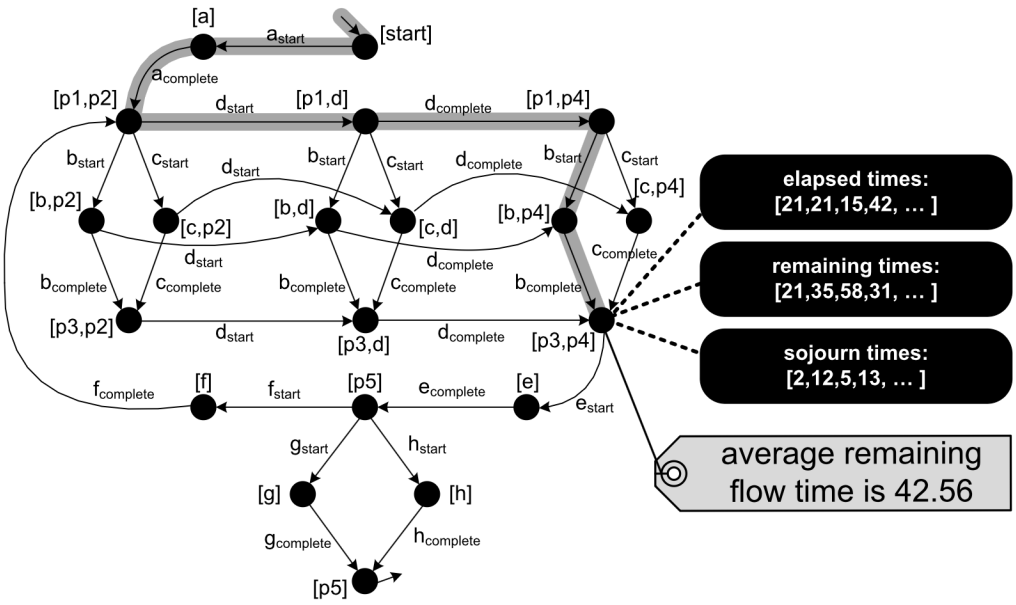


Figure 2.1: Transition system annotated with time measurements, taken from van der Aalst [28]. Each node is a state of a running case and each arc is the start or completion of an activity. Each state collects the elapsed, remaining, and sojourn times of all past cases that visited it, as shown for state [p3,p4]. The gray path is a running case that reaches [p3,p4]. Its predicted remaining flow time is 42.56, the average remaining time observed in this state.

intra-case perspective using calendar windows and feature extraction for concept drift detection.

Kraus et al. [18] detect steady states from Key Performance Indicators (KPIs) for each time window like the number of active cases. A fluctuating arrival rate is not seen as a concept drift by the paper whereas for this thesis it is a concept drift in the inter-case perspective.

2.5 Event Logs with Known Drifts

Adams et al. [2] compare eight detectors only on synthetic event logs with 132 control-flow drifts. None of the detectors performs well on all six evaluation goals, such as accuracy, latency, and robustness to noise. This kind of synthetic logs can be generated by the Concept Drift Log Generator (CDLG) [14]: It plays out process trees and injects drifts in the control-flow in four modes. The benchmark by Padella and Vinci [24], by contrast, has five real event logs, but with drifts only in control-flow and activity durations. None of these event logs has drifts in the resource perspective or arrival rates. Rheon follows the model of CDLG but adds multiple drift types from all three

perspectives supporting sudden and gradual drifts (Chapter 4).

2.6 Positioning of This Thesis

Three gaps motivate the approach of this thesis:

- *Perspectives:* Most detectors detect only control-flow drifts. Detectors for further perspectives recognize drift points in individual features or entire trace distributions but do not describe the process situation compactly.
- *States:* Clustering-based detectors mainly cluster control-flow behavior. The state-based works in Section 2.4 use states for prediction, object analysis, or steady-state detection, but not for concept drift detection.
- *Test data:* None of the aforementioned works and tools comprise resource drifts or arrival rate drifts.

This thesis addresses all three gaps. Our states describe a process in all three perspectives separately (research question RQ1), and we evaluate if their changes signal drifts (RQ2). Rheon delivers test data with injected drifts in all three perspectives (research goal RG2).

3 Preliminaries

In this chapter we introduce the notation and definitions that will be used in the next chapters. Section 3.1 formally defines event logs and traces, introduces process trees and defines calendar windows. Section 3.2 formalizes concept drifts with drift points and drift modes. Section 3.3 describes scaling of features and Principal Component Analysis (PCA). Section 3.4 presents the three clustering algorithms k-means, Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and Self-Organizing Maps (SOMs) used in our thesis. Section 3.5 defines divergences that compare two distributions.

3.1 Foundations

An event log captures how a process was executed. In the traditional case-centric approach, each event in an event log belongs to exactly one case, relates to exactly one activity and has a timestamp. Case identifier, activity and timestamp are traditionally the required columns in event logs for process mining. Additional attributes like resources, durations, event attributes as well as case attributes allow additional analysis. In Section 3.1.1, we introduce mathematical notation. Section 3.1.2 defines event logs, cases and traces, Section 3.1.3 introduces process trees, and Section 3.1.4 defines calendar windows.

3.1.1 Mathematical Preliminaries

Sets:

- A finite set $X = \{a_1, \ldots, a_n\}$ is a finite collection of distinct objects, and $|X| = n$ is its number of elements.
- $\mathbb{N}$ is the set of natural numbers, $\mathbb{Z}$ is the set of integers, $\mathbb{R}$ the set of real numbers and $\mathbb{N}_0 = \mathbb{N} \cup \{0\}$ is the set of natural numbers including 0.

Functions:

- A function f from a set X to a set Y maps each element of its domain $\text{dom}(f) \subseteq X$ to exactly one element of Y.
- $X \nrightarrow Y$ is the set of all functions from X to Y, which we call partial functions.
- $X \to Y$ is the set of all total functions from X to Y, i.e., of all $f \in X \nrightarrow Y$ with $\text{dom}(f) = X$.

- For a set Q, $f{\restriction_Q}$ is the restriction of f to Q with $\mathrm{dom}(f{\restriction_Q}) = \mathrm{dom}(f) \cap Q$ and $f{\restriction_Q}(x) = f(x)$ for all $x \in \mathrm{dom}(f{\restriction_Q})$.

Multisets:

- A multiset is a set where an element can occur multiple times.
- $\mathcal{B}(A) = A \to \mathbb{N}_0$ is the set of all multisets over a set A.
- For $X \in \mathcal{B}(A)$ and $a \in A$, $X(a)$ is the multiplicity of a in X.
- We write multisets as $X = [x^3, y^2, z]$, meaning $X(x) = 3$, $X(y) = 2$ and $X(z) = 1$.
- $[\,]$ is the empty multiset, and $|X| = \sum_{a \in A} X(a)$ is the size of a finite multiset X, i.e., its number of elements counted with their multiplicities.

Statistics:

- For a finite multiset $M = [x_1, \ldots, x_k]$ of real numbers with $k = |M|$, the mean and the sample standard deviation are

$$\mathit{mean}(M) = \frac{1}{k} \sum_{i=1}^{k} x_i \quad \text{and} \quad \mathit{std}(M) = \sqrt{\frac{1}{k-1} \sum_{i=1}^{k} (x_i - \mathit{mean}(M))^2}.$$

  We set $\mathit{mean}([\,]) = 0$, and $\mathit{std}(M) = 0$ for $k < 2$.
- For $p, q \in \mathbb{N}_0$, $\mathit{ratio}(p, q) = p/q$ for $q > 0$ and $\mathit{ratio}(p, 0) = 0$.

Sequences:

- A sequence is an ordered collection of objects in which an object can occur multiple times.
- X^* is the set of all finite sequences over a set X.
- For a sequence $\sigma = \langle a_1, \ldots, a_n \rangle \in X^*$, $\sigma(i) = a_i$ is its i-th element for $1 \leq i \leq n$.
- For $1 \leq k \leq n$, $\mathit{prefix}_k(\sigma) = \langle a_1, \ldots, a_k \rangle$ is the prefix of length k.
- $\partial_{\mathit{set}}(\sigma) = \{a_1, \ldots, a_n\}$ turns σ into a set.
- $\partial_{\mathit{multiset}}(\sigma) = [a_1, \ldots, a_n]$ turns σ into a multiset, e.g., $\partial_{\mathit{multiset}}(\langle a, b, b \rangle) = [a, b^2]$.
- $\mathit{df}(\sigma) = [(a_i, a_{i+1}) \mid 1 \leq i < n] \in \mathcal{B}(X \times X)$ is the multiset of directly-follows pairs of σ, e.g., $\mathit{df}(\langle a, b, b, c \rangle) = [(a, b), (b, b), (b, c)]$.

Vectors:

- We write a vector $\mathbf{x} \in \mathbb{R}^n$ as $\mathbf{x} = (x_1, \ldots, x_n)$ and treat it as a column vector, so $\mathbf{x}^\top$ is a row vector.

- $\mathbf{0}$ is the zero vector, and for $1 \leq i \leq p$, $onehot_p(i) \in \mathbb{R}^p$ has a 1 at position i and 0 elsewhere, e.g., $onehot_3(2) = (0, 1, 0)$.
- For $\mathbf{x} \in \mathbb{R}^p$ and $\mathbf{y} \in \mathbb{R}^q$, the concatenation $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{p+q}$ appends $\mathbf{y}$ to $\mathbf{x}$, e.g., $((1, 2), (3)) = (1, 2, 3)$.

3.1.2 Event Logs, Cases and Traces

We define event logs as a set of events using an attribute function. In this notation, each event is uniquely identifiable even if two events have the same attribute values.

Universes:

- $\mathcal{U}_{ev}$ is the universe of events.
- $\mathcal{U}_{case}$ is the universe of case identifiers, $\mathcal{U}_{act}$ is the universe of activities and $\mathcal{U}_{res}$ is the universe of resources.
- $\mathcal{U}_{time}$ is the universe of timestamps.
- $\mathcal{U}_{att}$ is the universe of attribute names with $\{case, act, time, res\} \subseteq \mathcal{U}_{att}$.
- $\mathcal{U}_{val}$ is the universe of attribute values with $\mathcal{U}_{case} \cup \mathcal{U}_{act} \cup \mathcal{U}_{time} \cup \mathcal{U}_{res} \subseteq \mathcal{U}_{val}$.
- $\mathcal{U}_{map} = \mathcal{U}_{att} \nrightarrow \mathcal{U}_{val}$ is the universe of attribute value mappings.

Definition 3.1.1 (Event Log)**.** An event log is a tuple $L = (E, \pi, \preceq)$ with

- a finite set of events $E \subseteq \mathcal{U}_{ev}$,
- an attribute function $\pi \in E \to \mathcal{U}_{map}$ such that for each event $e \in E$:
  - $\{case, act, time\} \subseteq \mathrm{dom}(\pi(e))$ with $\pi(e)(case) \in \mathcal{U}_{case}$, $\pi(e)(act) \in \mathcal{U}_{act}$ and $\pi(e)(time) \in \mathcal{U}_{time}$,
  - $res \in \mathrm{dom}(\pi(e)) \Rightarrow \pi(e)(res) \in \mathcal{U}_{res}$,
- a total order $\preceq$ on E that respects the timestamps, i.e., $e \preceq e'$ for all $e, e' \in E$ with $\pi(e)(time) < \pi(e')(time)$.

Furthermore:

- For $e \in E$ and $n \in \mathrm{dom}(\pi(e))$, $\pi_n(e) = \pi(e)(n)$ is the value of attribute n of e, e.g., $\pi_{act}(e)$ is the activity of e.
- An attribute name $n \in \mathcal{U}_{att} \setminus \{case, act, time, res\}$ is a case attribute of L if all events of a case have the same value for n, i.e., $\pi(e)|_{\{n\}} = \pi(e')|_{\{n\}}$ for all $e, e' \in E$ with $\pi_{case}(e) = \pi_{case}(e')$.

Definition 3.1.2 (Case, Trace and Prefix)**.** Let $L = (E, \pi, \preceq)$ be an event log.

- $C_L = \{\pi_{case}(e) \mid e \in E\}$ is the set of cases of L.

Table 3.1: Event log L_1 with three cases and eleven events. Each row is an event with its attributes, and the second header row gives the attribute names. The activities are a, b, c and d. The timestamp is the completion time of an event, and the duration is its processing time in minutes. The amount in euros is a case attribute.

Event	Case *case*	Activity *act*	Timestamp *time*	Resource *res*	Duration *duration*	Amount *amount*
e_1	1	a	2020-03-02 08:15	Anna	15	5000
e_2	1	b	2020-03-02 10:40	Ben	45	5000
e_3	1	c	2020-03-02 15:05	Clara	25	5000
e_4	1	d	2020-03-03 09:30	Anna	5	5000
e_5	2	a	2020-03-02 09:00	Anna	10	1200
e_6	2	c	2020-03-02 13:20	Clara	30	1200
e_7	2	d	2020-03-02 16:45	Ben	10	1200
e_8	3	a	2020-03-02 14:10	Anna	20	8000
e_9	3	b	2020-03-03 08:50	Ben	40	8000
e_{10}	3	b	2020-03-03 11:30	Ben	55	8000
e_{11}	3	c	2020-03-03 16:00	Clara	20	8000

- For a case $c \in C_L$, $E_L(c) = \{e \in E \mid \pi_{case}(e) = c\}$ is the set of its events.
- Let $e_1, \ldots, e_n$ be the events of a case $c \in C_L$ in the order $\preceq$, i.e., $\{e_1, \ldots, e_n\} = E_L(c)$, $n = |E_L(c)|$ and $e_i \preceq e_{i+1}$ for $1 \leq i < n$. Then
  - $\mathit{trace}_L(c) = \langle \pi_{act}(e_1), \ldots, \pi_{act}(e_n) \rangle \in \mathcal{U}_{act}^*$ is the trace of c,
  - $\mathit{start}_L(c) = \pi_{time}(e_1)$ is the start of c,
  - $\mathit{end}_L(c) = \pi_{time}(e_n)$ is the end of c and
  - $\mathit{prefix}_L(e_k) = \mathit{prefix}_k(\mathit{trace}_L(c))$ is the prefix of the event e_k for $1 \leq k \leq n$.

Example 3.1.3. Table 3.1 shows the event log $L_1 = (E, \pi, \preceq)$ with the events $E = \{e_1, \ldots, e_{11}\}$.

- $\mathcal{U}_{ev} = E = \{e_1, \ldots, e_{11}\}$.
- $\mathcal{U}_{case} = \{1, 2, 3\}$, $\mathcal{U}_{act} = \{a, b, c, d\}$ and $\mathcal{U}_{res} = \{\text{Anna}, \text{Ben}, \text{Clara}\}$.
- $\mathcal{U}_{att} = \{\mathit{case}, \mathit{act}, \mathit{time}, \mathit{res}, \mathit{duration}, \mathit{amount}\}$.
- $\mathcal{U}_{val} = \mathbb{R} \cup \{a, b, c, d, \text{Anna}, \text{Ben}, \text{Clara}\} \cup \mathcal{U}_{time} \supseteq \mathcal{U}_{case} \cup \mathcal{U}_{act} \cup \mathcal{U}_{time} \cup \mathcal{U}_{res}$. The case IDs, durations and amounts are real numbers and therefore also in $\mathcal{U}_{val}$.
- $\mathcal{U}_{map} = \mathcal{U}_{att} \rightharpoonup \mathcal{U}_{val}$ is the set of all partial functions from these six attribute names to values, e.g., $\{(\mathit{act}, b)\} \in \mathcal{U}_{map}$ and $\emptyset \in \mathcal{U}_{map}$.

- $\pi \in E \to \mathcal{U}_{map}$ maps each event to the attribute value mapping of its row in Table 3.1, e.g.,

$$\pi(e_9) = \{(case, 3), (act, b), (time, \text{2020-03-03 08:50}),\\ (res, \text{Ben}), (duration, 40), (amount, 8000)\}.$$

- For all attribute names $n \in \mathcal{U}_{att}$, the values of e_9 are $\pi_{case}(e_9) = 3$, $\pi_{act}(e_9) = b$, $\pi_{time}(e_9) = \text{2020-03-03 08:50}$, $\pi_{res}(e_9) = \text{Ben}$, $\pi_{duration}(e_9) = 40$ and $\pi_{amount}(e_9) = 8000$.

- *amount* is a case attribute, since all events of a case have the same amount, e.g., $\pi(e)\restriction_{\{amount\}} = \{(amount, 8000)\}$ for the four events e of case 3.

- *Cases and traces:* L_1 has the cases $C_{L_1} = \{1, 2, 3\}$. Case 3 has the events $e_8 \preceq e_9 \preceq e_{10} \preceq e_{11}$ and thus the trace $trace_{L_1}(3) = \langle a, b, b, c\rangle$. It starts at 2020-03-02 14:10 and ends at 2020-03-03 16:00. The prefix of e_{10} is $\sigma = prefix_{L_1}(e_{10}) = \langle a, b, b\rangle$ with $\partial_{multiset}(\sigma) = [a, b^2]$ and $df(\sigma) = [(a, b), (b, b)]$.

3.1.3 Process Trees

A process tree is a hierarchical process model whose leaves are activities and inner nodes operators (van der Aalst [28]). The children $Q_1, \ldots, Q_m$ are themselves also process trees.

- *Sequential composition:* $\to(Q_1, \ldots, Q_m)$ executes the children nodes step by step from left to right. E.g., $\to(a, b, c)$ produces only the trace $\langle a, b, c\rangle$.

- *Exclusive choice:* $\times(Q_1, \ldots, Q_m)$ executes exactly one child. E.g., $\times(a, b, c)$ produces the traces $\langle a\rangle$, $\langle b\rangle$ and $\langle c\rangle$.

- *Parallel composition:* $\wedge(Q_1, \ldots, Q_m)$ executes all children in any order. E.g., $\wedge(a, b, c)$ produces all six orderings of a, b and c.

- *Redo loop:* $\circlearrowleft(Q_1, \ldots, Q_m)$ executes first the Do part Q_1. Then, it stops or executes one of the Redo parts $Q_2, \ldots, Q_m$ and again Q_1. This can be repeated multiple times. E.g., $\circlearrowleft(a, b, c)$ produces the traces $\langle a\rangle$, $\langle a, b, a\rangle$, $\langle a, c, a\rangle$, $\langle a, b, a, c, a\rangle$ etc.

Figure 3.1 shows a nested process tree. It produces e.g., the traces $\langle a, b, d, e, g\rangle$ and $\langle a, d, c, e, f, b, d, e, h\rangle$.

3.1.4 Calendar Windows

We use calendar windows, i.e., time intervals of fixed length that start at calendar boundaries such as midnight. Thus, the window boundaries only depend on the origin and the window length, and not on the first event of the event log. To compute with timestamps, we treat them as real numbers, i.e., the minutes since a fixed point in time.

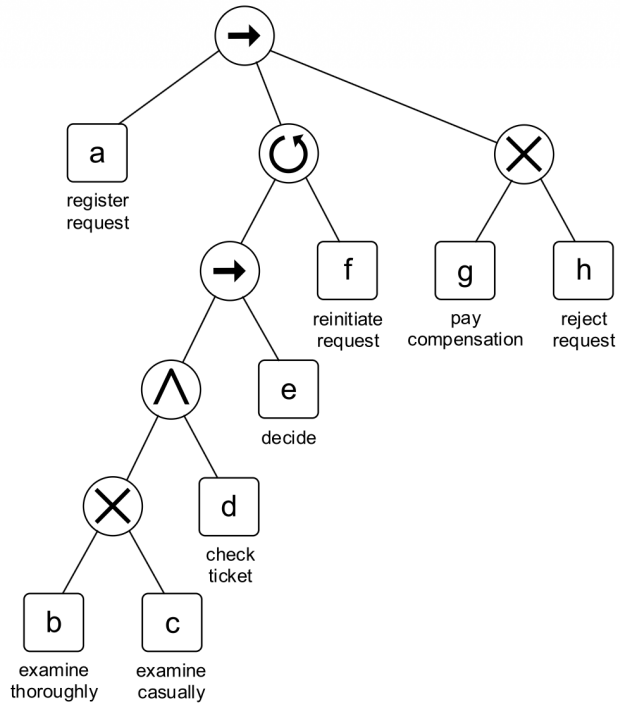


Figure 3.1: The process tree $\rightarrow(a, \circlearrowleft(\rightarrow(\wedge(\times(b, c), d), e), f), \times(g, h))$ showing different process tree operators. Taken from van der Aalst [28].

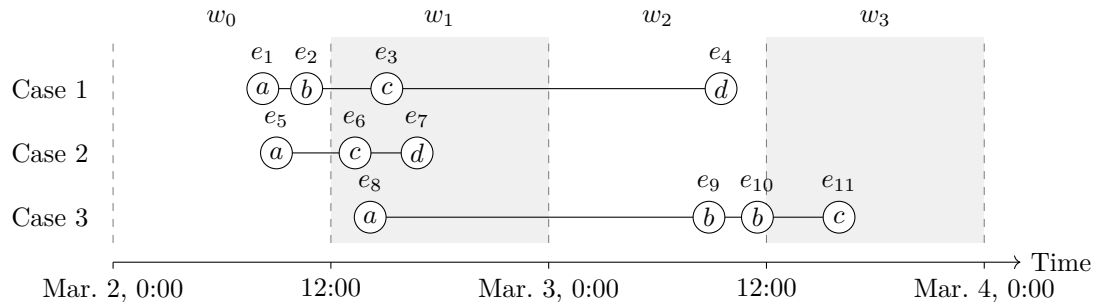


Figure 3.2: Event log L_1 from Table 3.1 on a time axis. Each row shows the events of a case in their order, and each circle is an event with its activity. Dashed lines separate the calendar windows w_0 to w_3 with the origin 2020-03-02 0:00 and the length $\Delta = 12$ hours. A case often spans several windows, and a window contains events of several cases.

Thus, time differences are in minutes. Chapter 5 uses calendar windows in two roles, as feature windows and as signal windows.

Definition 3.1.4 (Calendar Window)**.** Let $o \in \mathcal{U}_{time}$ be an origin and $\Delta \in \mathbb{R}$ with $\Delta > 0$ a window length.

- For $k \in \mathbb{Z}$, $w_k = [o + k\Delta, o + (k+1)\Delta)$ is the k-th calendar window.
- A timestamp $t \in \mathcal{U}_{time}$ lies in the window w_k if $t \in w_k$, i.e., $k = \lfloor (t - o)/\Delta \rfloor$.

Example 3.1.5. Figure 3.2 shows L_1 from Example 3.1.3 with the window length $\Delta = 12$ hours and the origin $o =$ 2020-03-02 0:00.

- For $k = 1$, the window is $w_1 =$ [2020-03-02 12:00, 2020-03-03 0:00).
- The timestamp $t =$ 2020-03-02 15:05 of e_3 lies in w_1.
- The timestamps of e_3, e_6, e_7 and e_8 lie in w_1, so w_1 contains events of all three cases.
- A case often spans several windows, e.g., the timestamps of case 3 lie in w_1, w_2 and w_3.

3.2 Concept Drift

In this section, we formalize concept drifts. The building block of a concept drift is the process change, which replaces one process version by another.

A concept drift is a situation in which a process changes. According to Kraus and van der Aa [20], an event log with a drift then contains data from different process versions. They describe a process version by the process model that governs its execution. We

use a broader notion. Besides the control-flow, a process version also determines which resources execute which activities, how long activities take and how long a case waits between two events. It also determines how many cases arrive and which values their case attributes take. The drift type describes in which of these properties two versions differ (Section 1.1).

Definition 3.2.1 (Process Change)**.** Let V be a set of process versions. A process change is a tuple $(v, v', t_s, t_e) \in V \times V \times \mathcal{U}_{time} \times \mathcal{U}_{time}$ with $v \neq v'$ and $t_s \leq t_e$ such that

- before t_s, the process behaves according to version v,
- from t_e on, the process behaves according to version v' and
- between t_s and t_e, the process changes gradually from v to v'.

t_s and t_e are the drift points of the change. The change is sudden if $t_s = t_e$, and gradual otherwise.

In a gradual change, both versions coexist for some time, and the old version is discontinued gradually. We assume that the share of behavior that follows v' grows linearly between t_s and t_e.

Definition 3.2.2 (Concept Drift, Drift Mode)**.** Let V be a set of process versions. A concept drift is a sequence

$$\delta = \langle (v_1, v_2, s_1, t_1), (v_2, v_3, s_2, t_2), \ldots, (v_m, v_{m+1}, s_m, t_m) \rangle$$

of $m \geq 1$ process changes with $t_i \leq s_{i+1}$ for $1 \leq i < m$. The drift points of δ are the drift points of its changes. The drift mode of δ is

- *sudden* if $m = 1$ and $s_1 = t_1$,
- *gradual* if $m = 1$ and $s_1 < t_1$,
- *recurring* if $m \geq 2$ and a version occurs more than once, i.e., $v_i = v_j$ for some $1 \leq i < j \leq m + 1$,
- *incremental* if $m \geq 2$, the versions $v_1, \ldots, v_{m+1}$ are pairwise different and each change alters the process only slightly, where all changes follow the same initiative.

The four drift modes originate from the taxonomy of Bose et al. [10], and Kraus and van der Aa [20] call them drift types. We call them drift modes instead and reserve the term drift type for what changes. In this thesis we will focus on sudden and gradual drifts. Figure 3.3 shows the four drift modes.

Example 3.2.3. The following drift illustrates Definitions 3.2.1 and 3.2.2.

- In the Rheon log of Figure 1.1, the mean waiting time between two consecutive events of a case rises from 14.2 to 45 minutes on April 19, 2020. This is a sudden drift $\langle (v_1, v_2, t_1, t_1) \rangle$ with $t_1 =$ April 19, 2020.
- The versions v_1 and v_2 only differ in the waiting time, so the drift type is a pure waiting time drift.

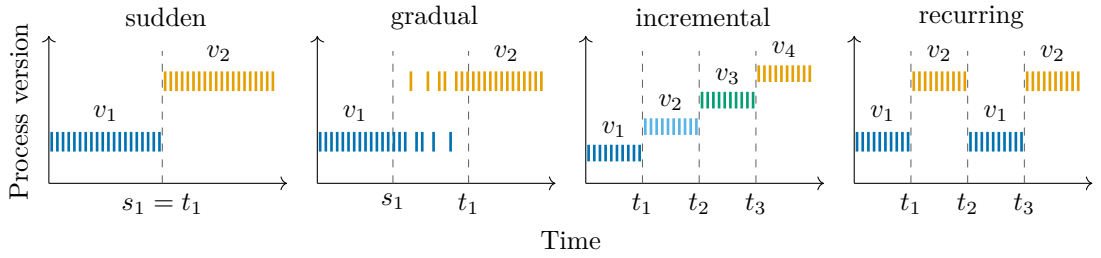


Figure 3.3: The four drift modes. Each tick is a case, and its height and color show the process version it follows. Dashed lines mark the drift points. A sudden change replaces v_1 by v_2 at one point in time. In a gradual change, more and more cases follow v_2 between s_1 and t_1. An incremental drift leads from v_1 to v_4 in small steps, and a recurring drift returns to earlier versions. The changes of the incremental and the recurring drift are sudden here, i.e., $s_i = t_i$.

3.3 Scaling and Principal Component Analysis

Each event or calendar window is described by numerical features, which we write as vectors.

Feature vectors:

- $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{R}^n$ is the feature vector of an event or a calendar window with its features $x_1, \ldots, x_n$.
- A feature matrix $X \in \mathbb{R}^{m \times n}$ contains m feature vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$ as its rows $\mathbf{x}_1^\top, \ldots, \mathbf{x}_m^\top$, and $x_{j,i}$ is the value of the i-th feature in the j-th vector.
- $\|\mathbf{x}\| = \sqrt{\sum_{i=1}^n x_i^2}$ is the Euclidean norm of $\mathbf{x}$, and $\|\mathbf{x}-\mathbf{y}\|$ is the Euclidean distance of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$.
- Further distances of $\mathbf{x}$ and $\mathbf{y}$ are the Manhattan distance $\sum_{i=1}^n |x_i - y_i|$, the Chebyshev distance $\max_{1 \le i \le n} |x_i - y_i|$ and, for $\mathbf{x}, \mathbf{y} \neq \mathbf{0}$, the cosine distance $1 - \mathbf{x}^\top \mathbf{y} / (\|\mathbf{x}\| \|\mathbf{y}\|)$.

Features may have different units and ranges, e.g., binary, one-hot, minutes, counts and shares. If their variances differ strongly, the variances determine the directions of the principal components more than the correlations. Therefore, each feature is usually brought to mean 0 and variance 1 before PCA. An alternative is the min-max normalization to the interval $[0, 1]$.

Definition 3.3.1 (Scaling)**.** Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with $m \geq 2$. For the i-th feature with the values $M_i = [x_{1,i}, \ldots, x_{m,i}]$, let

- $\mu_i = mean(M_i)$ be its mean,

- $s_i = std(M_i)$ its sample standard deviation and
- $\underline{x}_i = \min_{1 \le j \le m} x_{j,i}$ and $\overline{x}_i = \max_{1 \le j \le m} x_{j,i}$ its smallest and largest value.

The scaled matrix $X' \in \mathbb{R}^{m \times n}$ has the entries

- $x'_{j,i} = (x_{j,i} - \mu_i)/s_i$ for the *z-score standardization*, after which each non-constant feature has mean 0 and standard deviation 1, and
- $x'_{j,i} = (x_{j,i} - \underline{x}_i)/(\overline{x}_i - \underline{x}_i)$ for the *min-max normalization*, after which all values lie in $[0, 1]$.

For a constant feature, i.e., $s_i = 0$ or $\underline{x}_i = \overline{x}_i$, we set $x'_{j,i} = 0$ for $1 \le j \le m$.

PCA projects vectors onto a few directions in which they spread the most. It thus compresses many, often correlated features into a few components.

Definition 3.3.2 (Principal Component Analysis)**.** Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$.

- The mean vector $\boldsymbol{\mu} \in \mathbb{R}^n$ and the covariance matrix $\Sigma \in \mathbb{R}^{n \times n}$ of X are

$$\boldsymbol{\mu} = \frac{1}{m} \sum_{j=1}^{m} \mathbf{x}_j \quad \text{and} \quad \Sigma = \frac{1}{m} \sum_{j=1}^{m} (\mathbf{x}_j - \boldsymbol{\mu})(\mathbf{x}_j - \boldsymbol{\mu})^\top.$$

- Let $\lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_n \ge 0$ be the eigenvalues of Σ and $\mathbf{v}_1, \dots, \mathbf{v}_n$ corresponding orthonormal eigenvectors, i.e., $\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i$. Then $\mathbf{v}_i$ is the i-th principal component.
- For $1 \le d \le n$, let $W_d = [\mathbf{v}_1, \dots, \mathbf{v}_d] \in \mathbb{R}^{n \times d}$. The projection of a vector $\mathbf{x} \in \mathbb{R}^n$ onto the first d principal components is $\mathbf{z} = W_d^\top (\mathbf{x} - \boldsymbol{\mu}) \in \mathbb{R}^d$, and its reconstruction is $\hat{\mathbf{x}} = \boldsymbol{\mu} + W_d \mathbf{z}$.
- The explained variance ratio of the first d principal components and the reconstruction error are

$$evr_X(d) = \frac{\sum_{i=1}^{d} \lambda_i}{\sum_{i=1}^{n} \lambda_i} \quad \text{and} \quad err_X(d) = \frac{1}{m} \sum_{j=1}^{m} \|\mathbf{x}_j - \hat{\mathbf{x}}_j\|^2,$$

  where $\hat{\mathbf{x}}_j$ is the reconstruction of $\mathbf{x}_j$, and $evr_X(d)$ is only defined if $\sum_{i=1}^{n} \lambda_i > 0$.

Properties:

- The first principal component is the direction in which the projected vectors have the largest variance. Each further component maximizes the variance among all directions that are orthogonal to the previous ones, and the variance along $\mathbf{v}_i$ is λ_i (Alpaydin [4]).

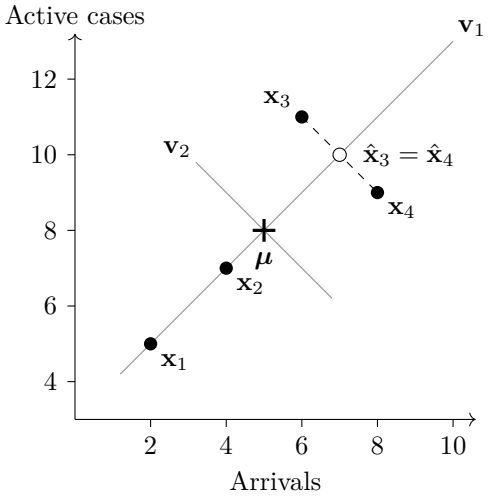


Figure 3.4: PCA for the four feature vectors of Example 3.3.3. The gray lines are the principal components through the mean $\boldsymbol{\mu}$ (cross). The variance along $\mathbf{v}_1$ is $\lambda_1 = 9$, along $\mathbf{v}_2$ only $\lambda_2 = 1$. Thus, the projection onto $\mathbf{v}_1$ keeps 90% of the variance. The reconstructions of $\mathbf{x}_1$ and $\mathbf{x}_2$ are exact. $\mathbf{x}_3$ and $\mathbf{x}_4$ are both reconstructed as $(7, 10)$ (empty circle), and the dashed lines are their reconstruction errors.

- Among all orthogonal linear projections onto d dimensions, PCA has the smallest reconstruction error (Alpaydin [4]).
- We choose d at the elbow of $err_X(d)$ over d, i.e., where a further component hardly lowers the reconstruction error.

Example 3.3.3. Four calendar windows are described by their number of arrivals and their number of active cases.

- The feature vectors $\mathbf{x}_1 = (2, 5)$, $\mathbf{x}_2 = (4, 7)$, $\mathbf{x}_3 = (6, 11)$ and $\mathbf{x}_4 = (8, 9)$ have the mean vector and the covariance matrix

$$\boldsymbol{\mu} = \begin{pmatrix} 5 \\ 8 \end{pmatrix} \quad \text{and} \quad \Sigma = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}.$$

- Both features have the same variance, so scaling does not change the principal components.
- Σ has the eigenvalues $\lambda_1 = 9$ and $\lambda_2 = 1$ with the eigenvectors $\mathbf{v}_1 = (1, 1)/\sqrt{2}$ and $\mathbf{v}_2 = (-1, 1)/\sqrt{2}$.
- The first principal component explains $evr_X(1) = 9/10$ of the variance, since windows with many arrivals also have many active cases.

- The projections onto $\mathbf{v}_1$ are $-3\sqrt{2}$, $-\sqrt{2}$, $2\sqrt{2}$ and $2\sqrt{2}$.
- The reconstructions of $\mathbf{x}_1$ and $\mathbf{x}_2$ are exact. $\mathbf{x}_3$ and $\mathbf{x}_4$ are both reconstructed as $(7, 10)$ with the squared error 2, so $err_X(1) = (0 + 0 + 2 + 2)/4 = 1 = \lambda_2$ (Figure 3.4).

3.4 Clustering

A clustering algorithm groups vectors such that vectors in the same group are similar to each other. Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \ldots, \mathbf{x}_m^\top$. A clustering of X is a function $cl \in \{1, \ldots, m\} \to K$ that assigns each vector a cluster from a set K. The following three algorithms form clusters in different ways:

- *k-means* forms a given number of clusters around centers (Section 3.4.1).
- A *self-organizing map* arranges prototypes on a grid whose cells are the clusters (Section 3.4.2).
- *DBSCAN* merges dense regions into clusters and marks the vectors outside them as noise (Section 3.4.3).

Figure 3.5 applies all three algorithms to the same vectors. Section 3.4.4 introduces metrics that evaluate the resulting clusterings.

3.4.1 k-Means

k-means searches k centers such that the squared distances of the vectors to their nearest center are as small as possible.

Definition 3.4.1 (k-Means)**.** Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \ldots, \mathbf{x}_m^\top$ and $k \in \mathbb{N}$ with $k \leq m$. A k-means clustering of X consists of centers $\mathbf{c}_1, \ldots, \mathbf{c}_k \in \mathbb{R}^n$ and a clustering $cl \in \{1, \ldots, m\} \to \{1, \ldots, k\}$ such that

- each vector belongs to its nearest center, i.e., $cl(j) \in \arg\min_{1 \leq l \leq k} \|\mathbf{x}_j - \mathbf{c}_l\|$ for $1 \leq j \leq m$, and
- each center is the mean of its vectors, i.e., for $1 \leq l \leq k$,

$$\mathbf{c}_l = \frac{1}{|J_l|} \sum_{j \in J_l} \mathbf{x}_j \quad \text{with} \quad J_l = \{j \mid cl(j) = l\} \neq \emptyset.$$

k-means finds such a clustering by starting with k randomly chosen vectors as centers. It then establishes both conditions alternately until the assignment does not change anymore.

Properties:

- k-means needs the number k of clusters in advance.

- Outliers always belong to a cluster.
- The result depends on the initial centers.

3.4.2 Self-Organizing Maps

A SOM is a neural network whose neurons lie on a usually two-dimensional grid. The model goes back to Kohonen [15]. We call a neuron a cell of the grid. Each cell has a prototype in the feature space, and a vector belongs to the cell with the nearest prototype. During training, the cells compete for each vector, the winning cell pulls its neighbors along, and the prototypes adapt to the vector.

Definition 3.4.2 (Self-Organizing Map)**.** Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \ldots, \mathbf{x}_m^\top$ and $G = \{1, \ldots, p\} \times \{1, \ldots, q\}$ a grid with $p, q \in \mathbb{N}$.

- A SOM assigns each cell $\mathbf{r} \in G$ a prototype $\mathbf{w}_\mathbf{r}(t) \in \mathbb{R}^n$ at time $t \in \mathbb{N}_0$.
- The Best-Matching Unit (BMU) of a vector $\mathbf{x} \in \mathbb{R}^n$ at time t is the cell $b_t(\mathbf{x}) = \arg\min_{\mathbf{r} \in G} \|\mathbf{x} - \mathbf{w}_\mathbf{r}(t)\|$.
- The initial prototypes $\mathbf{w}_\mathbf{r}(0)$ are, e.g., randomly chosen vectors of X.
- In each step $t = 0, 1, \ldots, T-1$, a vector $\mathbf{x}(t) \in \{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ is chosen, and all prototypes move towards it:

$$\mathbf{w}_\mathbf{r}(t+1) = \mathbf{w}_\mathbf{r}(t) + \eta(t)\, h_{b_t(\mathbf{x}(t)),\mathbf{r}}(t)\, \big(\mathbf{x}(t) - \mathbf{w}_\mathbf{r}(t)\big) \quad \text{for all } \mathbf{r} \in G.$$

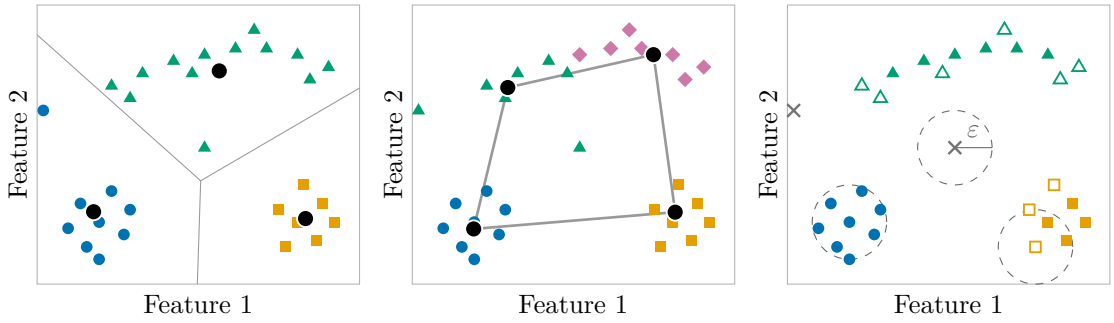
- The neighborhood function is the Gaussian kernel

$$h_{\mathbf{b},\mathbf{r}}(t) = \exp\!\big(-\|\mathbf{b} - \mathbf{r}\|^2 / (2\sigma(t)^2)\big).$$

  It decreases with the grid distance of the cells $\mathbf{b}$ and $\mathbf{r}$.
- The learning rate $\eta(t) > 0$ and the neighborhood radius $\sigma(t) > 0$ decrease monotonically in t.
- After T steps, the SOM yields the clustering $cl(j) = b_T(\mathbf{x}_j)$ with $K = G$.

Properties:

- The neighborhood function also pulls the neighbors of the BMU towards the vector. Thus, neighboring cells get similar prototypes, and vectors that are close in the feature space fall into close cells.
- Towards the end of the training, $\sigma(t)$ is so small that almost only the BMU adapts.
- The SOM forms at most $p \cdot q$ clusters.
- As in k-means, every vector belongs to a cluster.



(a) k-means, $k = 3$ (b) SOM, 2×2 grid (c) DBSCAN, $\varepsilon = 0.6$, $minPts = 4$

Figure 3.5: The three clustering algorithms of Example 3.4.4 on the same 29 vectors with three groups and two outliers. Color and shape of a symbol show its cluster, and black dots are the centers or prototypes. (a) k-means divides the plane into convex cells (gray lines) and also assigns the outliers to a cluster. (b) The gray line connects the prototypes of neighboring cells: $(1, 1)$ bottom left, $(1, 2)$ bottom right, $(2, 2)$ top right and $(2, 1)$ top left. The elongated group is split into the cells $(2, 1)$ and $(2, 2)$. (c) DBSCAN detects the outliers as noise (gray crosses). Filled symbols are core points and empty symbols border points. The dashed circles show the ε-neighborhood of a core point, a border point and an outlier.

- Similarly to k-means, the grid size must be specified in advance.
- Instead of the Euclidean distance, the BMU can be determined with another distance, e.g., the Manhattan, Chebyshev or cosine distance. The update of the prototypes stays the same.

3.4.3 DBSCAN

DBSCAN forms clusters from regions in which vectors lie densely (Ester et al. [13]). A region is dense if at least *minPts* vectors lie within the radius ε of a vector. Vectors in sparse regions belong to no cluster and count as noise.

Definition 3.4.3 (DBSCAN)**.** Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \ldots, \mathbf{x}_m^\top$, $I = \{1, \ldots, m\}$, $\varepsilon \in \mathbb{R}$ with $\varepsilon > 0$ and $minPts \in \mathbb{N}$.

- $N_\varepsilon(j) = \{l \in I \mid \|\mathbf{x}_j - \mathbf{x}_l\| \leq \varepsilon\}$ is the ε-neighborhood of $j \in I$. It contains j itself.
- $j \in I$ is a core point if $|N_\varepsilon(j)| \geq minPts$.
- $l \in I$ is directly density-reachable from $j \in I$ if j is a core point and $l \in N_\varepsilon(j)$. l is density-reachable from j if there are $i_1, \ldots, i_r \in I$ with $i_1 = j$ and $i_r = l$ such that i_{h+1} is directly density-reachable from i_h for $1 \leq h < r$.

- A cluster is a non-empty set $C \subseteq I$ that is maximal and connected. Maximal means that $l \in C$ if $j \in C$ and l is density-reachable from j. Connected means that for all $j, l \in C$, there is an $i \in I$ from which both j and l are density-reachable.
- The points of a cluster that are not core points are border points. Points that belong to no cluster are noise.
- For the clusters $C_1, \ldots, C_k$, the clustering is $cl(j) = l$ for $j \in C_l$ and $cl(j) = \mathit{noise}$ if j is noise. Thus, $K = \{1, \ldots, k, \mathit{noise}\}$.

Properties:

- Unlike in k-means and the SOM, the number of clusters follows from the data.
- Any distance can replace the Euclidean distance in the ε-neighborhood [13], e.g., the Manhattan, Chebyshev or cosine distance.
- A common approach to choose ε is the knee of the sorted k-dist graph with $k = \mathit{minPts}$, where the graph starts to rise steeply. This graph shows the distance of each point to its k-th nearest neighbor in ascending order.

Example 3.4.4. Figure 3.5 shows 29 vectors in $\mathbb{R}^2$: three groups, one of them elongated, and two outliers.

- k-means with $k = 3$ finds the three groups, but also assigns both outliers to a cluster.
- The SOM with a 2×2 grid has four cells for three groups. Therefore, it splits the elongated group into two neighboring cells.
- DBSCAN with $\varepsilon = 0.6$ and $\mathit{minPts} = 4$ finds exactly the three groups and detects both outliers as noise.

3.4.4 Clustering Evaluation

The true groups of the vectors are unknown. Thus, a clustering can only be evaluated with the vectors themselves, e.g., by how compact and how well separated its clusters are. We use five such metrics. The silhouette and the Caliński–Harabasz index evaluate the clusterings of all three algorithms. The quantization error and the occupancy entropy evaluate SOMs, and Density-Based Clustering Validation (DBCV) evaluates DBSCAN. For SOMs and DBSCAN, the silhouette, the quantization error and DBCV use the same distance as the clustering. The Caliński–Harabasz index always uses the Euclidean distance.

Clusters:

- The clusters of a clustering cl of a feature matrix $X \in \mathbb{R}^{m \times n}$ are the non-empty sets $\{j \mid cl(j) = \kappa\}$ with $\kappa \in K \setminus \{\mathit{noise}\}$. We number them $C_1, \ldots, C_k$.

- Noise belongs to no cluster, and $I' = C_1 \cup \cdots \cup C_k$ contains the $m' = |I'|$ clustered vectors.

The silhouette compares the distance of a vector to its own cluster with its distance to the nearest other cluster [25]. The Caliński–Harabasz index compares the dispersion between the clusters with the dispersion within them [12].

Definition 3.4.5 (Silhouette)**.** Let cl be a clustering of a feature matrix $X \in \mathbb{R}^{m \times n}$ with $k \geq 2$ clusters and $m' > k$. For $j \in C_l$, let

$$a(j) = \frac{1}{|C_l| - 1} \sum_{i \in C_l \setminus \{j\}} \|\mathbf{x}_j - \mathbf{x}_i\| \quad \text{and} \quad b(j) = \min_{l' \neq l} \frac{1}{|C_{l'}|} \sum_{i \in C_{l'}} \|\mathbf{x}_j - \mathbf{x}_i\|$$

be the mean distances of $\mathbf{x}_j$ to the other vectors of its cluster and to the vectors of the nearest other cluster.

- The silhouette of j is

$$s(j) = \begin{cases} \dfrac{b(j) - a(j)}{\max\{a(j), b(j)\}} & \text{if } |C_l| > 1, \\ 0 & \text{if } |C_l| = 1. \end{cases}$$

- $sil(cl) = \frac{1}{m'} \sum_{j \in I'} s(j)$ is the silhouette of cl.

Definition 3.4.6 (Caliński–Harabasz Index)**.** Let cl be a clustering of a feature matrix $X \in \mathbb{R}^{m \times n}$ with $k \geq 2$ clusters and $m' > k$. Let $\boldsymbol{\mu}'$ be the mean of the clustered vectors $\mathbf{x}_j$ with $j \in I'$, and $\mathbf{c}_l$ the mean of the vectors of C_l.

- $D_b = \sum_{l=1}^{k} |C_l| \|\mathbf{c}_l - \boldsymbol{\mu}'\|^2$ is the dispersion between the clusters.
- $D_w = \sum_{l=1}^{k} \sum_{j \in C_l} \|\mathbf{x}_j - \mathbf{c}_l\|^2$ is the dispersion within the clusters.
- For $D_w > 0$, the Caliński–Harabasz index of cl is

$$ch(cl) = \frac{D_b/(k-1)}{D_w/(m'-k)}.$$

Properties:

- $s(j) > 0$ if $\mathbf{x}_j$ lies closer to its own cluster than to the nearest other one. A silhouette close to 1 shows that $\mathbf{x}_j$ lies well within its cluster, and one close to 0 that it lies between two clusters.
- $-1 \leq sil(cl) \leq 1$, and Rousseeuw [25] uses the mean silhouette to choose the number of clusters.
- $ch(cl)$ is large if the clusters are compact and lie far apart. It has no upper bound and depends on the scale of the features. Thus, it only compares clusterings of the same vectors.

For SOMs, we also measure how closely the prototypes fit the vectors and how evenly the vectors spread over the grid (Berti et al. [7]).

Definition 3.4.7 (Quantization Error, Occupancy Entropy)**.** Let cl be the clustering of a SOM after T steps (Definition 3.4.2) with the prototypes $\mathbf{w_r} = \mathbf{w_r}(T)$ on a grid G with $p \cdot q \geq 2$ cells.

- $qe(cl) = \frac{1}{m} \sum_{j=1}^{m} \|\mathbf{x}_j - \mathbf{w}_{cl(j)}\|$ is the quantization error, i.e., the mean distance of the vectors to the prototypes of their cells.
- $occ(\mathbf{r}) = |\{j \mid cl(j) = \mathbf{r}\}|/m$ is the occupancy of a cell $\mathbf{r} \in G$, i.e., the share of the vectors in $\mathbf{r}$.
- The occupancy entropy of cl is

$$ent(cl) = -\frac{1}{\log(p \cdot q)} \sum_{\mathbf{r} \in G,\, occ(\mathbf{r}) > 0} occ(\mathbf{r}) \log occ(\mathbf{r}).$$

Properties:

- A small quantization error means that the prototypes lie close to their vectors.
- The occupancy entropy is 0 if all vectors lie in one cell and 1 if all cells hold equally many vectors.

Moulavi et al. [23] designed DBCV to compare the density within the clusters with the density between them. In a good clustering, even the sparsest region inside a cluster is denser than the region that separates it from the other clusters.

Definition 3.4.8 (Density-Based Clustering Validation)**.** Let cl be a DBSCAN clustering of a feature matrix $X \in \mathbb{R}^{m \times n}$ with $k \geq 2$ clusters.

- The density sparseness $dsc(C_l)$ is the largest gap that the vectors of C_l have to bridge to connect each other, i.e., the sparsest region inside C_l.
- The density separation $sep(C_l)$ is the smallest gap between C_l and another cluster, i.e., the densest region between them.
- The validity of C_l is

$$v(C_l) = \frac{sep(C_l) - dsc(C_l)}{\max\{sep(C_l), dsc(C_l)\}}.$$

- $dbcv(cl) = \sum_{l=1}^{k} \frac{|C_l|}{m} v(C_l)$ is the DBCV of cl, where m also counts the noise.

Properties:

- Both gaps are taken from a minimum spanning tree of each cluster, whose edge weights grow in sparse regions. The tree also captures the shape of the cluster, so DBCV evaluates clusters of arbitrary shape [23].

- $v(C_l) > 0$ if C_l is denser inside than towards the nearest other cluster. The DBCV lies between -1 and 1 [23].
- Noise lowers the DBCV, since it counts in m but belongs to no cluster [23].

Table 3.2: Metrics of the three clusterings in Figure 3.5.

Clustering	*sil*	*ch*	*qe*	*ent*	*dbcv*
k-means, $k = 3$	0.573	45.4	–	–	–
SOM, 2×2 grid	0.585	65.5	0.575	0.999	–
DBSCAN, $\varepsilon = 0.6$, $minPts = 4$	0.627	52.7	–	–	0.684

Example 3.4.9. Table 3.2 evaluates the clusterings of Example 3.4.4.

- DBSCAN has the highest silhouette, since it leaves out the two outliers. In k-means, the outlier $(2.7, 2.2)$ only has the silhouette 0.125.
- The elongated group has the lowest mean silhouette in all three clusterings.
- The SOM has the highest Caliński–Harabasz index, although it splits the elongated group.
- The SOM uses its four cells almost evenly with 8, 7, 7 and 7 vectors, so its occupancy entropy is close to 1. Its quantization error is the mean distance of the vectors to the black dots in Figure 3.5b.
- DBSCAN has the DBCV 0.684. Since the two noise vectors count in $m = 29$, the DBCV is at most $27/29 \approx 0.931$ here.

3.5 Divergences

A divergence measures how much two probability distributions over the same finite set differ. We use four divergences. In Chapter 5, we compute the drift signals with them.

Definition 3.5.1 (Divergences)**.** Let K be a finite set, and let $p, q \in K \to [0, 1]$ be distributions over K, i.e., $\sum_{\kappa \in K} p(\kappa) = \sum_{\kappa \in K} q(\kappa) = 1$. Let log be the natural logarithm.

- *Kullback-Leibler divergence:* If $q(\kappa) > 0$ for all $\kappa \in K$ with $p(\kappa) > 0$, then

$$D_{\mathrm{KL}}(p, q) = \sum_{\kappa \in K,\, p(\kappa) > 0} p(\kappa) \log \frac{p(\kappa)}{q(\kappa)},$$

  and otherwise $D_{\mathrm{KL}}(p, q) = \infty$.

Table 3.3: The four divergences in both directions for the distributions $p = (0.5, 0.5, 0)$ and $q = (0.25, 0.25, 0.5)$ over $K = \{1, 2, 3\}$ of Example 3.5.2. Since $p(3) = 0 < q(3)$, only $D_{\mathrm{KL}}(q, p)$ is infinite.

Divergence	$D(p, q)$	$D(q, p)$
D_{KL}	0.693	∞
D_{JS}	0.216	0.216
D_{TV}	0.500	0.500
D_{H}	0.541	0.541

- *Jensen-Shannon divergence:* $D_{\mathrm{JS}}(p, q) = \frac{1}{2} D_{\mathrm{KL}}(p, m) + \frac{1}{2} D_{\mathrm{KL}}(q, m)$ with the distribution $m(\kappa) = \frac{1}{2}(p(\kappa) + q(\kappa))$ for $\kappa \in K$.
- *Total variation distance:* $D_{\mathrm{TV}}(p, q) = \frac{1}{2} \sum_{\kappa \in K} |p(\kappa) - q(\kappa)|$.
- *Hellinger distance:* $D_{\mathrm{H}}(p, q) = \sqrt{\frac{1}{2} \sum_{\kappa \in K} \left(\sqrt{p(\kappa)} - \sqrt{q(\kappa)}\right)^2}$.

Properties:

- All four divergences are non-negative and 0 for $p = q$ [15, 21].
- D_{KL} is not symmetric and has no upper bound [21]. It weights each $\kappa \in K$ by its share $p(\kappa)$. If $q(\kappa) = 0$ for a κ with $p(\kappa) > 0$, $D_{\mathrm{KL}}(p, q)$ is infinite.
- D_{JS} is symmetric, defined for all distributions and at most $\log 2 \approx 0.69$ [21].
- D_{TV} and D_{H} are symmetric and lie in $[0, 1]$. D_{TV} is the share that has to move between the elements of K to turn q into p.

Example 3.5.2. We compare two distributions over $K = \{1, 2, 3\}$ (Table 3.3). We write a distribution p over K as $(p(1), p(2), p(3))$.

- The distributions are $p = (0.5, 0.5, 0)$ and $q = (0.25, 0.25, 0.5)$. Thus, only q gives the element 3 a share.
- $D_{\mathrm{KL}}(p, q) = 2 \cdot 0.5 \log \frac{0.5}{0.25} = \log 2 \approx 0.693$. Conversely, $D_{\mathrm{KL}}(q, p) = \infty$, since $q(3) = 0.5$ and $p(3) = 0$.
- With $m = (0.375, 0.375, 0.25)$, $D_{\mathrm{JS}}(p, q) = \frac{1}{2} D_{\mathrm{KL}}(p, m) + \frac{1}{2} D_{\mathrm{KL}}(q, m) \approx \frac{1}{2} \cdot 0.288 + \frac{1}{2} \cdot 0.144 \approx 0.216$.
- $D_{\mathrm{TV}}(p, q) = \frac{1}{2}(0.25 + 0.25 + 0.5) = 0.5$, i.e., a share of 0.5 moves from the element 3 to the elements 1 and 2. $D_{\mathrm{H}}(p, q) \approx 0.541$.
- D_{JS}, D_{TV} and D_{H} are finite and the same in both directions, whereas D_{KL} depends on the direction.

4 Implementation (Log Drift Generator)

To evaluate whether a detector detects a drift, we need event logs with known drifts. Public event logs with known drifts are hard to find and mostly contain drifts in the control-flow perspective or timing attributes. The Concept Drift Log Generator (CDLG) developed by Grimm et al. [14] generates synthetic event logs with control-flow drifts and the authors mention multi-perspective drifts as future work.

We introduce Rheon in this chapter in order to fill this gap. Rheon is a Python library developed in this thesis that generates synthetic event logs with injected concept drifts. Rheon is named after the Greek word for *flowing* coming from *panta rhei* (*everything flows*). Its source code[1] and documentation[2] are publicly available. It allows users to configure the generation parameters comprehensively and to inject concept drifts from intra-case, resource and inter-case perspectives. The input comprises parameters of a base process model and a list of drifts. The output is an event log with activities, timestamps, resources, two case attributes and a metadata file that comprehensively describes the base process parameters as well as the injected drifts together with their drift types. Rheon supports nine drift types from the three perspectives and the two drift modes sudden and gradual. We will use Rheon logs in Chapter 7 to evaluate our state-based framework on known drifts.

Section 4.1 introduces the log schema of Rheon-generated logs and gives an example. Section 4.2 gives an overview of the generation of an event log using Rheon. Section 4.3 introduces the nine drift types supported by Rheon. In Section 4.4 we give examples of generated logs and figures showing the drifts visually. Section 4.5 discusses the limitations and future work of Rheon.

4.1 Generated Log Schema

Rheon generates and writes an event log according to Definition 3.1.1 as a file in the format eXtensible Event Stream (XES) or Comma-Separated Values (CSV). Table 4.1 shows the nine columns of the event log schema. Amount and region are the case attributes. Rheon sorts the events according to case identifiers and start timestamps. The first event of a case starts at the arrival time of the case. All other events start after the end of the previous event and a waiting time. For all generated event logs, Rheon also creates a metadata file that includes the generation parameters and the drifts of the log. Table 4.2 provides an example of a Rheon-generated log.

[1] `https://github.com/boraderen/rheon`
[2] `https://boraderen.github.io/rheon/`

Table 4.1: The columns of a Rheon log. The level states whether an attribute belongs to an event or to a case. The column Name gives the attribute name in Definition 3.1.1.

Column	Level	Name	Meaning
`event:id`	event	–	event identifier
`case:concept:name`	case	*case*	case identifier
`concept:name`	event	*act*	activity
`start_timestamp`	event	–	start of processing
`time:timestamp`	event	*time*	completion of processing
`event:duration_min`	event	*duration*	processing time
`org:resource`	event	*res*	assigned resource
`case:amount`	case	*amount*	case amount
`case:region`	case	*region*	case region

Table 4.2: The first six rows of the example log with a control-flow drift from Section 4.4. Identifiers are abbreviated, and timestamps are shown without year and seconds.

event: id	case: concept: name	concept: name	start_ timestamp	time: timestamp	event: duration_ min	org: resource	case: amount	case: region
evt_1	case_1	a	Jan 1, 00:00	Jan 1, 00:24	24.20	res_03	1,151.36	region_3
evt_2	case_1	b	Jan 1, 00:41	Jan 1, 01:10	29.48	res_01	1,151.36	region_3
evt_3	case_1	c	Jan 1, 01:48	Jan 1, 02:08	20.33	res_01	1,151.36	region_3
evt_4	case_1	d	Jan 1, 02:21	Jan 1, 02:37	16.27	res_04	1,151.36	region_3
evt_5	case_1	e	Jan 1, 02:38	Jan 1, 02:59	21.57	res_01	1,151.36	region_3
evt_6	case_2	a	Jan 1, 09:00	Jan 1, 09:03	3.10	res_03	768.65	region_3

4.2 Generation of an Event Log

Similar to CDLG (Grimm et al. [14]), Rheon describes the control-flow through a process tree (Section 3.1.3) and generates traces with a play-out from a process tree. For the generation and the play-out, Rheon uses the library PM4Py (Berti et al. [9]). Rheon currently provides a single function `rheon.generate_log`. Table 4.3 gives an overview of the parameters of the function. A fixed seed makes the random numbers reproducible, but the play-out of PM4Py for parallel branches can yield non-deterministic interleavings of activities.

Figure 4.1 gives a visual overview of the workflow of log generation.

1. *Build process trees and pools:* Rheon generates a random base process tree T_{base} with `num_activities` activities and the operator weights `tree_weights`. For the n control-flow drifts in `drifts`, it generates n further process trees $T_1, \ldots, T_n$. T_i is the process tree after the i-th control-flow drift and differs from the process tree

Table 4.3: The parameters of `rheon.generate_log` with their default values. Means of times are given in minutes and their variances in squared minutes.

Parameter	Default	Meaning
`drifts`	required	list of drifts to inject
`output_path`	required	path of the log
`log_name`	file name	name in the metadata and of the metadata file
`format`	`"xes"`	output format `"xes"` or `"csv"`
`num_traces`	1,000	approximate number of cases
`num_activities`	10	activities of the base process tree
`num_resources`	8	size of the resource pool
`num_regions`	4	number of regions
`tree_weights`	0.6, 0.25, 0.1, 0.05	weights of sequence, choice, parallelism and loop
`start_date`, `end_date`	Jan 1, 2020, Dec 31, 2020	time horizon in which cases arrive and drifts are placed
`activity_duration`	30, 100	mean and variance of the processing time
`waiting_time`	15, 50	mean and variance of the waiting time
`amount`	1,000, 40,000	mean and variance of the amount
`seed`	42	seed of the random numbers

before it. Each process tree is thus valid in one period of the time horizon, and these periods are separated by the control-flow drifts. Then, the play-out of each process tree gives a pool of traces, i.e., many traces of this process version.

2. *Draw distributions:* For each activity, Rheon draws a mean duration and a mean waiting time. Each mean lies randomly between 60% and 140% of the mean from Table 4.3, so the activities differ in their times. The variances are the same for all activities. Additionally, Rheon draws a dominant resource for each activity and a dominant region for all cases.

3. *Generate arrivals:* The first case arrives at the start of the time horizon $[t_{min}, t_{max}]$. At each point in time, there is a mean inter-arrival time. The gap to the next case is drawn from an exponential distribution with this mean. Then, the gap to the following case is drawn, and so on, until the next arrival would lie after t_{max}. The mean inter-arrival time starts at $(t_{max} - t_{min})/N$, where N is `num_traces`, so that about N cases arrive. From its drift point on, an arrival rate drift replaces this mean by `inter_arrival` or multiplies it by `factor`, and a workload drift divides it by `workload_factor`. For a gradual drift, the mean moves linearly from its old to its new value across the drift window.

4. *Generate cases:* Each case samples a trace from the pool of the process tree that is valid at its arrival. In the drift window of a gradual control-flow drift, a case samples from the new pool with a probability that grows linearly from 0 to 1. The

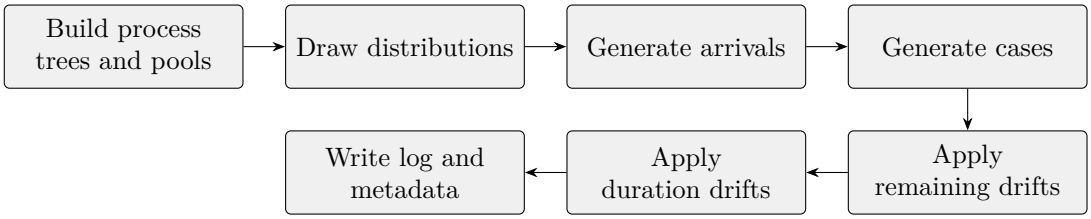


Figure 4.1: The steps in which Rheon generates an event log, in the direction of the arrows. Control-flow drifts take effect when building the process trees and when generating the cases, arrival rate and workload drifts when generating the arrivals. The remaining drifts are reassignment, region, amount, waiting time and pool size drifts.

first event of a case starts at the arrival of the case. For each event, Rheon samples a duration from the normal distribution of its activity and, except for the first event, a waiting time before the event. Each event is executed by the dominant resource of its activity with a probability of 80% and otherwise by a random resource. Similarly, each case receives the dominant region with a probability of 80% and an amount from the normal distribution of the amounts.

5. *Apply the remaining drifts:* The remaining drifts change the generated cases. These are the reassignment, region, amount, waiting time and pool size drifts, which Rheon applies in the order of their start. Drifts on cases, such as region and amount drifts, change the cases that arrive after the drift point. Drifts on events, such as reassignment and waiting time drifts, change the events that start after the drift point. Thus, they also change the later events of cases that arrived before it. For a gradual drift, the change grows linearly across the drift window.

6. *Apply the duration drifts:* Last, the duration drifts multiply the durations of the events of the selected resources by `factor`. Since Rheon applies them after all other drifts, they change exactly the events that a resource executes after all reassignments and pool size changes.

7. *Write log and metadata:* Each event starts after its waiting time from the completion of the previous event of its case and completes after its duration. Rheon computes these start and completion timestamps and rounds them to whole seconds. It then writes the event log as an XES or CSV file together with the metadata file, which also contains the process trees before and after each control-flow drift.

4.3 Drifts

Each drift in `drifts` specifies its drift type `type`, its drift mode `mode`, its position and type-specific parameters. For a sudden drift, the position is a drift point `drift_point`, and for a gradual drift it is a drift window from `start_point` to `end_point`. Positions are

Table 4.4: The nine drift types of Rheon with their perspective and their type-specific parameters.

Perspective	Drift type	What changes	Parameters
Intra-case	`control_flow`	Cases follow a different process tree.	`num_activities`, `tree_weights`
Resource	`pool_size`	Resources are added or removed, and the durations scale.	`delta`, `duration_factor`
Resource	`reassignment`	Each activity gets a new dominant resource.	–
Resource	`workload`	The number of cases per unit of time scales.	`workload_factor`
Resource	`duration`	The durations of selected resources scale.	`resources`, `factor`
Inter-case	`waiting_time`	The mean waiting time between events shifts.	`mean`, `variance`
Inter-case	`amount`	The amounts follow a shifted distribution.	`mean`, `variance`
Inter-case	`arrival_rate`	The mean inter-arrival time changes.	`inter_arrival` or `factor`
Inter-case	`region`	A new region becomes dominant.	–

fractions of the time horizon between 0 and 1, e.g., the drift point 0.5 lies in the middle of the time horizon. Each drift therefore realizes a process change as in Definition 3.2.1. In a gradual drift, the share of the new behavior grows linearly from 0 to 1 across the drift window. Drifts of the same type must not overlap.

Table 4.4 summarizes the nine drift types. They change the generated log as follows:

- *Control-flow* (`control_flow`): Cases after the drift draw their trace from a different process tree, so the trace variants and the directly-follows relations change.
- *Pool size* (`pool_size`): Resources are added or removed. The events of removed resources move to the remaining resources, and each added resource becomes the dominant resource of one activity. The durations are multiplied by `duration_factor` if the pool shrinks and by its reciprocal if the pool grows.
- *Reassignment* (`reassignment`): Each activity gets a new dominant resource.
- *Workload* (`workload`): `workload_factor` times as many cases arrive per unit of time, so each resource handles more or fewer cases.
- *Duration* (`duration`): The durations of the selected resources are multiplied by `factor`.
- *Waiting time* (`waiting_time`): The waiting times between consecutive events of a case follow a new mean or a new variance. A new mean applies to all activities.

```
import rheon

drifts = [
    {"type": "control_flow", "mode": "gradual",
     "start_point": 0.15, "end_point": 0.35, "num_activities": 9},
    {"type": "reassignment", "mode": "sudden", "drift_point": 0.45},
    {"type": "waiting_time", "mode": "gradual",
     "start_point": 0.55, "end_point": 0.75,
     "mean": 45.0, "variance": 80.0},
    {"type": "arrival_rate", "mode": "sudden", "drift_point": 0.85,
     "factor": 0.5},
]
rheon.generate_log(drifts, "L2.xes", num_traces=10000,
                   num_activities=7, num_resources=7, seed=7)
```

Figure 4.2: Python call that generates the event log L_2 with about 10,000 cases, seven activities and seven resources. The drift positions are fractions of the time horizon.

- *Amount* (`amount`): The amounts of the cases follow a shifted distribution.
- *Arrival rate* (`arrival_rate`): The mean inter-arrival time becomes `inter_arrival` minutes or is multiplied by `factor`.
- *Region* (`region`): A new region becomes the dominant region.

For example, the call in Figure 4.2 generates an event log L_2 with a gradual control-flow drift, a sudden reassignment drift, a gradual waiting time drift and a sudden arrival rate drift.

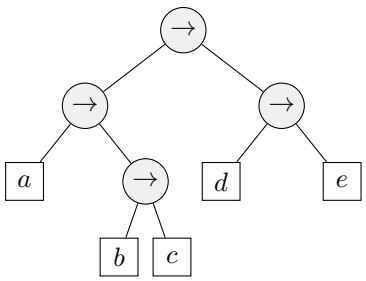
4.4 Examples

We illustrate one drift per perspective with three example logs (Table 4.5). In each log, the base process tree has five activities in a sequence, and a sudden drift lies in the middle of the time horizon 2020, i.e., on July 1, 2020. For each example log, a figure shows the log in the process mining tool Celonis. For the control-flow and the reassignment log, the figure also shows a simple measure before and after the drift.

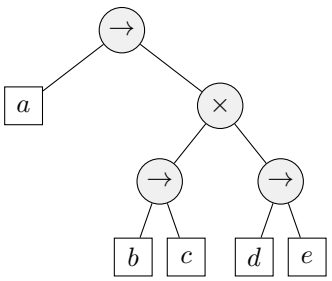
Intra-case perspective: Figure 4.3 shows the process trees of the control-flow drift. Before the drift, all 820 cases follow the trace $\langle a, b, c, d, e \rangle$ of the base process tree T_{base}. From July 1, 2020, the cases follow the process tree T_1. Therefore, 420 cases follow the trace $\langle a, b, c \rangle$ and 398 cases the trace $\langle a, d, e \rangle$. Figure 4.4a shows the directly-follows counts before and after the drift, and Figures 4.4b and 4.4c show the same log in the variant explorer of Celonis. After the drift, the relation from c to d disappears, and a new relation from a to d appears.

Table 4.5: The three example logs with their sudden drift on July 1, 2020, and the generated number of cases and events. The control-flow and the reassignment log have 1,600 desired cases, and the arrival rate log has 2,400. The seeds are 107, 202 and 303.

Perspective	Drift type	Change	Cases	Events
Intra-case	`control_flow`	new process tree $\rightarrow(a, \times(\rightarrow(b, c), \rightarrow(d, e)))$	1,638	6,554
Resource	`reassignment`	new dominant resource for each activity	1,695	8,475
Inter-case	`arrival_rate`	mean inter-arrival time from 219 to 4,380 minutes	1,288	6,440

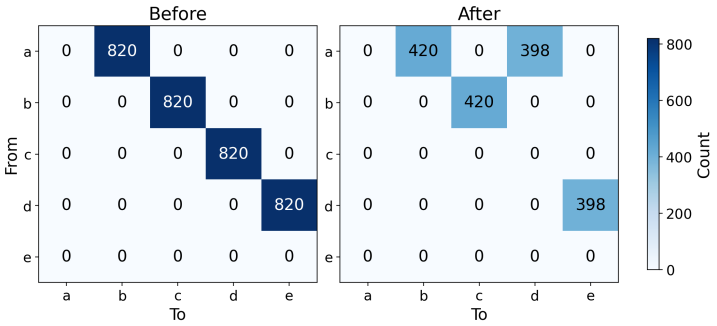


(a) Base process tree T_{base}



(b) Process tree T_1 after the drift

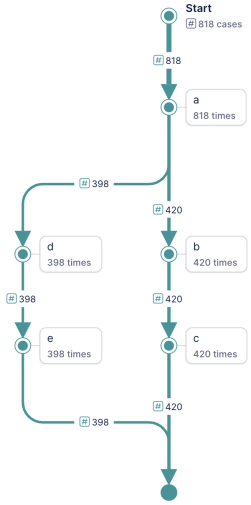
Figure 4.3: The process trees of the example log with a sudden control-flow drift on July 1, 2020. (a) The base process tree $\rightarrow(\rightarrow(a, \rightarrow(b, c)), \rightarrow(d, e))$ produces only the trace $\langle a, b, c, d, e\rangle$. (b) The process tree $\rightarrow(a, \times(\rightarrow(b, c), \rightarrow(d, e)))$ produces the traces $\langle a, b, c\rangle$ and $\langle a, d, e\rangle$.



(a) Directly-follows counts



(b) Before the drift



(c) After the drift

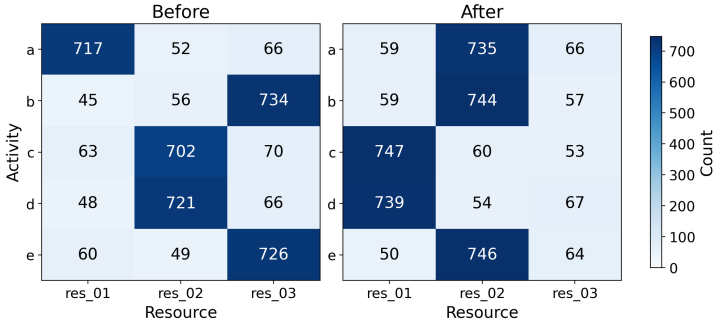
Figure 4.4: The example log with a sudden control-flow drift on July 1, 2020. (a) Number of directly-follows relations from an activity (rows) to the next activity (columns) in the cases that arrive before (left) and after (right) the drift. (b) and (c) Screenshots of the variant explorer of Celonis for the cases before and after the drift. The numbers give how often each activity and each directly-follows relation occurs. Before the drift, all cases follow the trace $\langle a, b, c, d, e \rangle$. After the drift, a case follows either $\langle a, b, c \rangle$ or $\langle a, d, e \rangle$.

Resource perspective: From July 1, 2020, each activity gets a new dominant resource, e.g., activity a moves from `res_01` to `res_02`. Figure 4.5 shows how many events of each activity each resource executes before and after the drift. Before and after the drift, the dominant resource executes between 84% and 88% of the events of its activity. This matches the expected share of $0.8 + 0.2/3 \approx 0.87$, i.e., 87%, for three resources.

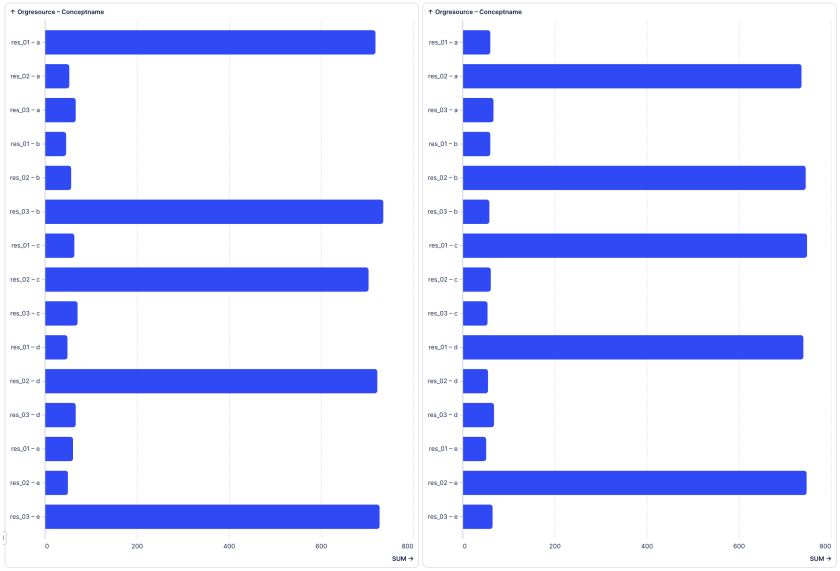
Inter-case perspective: From July 1, 2020, the mean inter-arrival time rises from 219 to 4,380 minutes, so cases arrive 20 times less often. Thus, 1,214 cases arrive before the drift and only 74 cases after it. All cases follow the same trace $\langle a, b, c, d, e \rangle$. Figure 4.6 shows the events of the cases over time in the dotted chart of Celonis. Before the drift, the cases form a steep line, and after the drift, only a few cases arrive.

4.5 Limitations and Future Work

Rheon is a prototype that we develop continuously, so it can still contain bugs. Rheon does not simulate a full operational process. It generates the behavior from process trees and adds times, resources and case attributes to the cases, but it does not model, e.g., the capacities of resources. The generated logs are therefore controlled test data and not replicas of real processes. Compared to CDLG, which also generates incremental and recurring drifts as well as noisy traces [14], Rheon currently supports only sudden and gradual drifts without noise. Future work could extend Rheon with further drift types and drift modes. We also plan to generate object-centric event logs, from which Berti et al. [7] compute execution states.



(a) Events per activity and resource



(b) Events per resource and activity in Celonis

Figure 4.5: The example log with a sudden reassignment drift on July 1, 2020, before (left) and after (right) the drift. (a) Number of events of each activity (rows) that each resource (columns) executes before and after the drift. (b) Screenshot of Celonis with the same numbers as bars for each pair of resource and activity. After the drift, each activity has a new dominant resource, e.g., *a* moves from `res_01` to `res_02`.

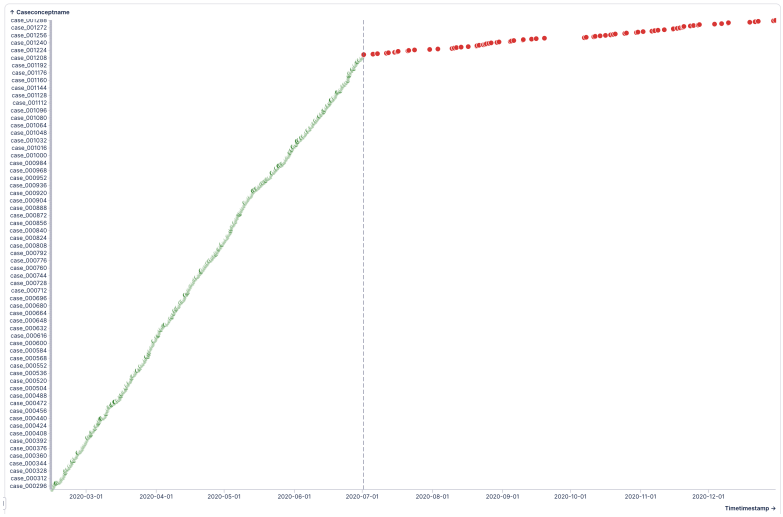


Figure 4.6: Screenshot of the dotted chart of Celonis for the example log with a sudden arrival rate drift on July 1, 2020 (dashed line). Each dot is an event, placed at its time and at its case, with the cases ordered by their arrival. The chart shows the cases from February 2020 on, with the cases before the drift in green and after the drift in red. After the drift, the mean inter-arrival time rises from 219 to 4,380 minutes, so only a few cases arrive.

5 State-Based Drift Detection

In this chapter, we present our state-based framework and thereby answer RQ1. The input is an event log according to Definition 3.1.1. The output consists of intra-case, resource and inter-case states and, for each perspective, a drift signal over time. A high value of this signal indicates a concept drift.

Berti et al. [7] compute execution states of objects: They encode each event as a vector and concatenate the vectors of consecutive events of an object in sliding windows. A PCA compresses these windows, and a SOM maps them onto its cells, which are the states. We transfer these steps from objects to cases and additionally apply them to time windows to obtain resource and inter-case states. A fourth step turns the states into drift signals.

Figure 5.1 shows the four steps with their inputs and outputs:

1. *Feature extraction* (Section 5.1): For each perspective, the event log yields a feature matrix $X \in \mathbb{R}^{m \times n}$. In the intra-case perspective, each row describes an event, and in the resource and inter-case perspectives a feature window.

2. *Scaling and dimensionality reduction* (Section 5.2): Scaling and PCA compress X into a matrix $Z \in \mathbb{R}^{m \times d}$ with $d \leq n$ principal components.

3. *Clustering into states* (Section 5.3): A SOM, k-means or DBSCAN assigns a cluster to each row of Z. Each cluster is a state.

4. *Drift signals* (Section 5.4): For each signal window, we compare the distribution of the states of its rows with a reference, e.g., the previous signal window, using a divergence. The divergence is the drift score of the signal window, and the drift signal is the time series of the drift scores.

The three perspectives pass through the steps separately. Thus, each perspective has its own states and its own drift signal. Each section of this chapter covers one step and contains one subsection per perspective. Chapter 6 introduces the AI-assisted analysis. It is not a step of the pipeline, but interprets the results of the pipeline with an LLM.

The framework uses calendar windows (Definition 3.1.4) in two roles, which Definition 5.1.1 introduces formally:

- *Feature windows:* In the resource and inter-case perspectives, a feature vector describes the events of a feature window, e.g., of one day. The intra-case perspective has no feature windows.

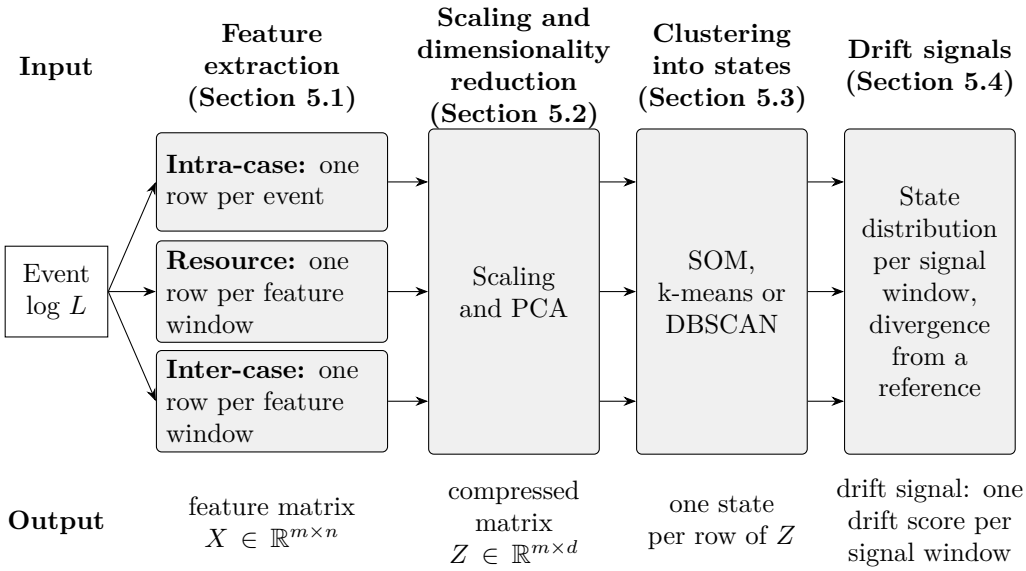


Figure 5.1: Overview of the state-based framework. The event log L passes through the four steps separately for each perspective, shown by the three arrows between two steps. The bottom row gives the output of each step, and each column is one section of this chapter.

- *Signal windows:* The drift signal has one drift score per signal window. A signal window summarizes the states of the events or feature windows that lie in it and is at least as long as a feature window.

As a running example, we generate the event log L_3 with 4,920 cases and 24,600 events from the year 2020 using Rheon (Chapter 4). Its cases execute the five activities a to e, four resources handle them, and the amount is a numerical case attribute. L_3 contains one sudden drift per perspective (Table 5.1):

- *Control-flow:* Before the drift, all cases follow the process tree $\rightarrow(a, \rightarrow(\rightarrow(b, c), \rightarrow(d, e)))$. Afterwards, they follow $\rightarrow(\rightarrow(a, \wedge(b, c)), \wedge(d, e))$, i.e., b and c as well as d and e swap their order.
- *Resource:* Each activity gets a new dominant resource.
- *Inter-case:* The mean amount of the cases rises from 1000 to 2500 euros.

Table 5.2 shows one case before and one case after all drifts. We choose the lengths of the feature and signal windows per perspective in each section.

5.1 Feature Extraction

Feature extraction translates an event log into numbers. It is the only step that reads the event log, so a drift that no feature captures stays invisible in all following steps. In

Table 5.1: The three sudden drifts of the running example L_3. The position is the fraction of the year 2020 at which Rheon injects the drift.

Position	Drift point	Perspective	Drift type
0.25	Apr. 1, 2020, 06:00	Intra-case	`control_flow`
0.50	July 1, 2020, 12:00	Resource	`reassignment`
0.75	Sep. 30, 2020, 18:00	Inter-case	`amount`

Table 5.2: Two cases of the running example L_3, case 1 before and case 3711 after all drifts. Identifiers are abbreviated, timestamps are completion times, durations are in minutes and amounts in euros. Case 3711 swaps b and c as well as d and e, its events are executed by the new dominant resources of their activities, and its amount is higher.

Case	Activity	Timestamp	Resource	Duration	Amount	Region
1	a	Jan. 1, 00:17	`res_01`	17.57	780.51	`region_4`
1	b	Jan. 1, 00:49	`res_01`	27.98	780.51	`region_4`
1	c	Jan. 1, 01:40	`res_02`	39.92	780.51	`region_4`
1	d	Jan. 1, 02:16	`res_01`	19.22	780.51	`region_4`
1	e	Jan. 1, 02:57	`res_04`	25.82	780.51	`region_4`
3711	a	Oct. 2, 01:07	`res_03`	26.25	2476.48	`region_4`
3711	c	Oct. 2, 01:49	`res_03`	31.48	2476.48	`region_4`
3711	b	Oct. 2, 02:19	`res_02`	28.48	2476.48	`region_4`
3711	e	Oct. 2, 03:00	`res_02`	28.42	2476.48	`region_4`
3711	d	Oct. 2, 03:45	`res_02`	27.78	2476.48	`region_4`

the intra-case perspective, a row of the feature matrix describes an event by the prefix of its case. In the other two perspectives, it describes a feature window by the events that lie in it.

The feature groups use the notation for vectors and statistics of Section 3.1.1, and time differences are in minutes (Section 3.1.4).

Definition 5.1.1 (Feature and Signal Windows)**.** Let $o \in \mathcal{U}_{time}$ be an origin, and let $\Delta_f, \Delta_s \in \mathbb{R}$ with $0 < \Delta_f \leq \Delta_s$.

- The feature windows are the calendar windows with the origin o and the length Δ_f, and the signal windows are those with the origin o and the length Δ_s (Definition 3.1.4).
- A feature window $w = [t, t + \Delta_f)$ lies in the signal window that contains its start t. An event lies in the signal window that contains its timestamp.

Thus, each feature window lies in exactly one signal window. If Δ_s is a multiple of Δ_f, the feature window lies completely in it. Otherwise, some feature windows reach

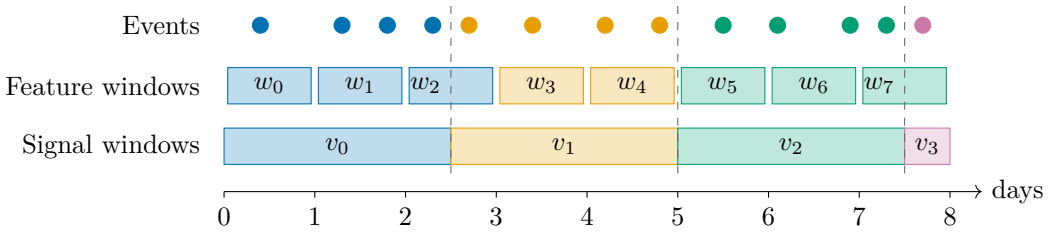


Figure 5.2: Feature windows of $\Delta_f = 1$ day and signal windows of $\Delta_s = 2.5$ days with the same origin at day 0. Each feature window has the color of the signal window that contains its start, each event the color of the signal window that contains its timestamp, and dashed lines are the boundaries of the signal windows. Since 2.5 is no multiple of 1, w_2 and w_7 reach into the next signal window, and v_0, v_1 and v_2 hold three, two and three feature windows. The event after 2.5 days lies in v_1, whereas its feature window w_2 lies in v_0.

into the next signal window, and the signal windows contain different numbers of feature windows (Figure 5.2).

Definition 5.1.2 (Predecessors, Gaps and Feature Windows of an Event Log)**.** Let $L = (E, \pi, \preceq)$ be an event log, and let feature windows of the length Δ_f be given (Definition 5.1.1).

- For a case with the events $e_1 \preceq \cdots \preceq e_l$ (Definition 3.1.2), $prev_L(e_i) = e_{i-1}$ for $1 < i \leq l$ is the predecessor of e_i. The first event of a case has no predecessor, so $prev_L \in E \rightharpoonup E$.
- For $e \in \mathrm{dom}(prev_L)$, $gap_L(e) = \pi_{time}(e) - \pi_{time}(prev_L(e))$ is the gap between e and its predecessor.
- For a feature window w, $E_L^w = \{e \in E \mid \pi_{time}(e) \in w\}$ is the set of its events.
- W_L are the feature windows from the window of the first to the window of the last event of L in chronological order, including those without events.

Definition 5.1.3 (Feature Extraction)**.** Let $L = (E, \pi, \preceq)$ be an event log with the rows $s_1, \ldots, s_m$, i.e., the events in the order $\preceq$ in the intra-case perspective and the feature windows W_L (Definition 5.1.2) in the other two perspectives.

- A feature group f computes from L a vector $f_L(s) \in \mathbb{R}^p$ for each row s, e.g., with one entry per activity.
- For the chosen feature groups $f^1, \ldots, f^g$, $\mathbf{x}_s = (f_L^1(s), \ldots, f_L^g(s)) \in \mathbb{R}^n$ is the feature vector of the row s.
- Together, the feature vectors of all rows form the feature matrix $X \in \mathbb{R}^{m \times n}$ with the rows $\mathbf{x}_{s_1}^\top, \ldots, \mathbf{x}_{s_m}^\top$ (Section 3.3).

The following subsections define the feature groups of each perspective that we use in this thesis. The framework also works with only a part of these feature groups or with self-defined ones. In the dashboard, users choose the feature groups (Section 6.2), and our library kairo is flexible for custom features (Section 6.1).

For each perspective, a figure shows to which Rheon drift types our feature groups respond.

5.1.1 Intra-Case Perspective

The intra-case features describe each event e by the prefix of its case, i.e., by the activities of the case up to and including e. Since a prefix has a variable length, the feature groups abstract it to different degrees. A multiset of the prefix ignores the order of the activities, a set additionally ignores their frequencies, and the last activity ignores all earlier ones. The current and the past activities adopt the encoding of Berti et al. [7], which concatenates the one-hot vectors of the activities of consecutive events.

Definition 5.1.4 (Intra-Case Features)**.** Let $L = (E, \pi, \preceq)$ be an event log with the activities $A_L = \{\pi_{act}(e) \mid e \in E\} = \{a_1, \ldots, a_p\}$, let $D_L = \{d_1, \ldots, d_u\} \subseteq A_L \times A_L$ be the set of directly-follows pairs that occur in a trace of L, and let $h \in \mathbb{N}_0$. Let e_k be the k-th of l events of a case and $\sigma = \mathit{prefix}_L(e_k)$ (Definition 3.1.2). The built-in feature groups of e_k are:

- *Activity frequencies* (`act_freqs`): the share of each activity among the first k events of the case:

$$\mathit{freq}_L(e_k) = (\partial_{\mathit{multiset}}(\sigma)(a_1), \ldots, \partial_{\mathit{multiset}}(\sigma)(a_p))/k.$$

- *Directly-follows counts* (`df_counts`): how often each directly-follows pair occurs in the prefix:

$$\mathit{dfc}_L(e_k) = (\mathit{df}(\sigma)(d_1), \ldots, \mathit{df}(\sigma)(d_u)).$$

- *Activity set* (`act_set`): $\mathit{set}_L(e_k) = \sum_{a_i \in \partial_{set}(\sigma)} \mathit{onehot}_p(i)$, i.e., a 1 for each activity that occurs in the prefix.

- *Case progress* (`case_progress`): $\mathit{progress}_L(e_k) = k/l$.

- *Current activity* (`current_act`): $\mathit{cur}_L(e_k) = \mathit{onehot}_p(i)$ for $\sigma(k) = a_i$.

- *Past activities* (`past_acts`): $\mathit{past}^h_L(e_k) = (\mathbf{y}_1, \ldots, \mathbf{y}_h)$, i.e., the activities of the h previous events of the case, with $\mathbf{y}_j = \mathit{onehot}_p(i)$ for $j < k$ and $\sigma(k - j) = a_i$, and $\mathbf{y}_j = \mathbf{0}$ otherwise.

Properties:

- The feature groups only read the activities of a case. Only the directly-follows counts and the past together with the current activity capture the order.

Table 5.3: Two rows of the intra-case feature matrix of L_3 with three feature groups: the third events of case 1 before and of case 3711 after the control-flow drift (Table 5.2). The activity columns are in the order a to e, the directly-follows columns in the order $a \to b$, $a \to c$, $b \to c$, $b \to d$, $b \to e$, $c \to b$, $c \to d$, $c \to e$, $d \to e$ and $e \to d$. Only the directly-follows counts and the current activity differ between the two events.

Feature group	Case 1, prefix $\langle a, b, c \rangle$	Case 3711, prefix $\langle a, c, b \rangle$
Activity frequencies	$(1/3, 1/3, 1/3, 0, 0)$	$(1/3, 1/3, 1/3, 0, 0)$
Directly-follows counts	$(1, 0, 1, 0, 0, 0, 0, 0, 0, 0)$	$(0, 1, 0, 0, 0, 1, 0, 0, 0, 0)$
Current activity	$(0, 0, 1, 0, 0)$	$(0, 1, 0, 0, 0)$

- The case progress knows the length of the whole case and thus requires a completely recorded event log (Section 1.2).
- An activity or a directly-follows pair that only occurs after a drift gets its own column, which is 0 before the drift.

Example 5.1.5. For the intra-case features of L_3, we choose the activity frequencies, the directly-follows counts and the current activity. With five activities and ten directly-follows pairs, a feature vector has $5+10+5 = 20$ entries. The prefixes $\langle a, b, c \rangle$ and $\langle a, c, b \rangle$ in Table 5.3 contain the same activities in a different order, so only the directly-follows counts and the current activity differ.

Figure 5.3 shows the mapping of the intra-case features. The directly-follows counts and the past activities capture a control-flow drift most directly (Section 4.3), and the other feature groups only if the drift changes the activities or the length of the cases. Among the Rheon drift types, a change of the intra-case states therefore only indicates control-flow drifts.

5.1.2 Resource Perspective

The resource features describe each feature window by how the resources work in it. Adams et al. [3] list, among others, the workload, service times, waiting times and handovers as features of time windows. The activity-resource shares correspond to the probabilities $P(a \to r)$ with which Khraiwesh et al. [16] describe which resource r executes an activity a. Klijn et al. [17] use handovers of cases between actors as a feature for drifts in the actor perspective.

Definition 5.1.6 (Resource Features)**.** Let $L = (E, \pi, \preceq)$ be an event log with a resource for each event, let $prev_L$, gap_L, E_L^w and W_L be as in Definition 5.1.2, and let $R_L = \{\pi_{res}(e) \mid e \in E\} = \{r_1, \ldots, r_q\}$ be the set of resources.

- $H_L = \{e \in \mathrm{dom}(prev_L) \mid \pi_{res}(prev_L(e)) \neq \pi_{res}(e)\}$ is the set of handovers, i.e., of the events whose predecessor another resource executed, and $H_L^w = H_L \cap E_L^w$.

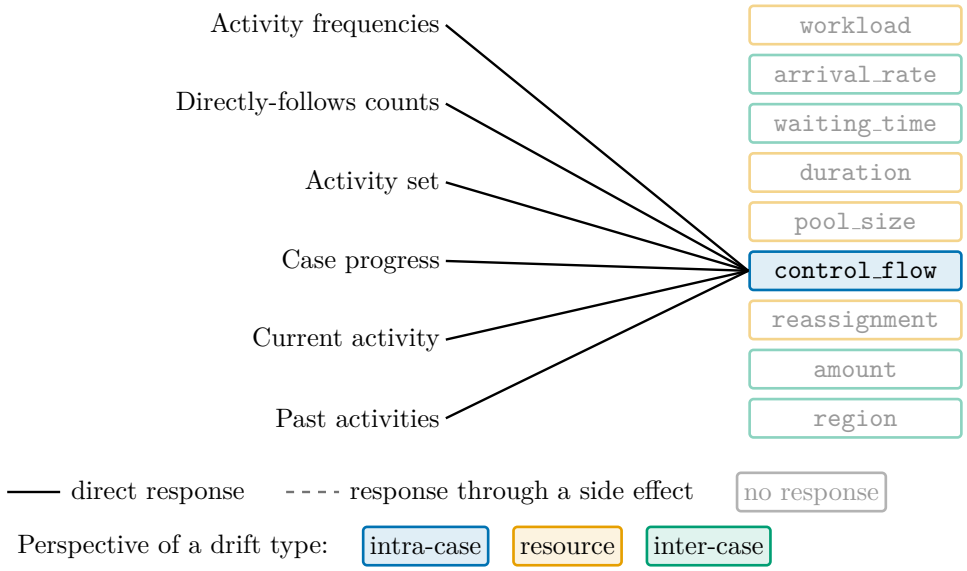


Figure 5.3: The Rheon drift types (right) to which the intra-case feature groups (left) of Definition 5.1.4 respond. Only control-flow drifts reach them.

- For $w \in W_L$ and $r \in R_L$, $E_L^{w,r} = \{e \in E_L^w \mid \pi_{res}(e) = r\}$ is the set of the events of r in w.
- $P_L = \{p_1, \ldots, p_u\}$ is the set of pairs of an activity and a resource, and $Q_L = \{q_1, \ldots, q_v\}$ the set of pairs of a handing-over and a receiving resource that occur in L.

For a feature window $w \in W_L$, the feature groups we use are:

- *Events per resource* (`res_events`):

$$events_L(w) = (|E_L^{w,r_1}|, \ldots, |E_L^{w,r_q}|).$$

- *Active cases per resource* (`res_cases`):

$$cases_L(w) = (x_1, \ldots, x_q) \quad \text{with} \quad x_i = |\{\pi_{case}(e) \mid e \in E_L^{w,r_i}\}|.$$

- *Mean event duration per resource* (`res_durations`), if each event has a duration:

$$dur_L(w) = (x_1, \ldots, x_q) \quad \text{with} \quad x_i = mean([\pi_{duration}(e) \mid e \in E_L^{w,r_i}]).$$

- *Mean wait into resource* (`res_waits`):

$$wait_L(w) = (x_1, \ldots, x_q) \quad \text{with} \quad x_i = mean([gap_L(e) \mid e \in H_L^w \cap E_L^{w,r_i}]).$$

Table 5.4: Rows of the resource feature matrix of L_3 with feature windows of one day and two feature groups, around the resource drift on July 1, 2020. The table is transposed: each column is the feature vector of a feature window. Of the activity-resource event shares, only those of activity a for `res_01` and `res_03` are shown.

Feature	June 29	June 30	July 2	July 3
Events per resource				
`res_01`	45	40	3	4
`res_02`	16	16	32	45
`res_03`	2	3	26	33
`res_04`	18	18	5	9
Activity-resource event shares				
a by `res_01`	0.94	0.88	0.08	0.05
a by `res_03`	0.00	0.06	0.83	0.79

- *Activity-resource event shares* (`act_res_shares`): the share of the events of a in w that r executes, i.e., $ar_L(w) = (x_1, \ldots, x_u)$, where for $p_i = (a, r)$:

$$x_i = ratio(|\{e \in E_L^{w,r} \mid \pi_{act}(e) = a\}|, |\{e \in E_L^w \mid \pi_{act}(e) = a\}|).$$

- *Handover shares* (`handover_shares`): the share of the handovers of r in w that go to r', i.e., $ho_L(w) = (x_1, \ldots, x_v)$, where for $q_i = (r, r')$:

$$x_i = ratio(|\{e \in H_L^w \cap E_L^{w,r'} \mid \pi_{res}(prev_L(e)) = r\}|, \\ |\{e \in H_L^w \mid \pi_{res}(prev_L(e)) = r\}|).$$

Properties:

- The counts grow with the number of cases per unit of time. The shares are normalized per activity or per handing-over resource and thus independent of this number.
- A handover counts in the feature window of the receiving event e. Since the timestamps are completion times, $gap_L(e)$ covers the waiting time before e and the duration of e.

Example 5.1.7. For the resource features of L_3, we choose feature windows of one day and the feature groups events per resource and activity-resource event shares. With four resources and 20 pairs of an activity and a resource, a feature vector has $4 + 20 = 24$ entries. Before the drift, `res_01` is the dominant resource of a, b and d. Afterwards, `res_02` and `res_03` take over these activities (Table 5.4).

Figure 5.4 shows the mapping of the resource features:

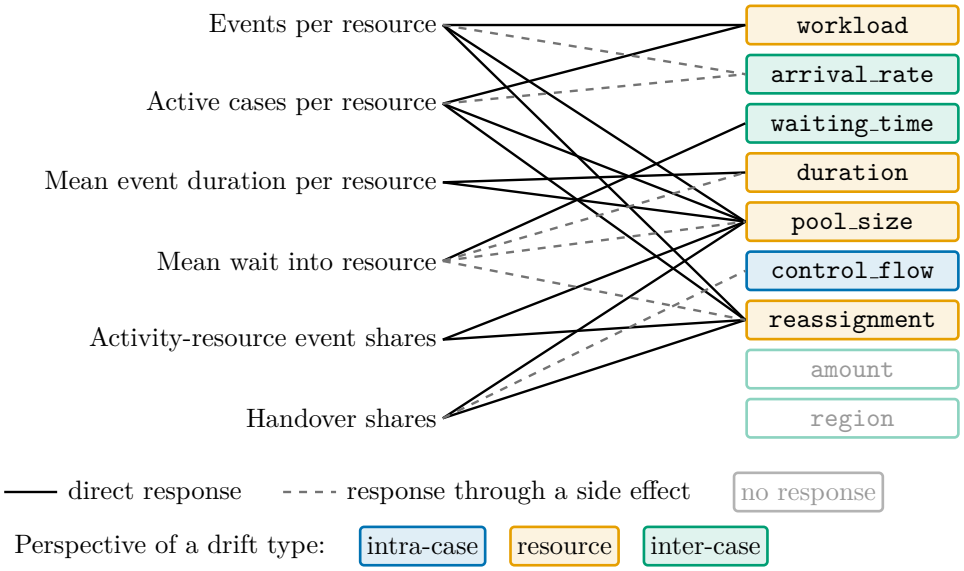


Figure 5.4: The Rheon drift types (right) to which the resource feature groups (left) of Definition 5.1.6 respond. Besides the resource drift types, they also respond to arrival rate, waiting time and control-flow drifts.

- *Reassignment and pool size* change which resource executes an activity and thus the counts and the shares. A pool size drift additionally scales the durations (Section 4.3) and therefore also reaches the feature groups of a duration drift.
- *Duration:* Since a gap includes the duration of the receiving event, the waiting times also respond to duration drifts.
- *Workload and arrival rate* both change the mean inter-arrival time in Rheon (Section 4.2) and thus the counts in the same way. No resource feature distinguishes them.
- *Control-flow:* Since resources are bound to activities, a new order of the activities also changes the handovers.

5.1.3 Inter-Case Perspective

The inter-case features describe each feature window by all cases that run in it. Kraus et al. [18] describe a process as a system by the number of arriving, active and completed cases per period. Similarly, Senderovich et al. [27] use the number of cases running at a point in time in their coarsest inter-case encoding.

Definition 5.1.8 (Inter-Case Features)**.** Let $L = (E, \pi, \preceq)$ be an event log, let $prev_L$, gap_L, E_L^w and W_L be as in Definition 5.1.2 with the length Δ_f of the feature windows, and let $\tau > 0$ be a stall threshold. Let $N_{num} = \{n_1, \ldots, n_s\}$ and N_{cat} be disjoint sets

of numerical and categorical case attributes of L (Definition 3.1.1) that every event has, and let $V = \{u_1, \ldots, u_t\}$ be the set of pairs (n, v) of an attribute $n \in N_{cat}$ and one of its values v in L. For a feature window $w = [t, t + \Delta_f) \in W_L$, our feature groups are:

- *Active cases* (`active_cases`): $active_L(w) = |\{\pi_{case}(e) \mid e \in E_L^w\}|$.
- *New arrivals* (`new_arrivals`): $arrivals_L(w) = |\{c \in C_L \mid start_L(c) \in w\}|$.
- *Completions* (`completions`): $completions_L(w) = |\{c \in C_L \mid end_L(c) \in w\}|$.
- *Events per active case* (`events_per_case`):

$$epc_L(w) = ratio(|E_L^w|, active_L(w)).$$

- *Mean within-case gap* (`mean_delta_t`) and *standard deviation of within-case gaps* (`std_delta_t`): $gapmean_L(w) = mean(G)$ and $gapstd_L(w) = std(G)$ with

$$G = [gap_L(e) \mid e \in E_L^w \cap \mathrm{dom}(prev_L)].$$

- *Stalled cases* (`stalled_cases`): the cases whose last event lies more than τ before the end $t + \Delta_f$ of w and that still have a later event:

$$stalled_L(w) = |\{\pi_{case}(e) | e \in \mathrm{dom}(prev_L) \wedge \pi_{time}(prev_L(e)) + \tau < t + \Delta_f < \pi_{time}(e)\}|.$$

- *Means* (`attr_means`) and *standard deviations of numerical attributes* (`attr_stds`) with $M_n = [\pi_n(e) \mid e \in E_L^w]$:

$$attrmean_L(w) = (mean(M_{n_1}), \ldots, mean(M_{n_s})),$$
$$attrstd_L(w) = (std(M_{n_1}), \ldots, std(M_{n_s})).$$

- *Value shares of categorical attributes* (`attr_shares`): the share of the events in w with the value v of n, for $u_i = (n, v)$:

$$attrshare_L(w) = (x_1, \ldots, x_t) \quad \text{with} \quad x_i = ratio(|\{e \in E_L^w \mid \pi_n(e) = v\}|, |E_L^w|).$$

Properties:

- A case is active in every feature window that contains one of its events. Arrivals and completions, in contrast, count it only once.
- If only the number of cases per unit of time rises, the events per active case stay about the same.
- The stalled cases are only informative if τ is of the order of the gaps and many ends of feature windows fall into gaps.

Table 5.5: Rows of the inter-case feature matrix of L_3 with feature windows of 12 hours and four feature groups, around the inter-case drift on September 30, 2020, 18:00. The table is transposed: each column is the feature vector of a feature window, given by its start. Gaps are in minutes and amounts in euros.

Feature	Sep. 29, 12:00	Sep. 30, 00:00	Oct. 1, 00:00	Oct. 1, 12:00
Active cases	6	9	7	8
New arrivals	5	7	4	7
Mean within-case gap	50.66	45.22	44.29	42.50
Mean of the amount	912.46	931.22	2375.81	2599.66

- The attribute features weight events, not cases. Without chosen case attributes, drifts in the case attributes stay invisible.

Example 5.1.9. For the inter-case features of L_3, we choose shorter feature windows of 12 hours and the feature groups active cases, new arrivals, mean within-case gap and the mean of the amount. Only the mean amount jumps at the drift, while the other features fluctuate from window to window (Table 5.5).

Figure 5.5 shows the mapping of the inter-case features:

- *Arrival rate and workload* change the active cases, arrivals and completions in the same way, so that, as in the resource perspective, no feature distinguishes them.
- *Waiting time, duration and pool size* change the gaps and the stalled cases. Since slower cases run longer, the number of active cases also rises.
- *Control-flow:* If a control-flow drift changes the length of the cases, the events per active case change.

Together, Figures 5.3, 5.4 and 5.5 show that each Rheon drift type directly reaches at least one feature group of its perspective, but some drift types also reach other perspectives. The drift perspective of a drift therefore only follows from comparing the states of all three perspectives.

5.2 Scaling and Dimensionality Reduction

The features of a perspective have different units and value ranges. Many of them are also correlated, e.g., the frequency of an activity and the directly-follows counts that contain the activity. Since the clustering compares rows (feature windows or events) by their distances, we first scale the features such that no feature dominates due to its value range. Then, PCA compresses the scaled features into a few principal components (Section 3.3). Similar to Berti et al. [7], to choose the number of components we use the elbow of the reconstruction error as a heuristic. As a second heuristic, the explained

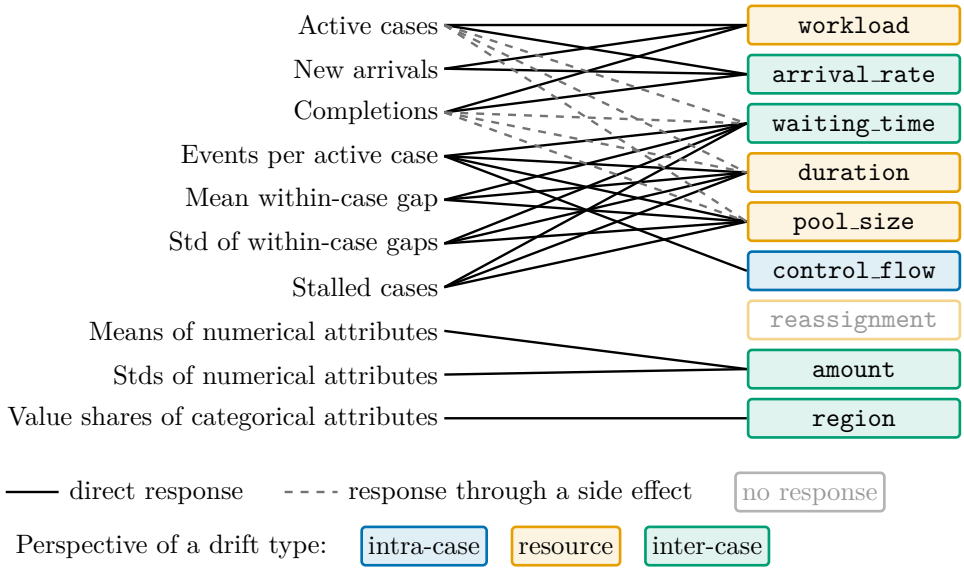


Figure 5.5: The Rheon drift types (right) to which the inter-case feature groups (left) of Definition 5.1.8 respond. The attribute features reach amount and region drifts only if the amount and the region are chosen as case attributes.

variance of the kept components is also useful. Both scaling and PCA are optional. Our framework is flexible and can easily be extended to different scaling techniques such as robust scaling or different dimensionality reduction techniques like autoencoders.

Definition 5.2.1 (Scaling and Dimensionality Reduction)**.** Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with $m \geq 2$ (Definition 5.1.3), and let $B \subseteq \{1, \ldots, n\}$ be the columns of its binary feature groups, i.e., of the activity set, the current activity and the past activities (Definition 5.1.4). In the resource and inter-case perspectives, $B = \emptyset$.

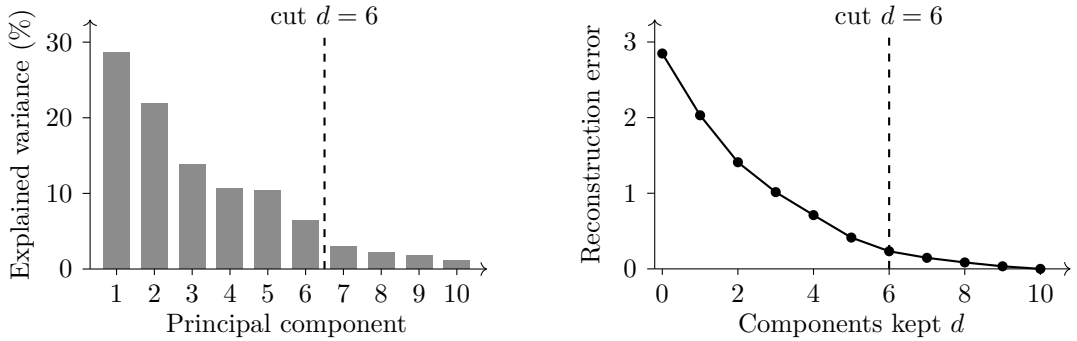
- The scaled matrix $X' \in \mathbb{R}^{m \times n}$ is either X itself, or it has the entries $x'_{j,i} = x_{j,i}$ for $i \in B$ and, for $i \in \{1, \ldots, n\} \setminus B$, the entries of the z-score standardization or of the min-max normalization of X (Definition 3.3.1).
- Let $\mathbf{x}'_1, \ldots, \mathbf{x}'_m$ be the rows of X', $\boldsymbol{\mu}$ its mean vector and $\mathbf{v}_1, \ldots, \mathbf{v}_n$ its principal components (Definition 3.3.2). The reduced matrix $Z \in \mathbb{R}^{m \times d}$ is either X' with $d = n$, or, for a cut $1 \leq d \leq n$, the matrix with the rows $\mathbf{z}_1^\top, \ldots, \mathbf{z}_m^\top$, where

$$\mathbf{z}_j = W_d^\top (\mathbf{x}'_j - \boldsymbol{\mu}) \quad \text{with} \quad W_d = [\mathbf{v}_1, \ldots, \mathbf{v}_d]$$

  is the projection of $\mathbf{x}'_j$ onto the first d principal components.

Properties:

- PCA is fitted on all rows of the feature matrix. Thus, the principal components describe the variance across all process versions.



(a) Explained variance per component

(b) Reconstruction error

Figure 5.6: PCA of the min-max normalized intra-case features of the running example L_3 (Example 5.2.2). (a) Share of the variance that each principal component explains. Components 11 to 20 explain no variance. (b) Reconstruction error $err_{X'}(d)$ over the number d of kept components. After the cut at $d = 6$ (dashed), a further component barely lowers the error.

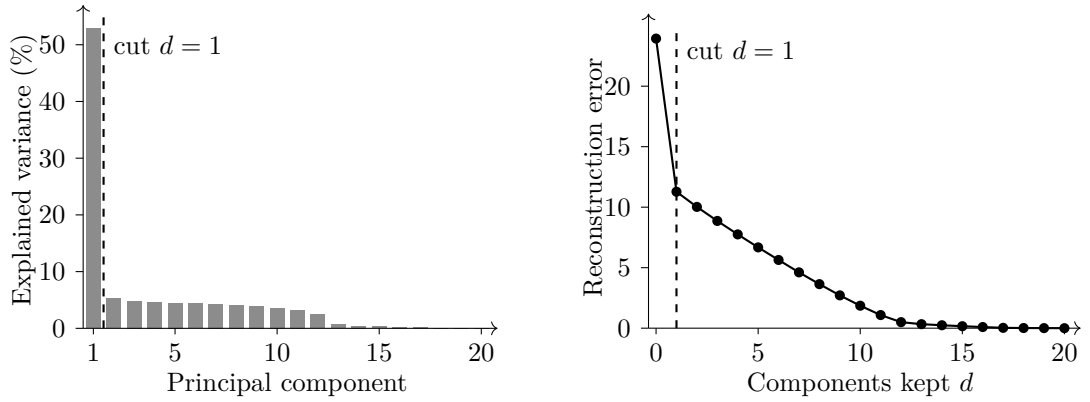
- We leave the binary feature groups unscaled.
- The z-score standardization gives each feature the same variance. The min-max normalization keeps the features in $[0, 1]$. A single outlier thus compresses all other values of its feature.
- Each principal component is a weighted sum of the features, and its largest weights show which features it describes. Without PCA, each dimension of Z remains a single, named feature.

5.2.1 Intra-Case Perspective

The intra-case feature matrix has a row per event and can be long, but its columns depend strongly on each other, e.g., the activity frequencies of a prefix sum to 1.

Example 5.2.2. We normalize the 20 intra-case feature columns of L_3 (Example 5.1.5) except for the five binary columns of the current activity. Then, we fit the PCA on the 24,600 events (Figure 5.6).

- The min-max normalization stretches, e.g., the frequencies of d and e from $[0, 0.25]$ to $[0, 1]$.
- The 24,600 rows only take 13 different vectors, one per prefix, and the columns are linearly dependent. Thus, only ten of the 20 principal components explain variance.
- The reconstruction error knee is reached after six components, which keep 91.8% of the variance. Thus, we cut at $d = 6$.



(a) Explained variance per component

(b) Reconstruction error

Figure 5.7: PCA of the z-score standardized resource features of the running example L_3 (Example 5.2.3). (a) Share of the variance that each principal component explains. (b) Reconstruction error $err_{X'}(d)$ over the number d of kept components.

- The first component mainly weights the directly-follows counts $b \to c$ and $a \to b$, the second $a \to c$ and $c \to b$. Thus, the first two principal components already separate the orders before and after the control-flow drift.

5.2.2 Resource Perspective

The resource feature matrix has a row per feature window and is therefore short. Its columns mix counts and shares with very different value ranges. Moreover, many shares change together, e.g., when another resource takes over an activity, its share rises and that of the previous resource falls.

Example 5.2.3. We choose the z-score standardization, so that each of the 24 resource feature columns of L_3 (Example 5.1.7) contributes the same variance to the PCA. Then, we fit the PCA on the 366 feature windows (Figure 5.7).

- Without scaling, the events per resource with standard deviations of up to 17.75 would dominate the shares with at most 0.41. The first component would then explain 83.8% of the variance and weight almost only the events per resource.
- After the standardization, the first component explains 52.9% of the variance. It weights the ten shares of the dominant resources before and after the reassignment with about ± 0.27. The weights are negative for the dominant resources before the drift and positive for those after it, e.g., -0.27 for a by `res_01` and $+0.27$ for a by `res_03` (Table 5.4).
- The reconstruction error with the first component is 11.28 and we keep only the first component.

- The first component alone separates the feature windows before and after the drift. It lies between -4.36 and -2.63 for the 182 days before July 1 and between 1.95 and 4.56 for the 183 days after it.

5.2.3 Inter-Case Perspective

Most inter-case feature groups have a single column, so the inter-case feature matrix has only a few columns. These columns measure different quantities, e.g., counts, minutes and euros. Thus, scaling is essential, whereas PCA only saves a few dimensions. Without PCA, each dimension of Z remains a named feature.

Example 5.2.4. The inter-case feature matrix of L_3 has four columns and 731 feature windows (Example 5.1.9). We choose the z-score standardization and skip PCA, so the reduced matrix consists of the four standardized features.

- Without scaling, the mean amount with a standard deviation of 674 euros would explain 99.99% of the variance, and the other three features would hardly count.
- The feature window of July 18, 00:00, contains no events, so all its features are 0. With the min-max normalization, this window would squeeze more than 90% of the mean within-case gaps into $[0.73, 0.89]$. The z-score standardization gives all four features the standard deviation 1, and this window becomes an outlier with the standardized mean gap -14.53.
- The active cases and the new arrivals have the correlation 0.90. PCA merges them into the first component with 47.6% of the variance, which gives the mean amount almost no weight. A cut at $d = 1$ would therefore lose the drift.
- The mean amount and the mean within-case gap are hardly correlated (0.07), so the second and third components mix them with almost equal weights. A cut at the elbow $d = 3$ would keep 97.4% of the variance, but spread the amount over two components and only save one dimension.
- Without PCA, the standardized mean amount rises directly from -0.67 in the feature window of September 30, 00:00, to 1.48 in that of October 1, 00:00 (Table 5.5).

The three examples show that the explained variance alone does not decide the cut. The first resource component keeps only 52.9% of the variance, but separates the days before and after the reassignment. The first inter-case component keeps almost as much, 47.6%, but misses the amount drift.

5.3 Clustering into States

A clustering groups the rows (feature windows or events) of the reduced matrix such that rows that describe similar behavior end up in the same cluster. We treat each

cluster as a state. We use the three clustering algorithms k-means, SOM and DBSCAN (Section 3.4). Similar to the previous steps, our framework is flexible and can easily be extended with further clustering algorithms such as k-medoids or hierarchical clustering. For each perspective, users choose the algorithm and its hyperparameters depending on the event log, e.g., in the dashboard (Section 6.2).

The algorithms also help to choose each other's hyperparameters. For example, a SOM with a large grid shows how many groups of frequently used cells lie far apart from each other, which suggests the number k of clusters for k-means. The evaluation measures of Section 3.4.4 additionally show how good a clustering is and support the choice of the grid size, k, ε and *minPts*.

Our framework supports various distances for the clustering. Besides the Euclidean distance, the SOM and DBSCAN also work with the Manhattan, Chebyshev and cosine distances (Section 3.3). k-means only uses the Euclidean distance, since each of its centers is the mean of its vectors (Definition 3.4.1). In the running example, all three algorithms use the Euclidean distance.

Definition 5.3.1 (States and Trajectories)**.** Let L be an event log, let $Z \in \mathbb{R}^{m \times d}$ be its reduced matrix with the rows $\mathbf{z}_1^\top, \ldots, \mathbf{z}_m^\top$ (Definition 5.2.1), and let $cl \in \{1, \ldots, m\} \to K$ be a clustering of Z by a SOM, k-means or DBSCAN (Definitions 3.4.2, 3.4.1 and 3.4.3).

- K is the set of states, and $\mathit{state}(\mathbf{z}_j) = cl(j)$ is the state of the row $\mathbf{z}_j$ for $1 \leq j \leq m$. For DBSCAN, *noise* is also a state.
- In the intra-case perspective, each row belongs to an event (Definition 5.1.3). A case whose events belong to the rows $\mathbf{z}_{j_1}, \ldots, \mathbf{z}_{j_l}$ in the order $\preceq$ has the trajectory $\langle \mathit{state}(\mathbf{z}_{j_1}), \ldots, \mathit{state}(\mathbf{z}_{j_l}) \rangle$.
- In the resource and inter-case perspectives, the rows $\mathbf{z}_1, \ldots, \mathbf{z}_m$ belong to the feature windows $W_L = \langle w_1, \ldots, w_m \rangle$ (Definition 5.1.2). The event log has the trajectory $\langle \mathit{state}(\mathbf{z}_1), \ldots, \mathit{state}(\mathbf{z}_m) \rangle$.

Properties:

- Similar to PCA, the clustering uses all rows of Z. Therefore, all process versions share the same states. A drift does not change the states, but how often each state occurs. Section 5.4 measures this change.
- In the intra-case perspective, each event has a state, whereas in the resource and inter-case perspectives each feature window has a state.
- Consecutive equal states of a trajectory form a visit of the state. A trajectory thus shows how long a case or the event log stays in a state.
- In the intra-case perspective, a trajectory only covers a single case. Thus, a drift only shows when we compare the trajectories of cases before and after it. In the resource and inter-case perspectives, the trajectory covers the whole event log, so a change of its state already indicates a drift.

Table 5.6: Metrics of the chosen clusterings of L_3 (Section 3.4.4). DBSCAN has two clusters and noise.

Perspective	Clustering	States	*sil*	*ch*	*qe*	*ent*	*dbcv*
Intra-case	SOM, 3×3	9	0.857	36,694	0.288	0.973	–
Resource	k-means, $k = 2$	2	0.934	24,779	–	–	–
Inter-case	DBSCAN, $\varepsilon = 0.7$, $minPts = 8$	3	0.382	271	–	–	0.141

- Each state has a representative vector: the prototype of its cell, its center or the mean of its core points. The distance between two representatives shows how similar two states are. Our implementation gives close states similar colors (Section 6.1).

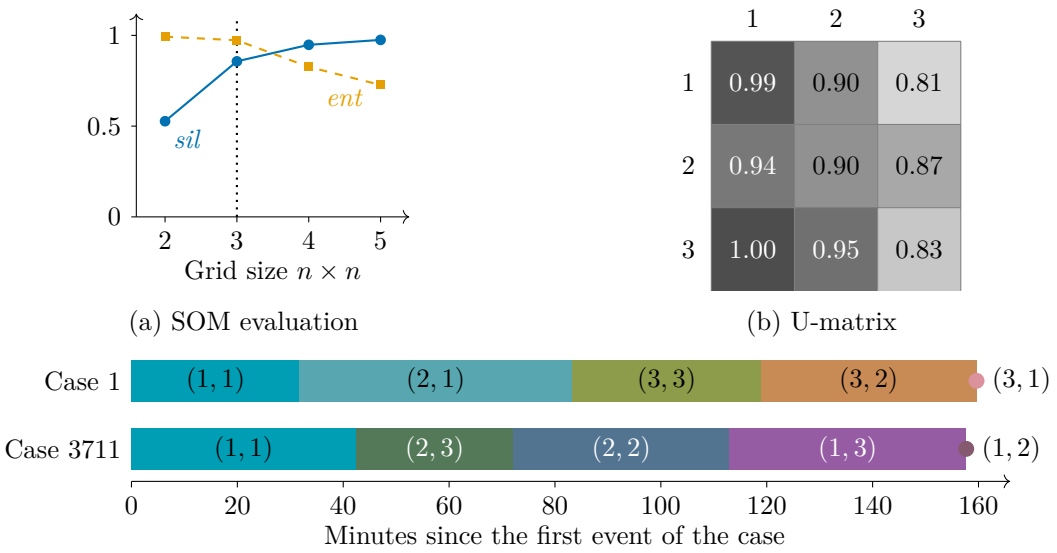
In each perspective of the running example, we use a different algorithm. Table 5.6 shows the chosen clusterings, and the following examples interpret the resulting states.

5.3.1 Intra-Case Perspective

An intra-case state describes how far a case has progressed and on which path. Since the intra-case features only read the activities of a case, all cases of the same trace variant have the same trajectory. Thus, many events share the same row of the reduced matrix, e.g., the k-th events of all cases of a trace variant.

Example 5.3.2. Like Berti et al. [7], we choose a SOM, interpret its cells as states and read the trajectory of a case as a path on the grid. Neighboring cells have similar prototypes, so the grid also shows which states are similar. We cluster the reduced intra-case matrix of L_3 (Example 5.2.2) with SOMs of the grid sizes 2×2 to 5×5 (Figure 5.8a). With the grid size, the quantization error falls from 0.917 to 0.052, and the silhouette rises from 0.527 to 0.975. At the same time, the occupancy entropy falls from 0.993 to 0.727, since the 5×5 grid only uses 12 cells and gives almost each of the 13 prefixes its own cell. The 3×3 grid uses all cells almost evenly with the occupancy entropy 0.973 and reaches the silhouette 0.857 with the quantization error 0.288. Thus, we choose the 3×3 grid (Figure 5.9).

- Before the control-flow drift, all cases follow the trace $\langle a, b, c, d, e \rangle$. Its five prefixes occupy five cells, and case 1 has the trajectory $\langle (1,1), (2,1), (3,3), (3,2), (3,1) \rangle$.
- After the drift, the prefixes that start with $\langle a, c \rangle$ occupy the four remaining cells. Case 3711 has the trajectory $\langle (1,1), (2,3), (2,2), (1,3), (1,2) \rangle$ and thus only shares its first state with case 1.
- The prefixes $\langle a, b, c, e \rangle$ and $\langle a, b, c, e, d \rangle$ only occur after the drift, but share their cells with older prefixes. Thus, the cells $(3,1)$ and $(3,2)$ occur before and after the drift.



(a) SOM evaluation

(b) U-matrix

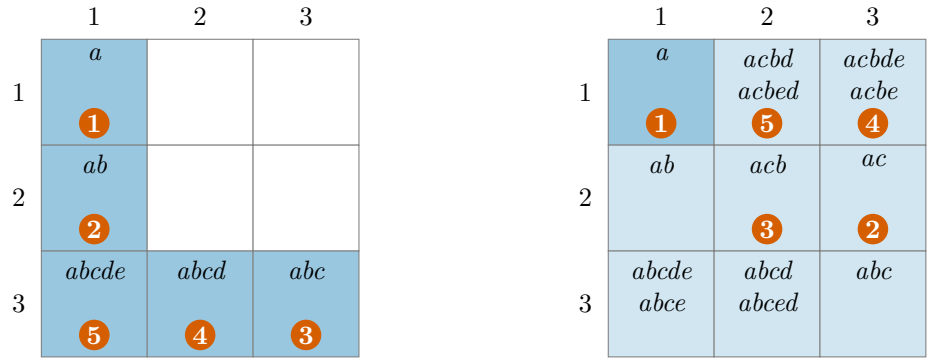
(c) Trajectories of case 1 and case 3711

Figure 5.8: SOM clustering of the intra-case features of the running example L_3 (Example 5.3.2). (a) Silhouette (blue) and occupancy entropy (orange) over the grid size. The dotted line marks the chosen 3×3 grid. (b) Mean distance of the prototype of each cell to the prototypes of its neighboring cells, divided by the largest mean. (c) Each box is a visit of a state with its cell of Figure 5.9. The last state of a case is a dot.

- Both cases take about 160 minutes, and each of their first four visits lasts 30 to 51 minutes (Figure 5.8c). Thus, the drift changes the states, but not the duration of the cases.
- The prototypes of consecutive states of both cases lie 1.63 to 2.00 apart, and those of all states 2.23 on average. Since our implementation chooses the colors by these distances, the color only changes gradually along a trajectory.
- The U-matrix only ranges from 0.81 to 1.00, so the grid shows no border between larger groups of states (Figure 5.8b). The four new cells form the top right region. Their prototypes lie closer to their neighbors than those of the old cells, with 0.87 against 0.94 on average.

5.3.2 Resource Perspective

A resource state describes how the work of a feature window is distributed among the resources. Consecutive feature windows of a stable process lie close together, so a few compact clusters suffice.



(a) Before the drift, with case 1

(b) After the drift, with case 3711

Figure 5.9: The 3×3 SOM of the intra-case states of L_3 (Example 5.3.2). Each cell lists the prefixes of its events, e.g., *abc* for $\langle a, b, c \rangle$. Dark cells hold about 20% and light cells about 10% of the events. A circle with the number k marks the state of the k-th event of the case. (a) The 6,255 events before the drift fall into five cells. (b) The 18,345 events after the drift use all nine cells.

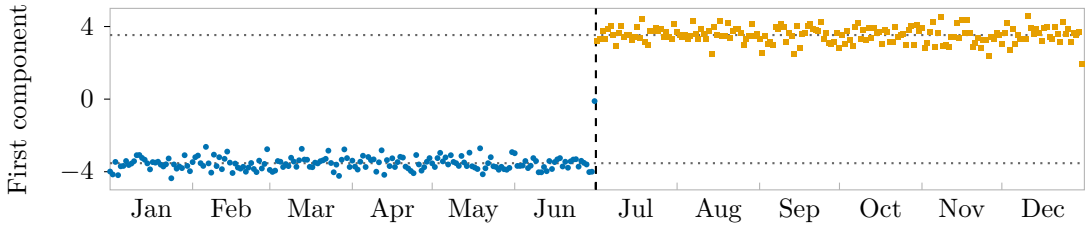
Example 5.3.3. Since a few compact clusters suffice, we choose k-means with a small k. The reduced resource matrix of L_3 only has the first principal component (Example 5.2.3). For $k = 2$, k-means reaches the highest silhouette of 0.934 and the highest Caliński–Harabasz index of 24,779. For $k = 3$ to 5, the silhouette drops to between 0.719 and 0.739, and for $k = 6$ to 0.539. Thus, we choose $k = 2$ with the centers -3.53 and 3.53 (Figure 5.10a).

- State 1 contains the 183 days up to July 1, and state 2 the 183 days from July 2. Thus, the trajectory of the event log consists of two visits of 183 days each (Figure 5.10b).
- The feature window of July 1 contains the reassignment at 12:00. With -0.11, the feature window lies between both centers, but since k-means assigns each row to a state, it is assigned to state 1.
- With $k = 3$, k-means splits the days after the drift into two states, and the trajectory has 96 instead of two visits. With $k = 6$, it also splits the days before the drift. These state changes look like drifts, although the process is stable.

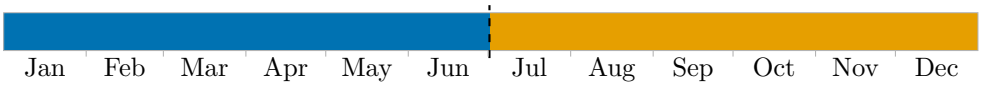
5.3.3 Inter-Case Perspective

An inter-case state describes the load and the cases of a feature window. Unusual feature windows, e.g., with very few arrivals, occur in every process version, but do not describe a typical situation of the process.

Example 5.3.4. We choose DBSCAN, which marks unusual feature windows as noise instead of forcing them into a state. We cluster the four standardized inter-case features



(a) First principal component of the feature windows



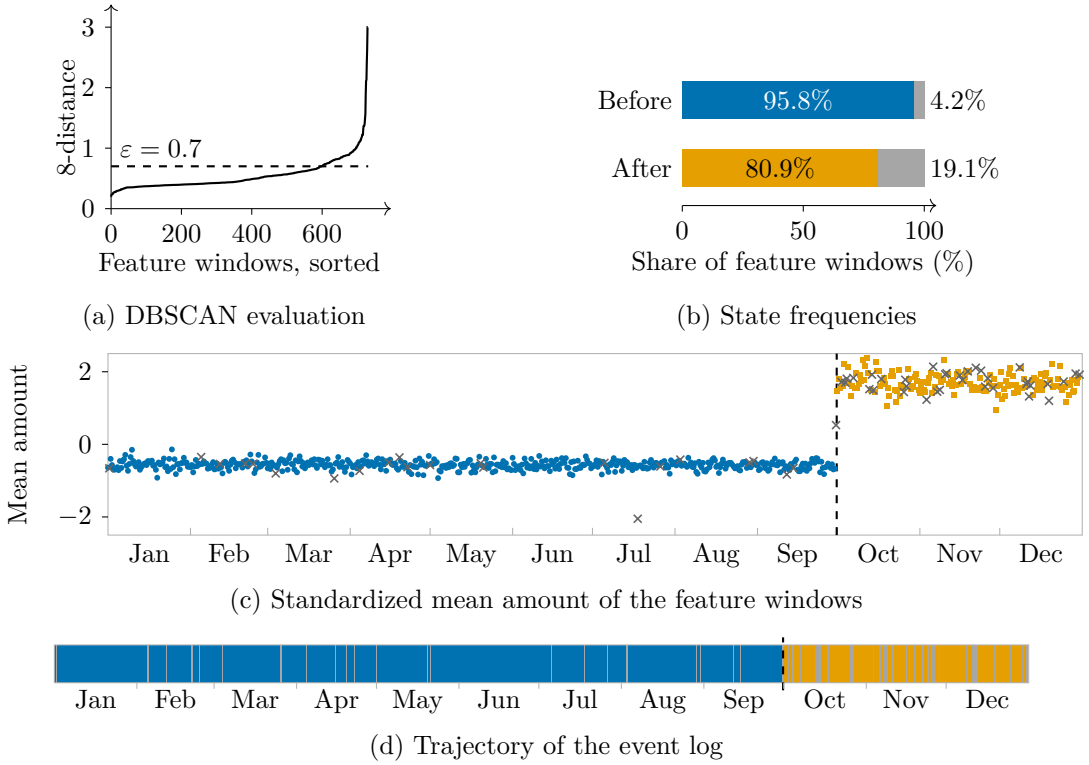
(b) Trajectory of the event log

Figure 5.10: k-means clustering of the resource features of the running example L_3 (Example 5.3.3). Blue is state 1 and orange state 2. (a) Each symbol is a feature window. The dotted lines are the two centers. (b) Each box is a visit of a state. Dashed lines mark the reassignment drift.

of L_3 (Example 5.2.4) with $minPts = 8$, i.e., twice the number of features. The sorted 8-distance graph rises gradually and only steeply for a few outliers, so it has no clear knee (Figure 5.11a). The largest 8-distance of 13.06 belongs to the empty feature window of July 18, 00:00, with the standardized mean gap -14.53. Therefore, we compare the radii $\varepsilon \in \{0.6, 0.7, 0.8, 0.9\}$ with DBCV, and $\varepsilon = 0.7$ reaches the highest DBCV of 0.141.

- State 1 contains 525 feature windows before the amount drift, and state 2 contains 148 after it (Figure 5.11c). The remaining 58 feature windows are noise.
- The feature window of September 30, 12:00, contains cases before and after the drift. Its standardized mean amount of 0.53 lies between both clusters, so it is noise.
- Before the drift, 4.2% of the feature windows are noise, and after it 19.1% (Figure 5.11b).
- The trajectory of the event log has 98 visits (Figure 5.11d). State 1 has 23 visits of up to 45 days and state 2 has 26 visits of up to 9.5 days. The 49 visits of the noise last at most 2 days and only briefly interrupt the visits of the clusters.
- For $\varepsilon \geq 1.0$, DBSCAN merges both clusters into one state, and the drift disappears from the states.

Table 5.6 compares the chosen clusterings of the three perspectives. In the intra-case and the resource perspective, the drift clearly separates the rows, so the silhouettes reach 0.857 and 0.934. In the inter-case perspective, the drift only changes one of the four features, and the other features scatter the clusters. Thus, the silhouette of 0.382



(a) DBSCAN evaluation

(b) State frequencies

(c) Standardized mean amount of the feature windows

(d) Trajectory of the event log

Figure 5.11: DBSCAN clustering of the inter-case features of the running example L_3 (Example 5.3.4). Blue is state 1, orange state 2 and gray noise. (a) Sorted 8-distance graph with the chosen radius $\varepsilon = 0.7$ (dashed), cut at 3. (b) Shares of the states before and after the amount drift. (c) Each symbol is a feature window. Since DBSCAN also uses the other three features, some noise lies within the clusters. (d) Each box is a visit of a state. In (c) and (d), dashed lines mark the amount drift.

and the DBCV of 0.141 stay low, although the states show the drift. The Caliński–Harabasz index depends on the scale of the features and thus only compares clusterings of the same perspective.

5.4 Drift Signals

A state describes a single row, whereas a drift describes a change over time. Since all process versions share the same states, a drift can be seen in how often each state occurs. A state that only occurs after a drift has the frequency 0 before the drift. A drift signal makes these changes measurable. For each signal window, we count how often the rows have the states and obtain a state distribution. A divergence compares these state distributions with a reference, e.g., the state distribution of the previous signal window. The divergence of a signal window is the drift score of that signal window. The time series of drift scores over the signal windows is the drift signal.

Clustering-based detectors observe how the composition of clusters changes over time (Sato et al. [26]). The drift signal quantifies this change for each signal window. The comparison with a reference follows the window-based detectors. ProDrift compares the distribution of the runs of a reference window and a detection window using a χ^2 test (Maaradji et al. [22]). In our framework, we compare states instead of runs. Therefore, this step works the same way in all three perspectives.

Our framework uses the four divergences of Section 3.5, which kairo implements as `kl`, `js`, `tv` and `hellinger` (Section 6.1).

Definition 5.4.1 (State Distributions, Drift Scores and Drift Signal)**.** Let $Z \in \mathbb{R}^{m \times d}$ be a reduced matrix with the rows $\mathbf{z}_1^\top, \ldots, \mathbf{z}_m^\top$ (Definition 5.2.1), let K be the set of states with $\mathit{state}(\mathbf{z}_j) \in K$ for $1 \leq j \leq m$ (Definition 5.3.1), and let signal windows of the length Δ_s be given (Definition 5.1.1).

- For $1 \leq j \leq m$, $\mathit{time}(\mathbf{z}_j)$ is the time of the row $\mathbf{z}_j$, i.e., the timestamp of its event in the intra-case perspective and the start of its feature window in the resource and inter-case perspectives.
- For a signal window v, $J_v = \{j \in \{1, \ldots, m\} \mid \mathit{time}(\mathbf{z}_j) \in v\}$ is the set of its rows.
- $v_1, \ldots, v_n$ are the signal windows with $J_{v_i} \neq \emptyset$ in chronological order, and t_i is the start of v_i.
- The state distribution of v_i is $p_i \in K \to [0, 1]$ with

$$p_i(\kappa) = \frac{|\{j \in J_{v_i} \mid \mathit{state}(\mathbf{z}_j) = \kappa\}|}{|J_{v_i}|} \quad \text{for } \kappa \in K.$$

- For $i_0 \leq i \leq n$, the reference q_i of v_i is one of the following distributions over K, where we take means of distributions separately for each $\kappa \in K$:
  - *previous signal window* (`previous`): $q_i = p_{i-1}$ with $i_0 = 2$,

  - *mean of the h previous signal windows* (`recent`): $q_i = \frac{1}{h} \sum_{l=i-h}^{i-1} p_l$ for an $h \in \mathbb{N}$ with $i_0 = h + 1$,
  - *mean of all signal windows* (`baseline`): $q_i = \frac{1}{n} \sum_{l=1}^{n} p_l$ with $i_0 = 1$.

- For a divergence D of Definition 3.5.1, $score_i = D(p_i, q_i)$ is the drift score of v_i for $i_0 \leq i \leq n$.
- The drift signal is the sequence $\langle (t_{i_0}, score_{i_0}), \ldots, (t_n, score_n) \rangle$.

Properties:

- Each divergence is 0 for $p_i = q_i$ (Section 3.5).
- A high drift score indicates that the state distribution of a signal window diverges from its reference.
- If a state from the signal window does not occur in the reference, the drift score based on the KL divergence $D_{\mathrm{KL}}(p_i, q_i)$ is infinite.
- The divergences only compare the shares of the states but not their similarities.
- A transition into a similar state, thus, counts as much as a transition to a very different state.

The reference determines which changes the drift score measures:

- *Previous signal window*: This reference compares neighboring signal windows. A sudden drift causes a spike in the first signal window after it. The drift score drops afterwards since the state shares of the signal windows are similar again. Random fluctuations can also cause a false positive spike in the drift score of a signal window.
- *Mean of the h previous signal windows*: This reference fluctuates less and is more stable. For each state, it is defined over the mean of that state's share in the h state distributions.
- *Mean of all signal windows*: This reference compares each signal window with the mean behavior of the entire event log. A drift shows itself more in the form of a step between two levels instead of a spike. This reference assumes complete event logs and is not applicable for online concept drift detection.

The size Δ_s of the signal window balances fluctuations against granularity in time. A short signal window has fewer rows and can have more random fluctuations in the state distributions and divergences. A long signal window smooths the frequencies but only signals a drift point as granular as Δ_s.

We should be skeptical about the drift scores at the end of an event log. The last signal window often only contains the remaining events of cases that started earlier, while no new cases arrive. Thus, only certain states occur, e.g., the states of late prefixes in the

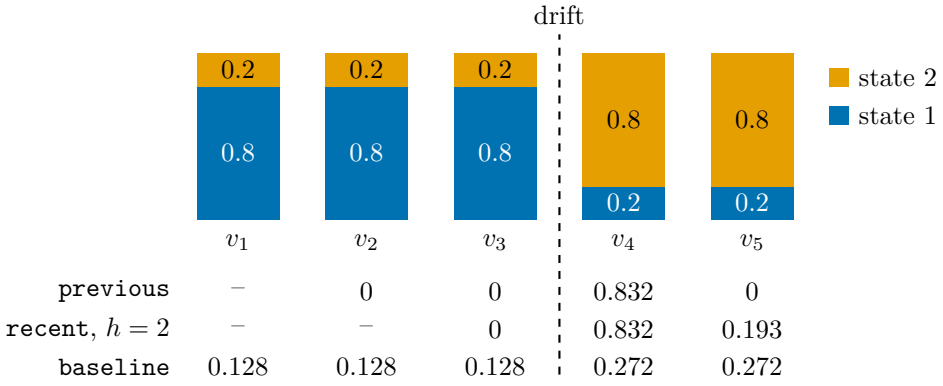


Figure 5.12: Drift signal of Example 5.4.2 with the states 1 (blue) and 2 (orange). Each bar is the state distribution of a signal window v_i with the shares of both states, and the dashed line marks a sudden drift between v_3 and v_4. The rows below give the KL drift score of each signal window against the three references of Definition 5.4.1: the previous signal window (`previous`), the mean of the $h = 2$ previous signal windows (`recent`) and the mean of all signal windows (`baseline`). A dash marks a signal window without a reference.

intra-case perspective, and the state distribution can be misleading without any drift. An example is the last intra-case signal window of the running example, which only holds the three events of December 31 (Figure 5.13).

Similar to the earlier steps of our pipeline, this step is also flexible. The framework is extendable to different divergences (e.g., Earth Mover's Distance) as well as to distances between the state distributions of the signal windows.

In the resource and inter-case perspectives, each row belongs to a feature window. Therefore, in these perspectives, we obtain a log-level trajectory of states from the clustering. In this case, the state vectors of the signal windows can also be compared directly using distance measures. In this thesis, we focus on our approach over frequencies and divergences. The main benefit of our choice is that it aligns the result format of the three perspectives into a drift signal.

The drift signal alone cannot decide whether there is a drift. In Chapter 7, we detect drifts with thresholds on the drift scores, which depend on the event log and the used divergence. Since each drift signal is a time series, other well-known drift detection approaches like PELT (Adams et al. [3]) or ADWIN (Baier et al. [5]) can be used with those drift signals.

Example 5.4.2. Figure 5.12 shows five signal windows $v_1, \ldots, v_5$ with the states $K = \{1, 2\}$. We write a distribution p over K as $(p(1), p(2))$.

- The state distributions are $p_1 = p_2 = p_3 = (0.8, 0.2)$ and $p_4 = p_5 = (0.2, 0.8)$. Thus, 60% of the rows move from state 1 to state 2 between v_3 and v_4.

- With the previous signal window as reference,

$$score_4 = D_{\mathrm{KL}}(p_4, p_3) = 0.2 \log \frac{0.2}{0.8} + 0.8 \log \frac{0.8}{0.2} \approx 0.832.$$

  All other drift scores are 0, and v_1 has no drift score.

- With the mean of the $h = 2$ previous signal windows as reference, v_4 also gets the drift score 0.832. The reference $q_5 = (0.5, 0.5)$ still contains v_3, so v_5 gets the drift score 0.193.

- With the mean of all signal windows $(0.56, 0.44)$ as reference, v_1 to v_3 get the drift score 0.128 and v_4 and v_5 the drift score 0.272. Thus, the drift shows as a step. Since only two signal windows follow the drift, they lie further from the reference.

5.4.1 Intra-Case Perspective

In the intra-case perspective, the rows are events. A signal window counts the states of all events whose timestamps lie in it, i.e., prefixes of many cases at different positions. Thus, the state distribution shows how often each prefix occurs in a signal window. A control-flow drift changes this mix as soon as cases of the new process version produce events.

Example 5.4.3. We compute two drift signals of the intra-case states of L_3 (Example 5.3.2) with the KL divergence (Figure 5.13).

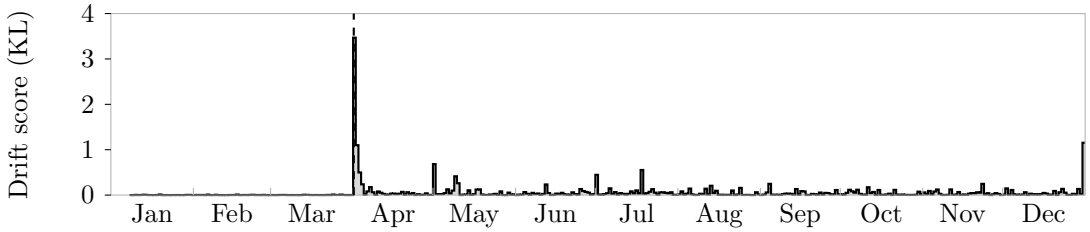
- First, we choose signal windows of one day and the mean of the seven previous signal windows as reference. The 366 signal windows hold 66 events in the median.

- Before the drift, all drift scores are at most 0.013. On April 1, the drift score reaches its highest value 3.47, since the four new cells of the SOM (Figure 5.9) do not occur in the reference.

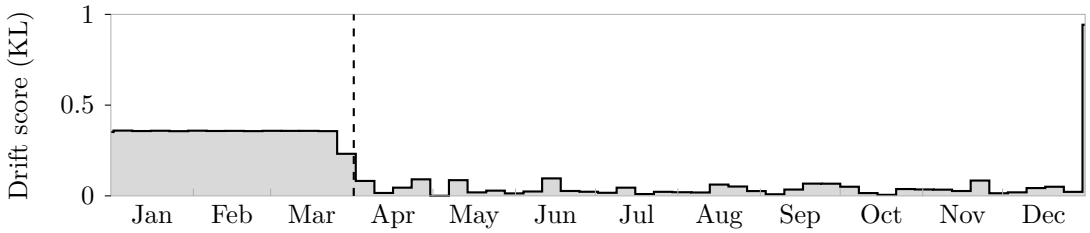
- With the previous signal window as reference, July 19 (7.98) and May 2 (5.52) exceed the drift. On July 18, all five cases execute c before b, and on May 1, all eight cases execute b before c. Thus, the cells of the other order are missing in the reference of the next day.

- Second, we choose signal windows of one week and the mean of all signal windows as reference. Before the drift, the drift scores lie between 0.35 and 0.36, and after it, their median is 0.028.

- Since 40 of the 54 signal windows lie after the drift, the reference is close to the new process version. Thus, the drift shows as a step downwards.



(a) Signal windows of one day, mean of the 7 previous signal windows as reference



(b) Signal windows of one week, mean of all signal windows as reference

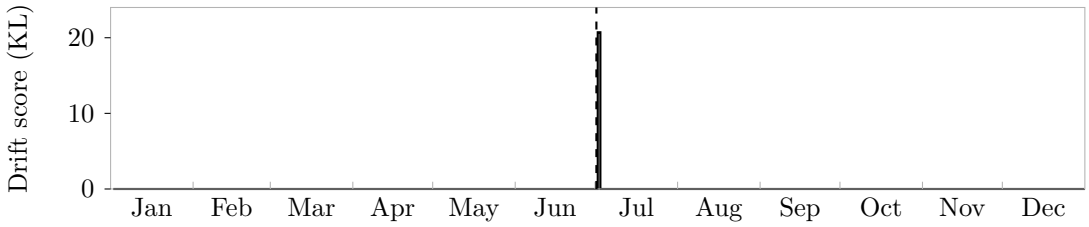
Figure 5.13: Drift signals of the intra-case states of the running example L_3 with the KL divergence (Example 5.4.3). Each step is a signal window at its drift score, cut at the bounds of the year 2020. The dashed line marks the control-flow drift on April 1, 2020, 06:00. (a) The first seven signal windows have no reference. The drift gets the highest drift score of 3.47. (b) The signal windows before the drift lie far from the mean of all signal windows and those after it close to it, so the drift shows as a step. The last signal window only holds the three events of December 31.

5.4.2 Resource Perspective

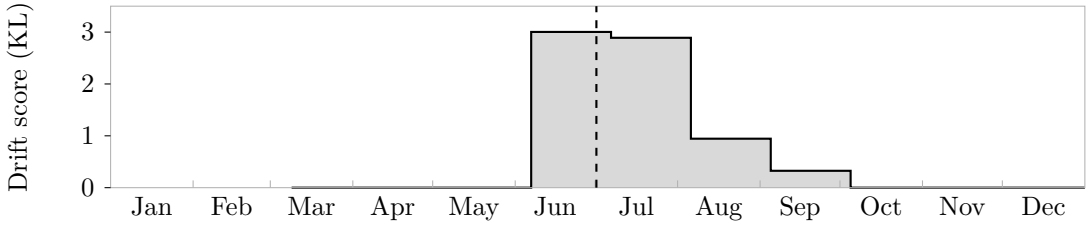
In the resource perspective, the rows are feature windows. Thus, a signal window holds far fewer rows than in the intra-case perspective. If $\Delta_s = \Delta_f$, each signal window holds exactly one feature window. Then, the drift signal with the previous signal window as reference shows exactly the state changes in the trajectory of the event log.

Example 5.4.4. We compute two drift signals of the resource states of L_3 (Example 5.3.3) with the KL divergence (Figure 5.14).

- First, we choose signal windows of one day and the previous signal window as reference. All drift scores are 0 except the one of July 2, the first feature window in state 2.
- State 2 does not occur in the reference, so the KL divergence is infinite. Our implementation adds a share of 10^{-9} to each state, so the drift score is $\log 10^9 \approx 20.72$ instead.
- With the mean of all signal windows as reference, each drift score is $\log 2 \approx 0.69$, since both states cover 183 days. Thus, the drift disappears from the drift signal.



(a) Signal windows of one day, previous signal window as reference



(b) Signal windows of 30 days, mean of the 3 previous signal windows as reference

Figure 5.14: Drift signals of the resource states of the running example L_3 with the KL divergence (Example 5.4.4). Each step is a signal window at its drift score, cut at the bounds of the year 2020, and signal windows without a reference have no drift score. The dashed line marks the reassignment drift on July 1, 2020, 12:00. (a) Each signal window holds one feature window. The only drift score above 0 is $\log 10^9 \approx 20.72$ on July 2, the first day in state 2. (b) The drift lies in the signal window from June 7 to July 6 and spreads over two signal windows. Afterwards, the drift score decays over three signal windows.

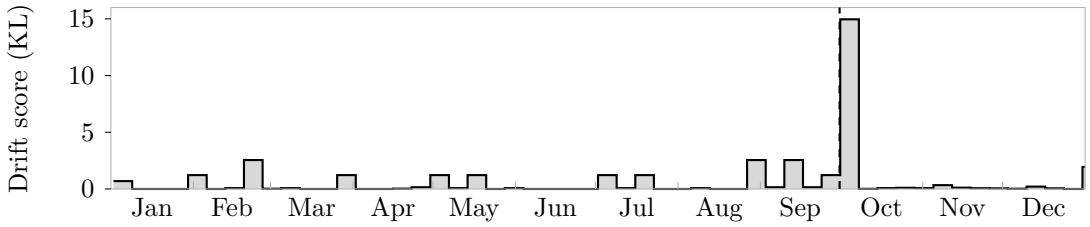
- Second, we choose signal windows of 30 days and the mean of the three previous signal windows as reference. The signal window from June 7 to July 6 contains the drift with 25 days in state 1 and 5 days in state 2 and reaches 3.00. The next signal window lies completely in state 2 and reaches 2.89.

- Afterwards, the drift score decays to 0.94 and 0.33, since the reference still contains signal windows in state 1.

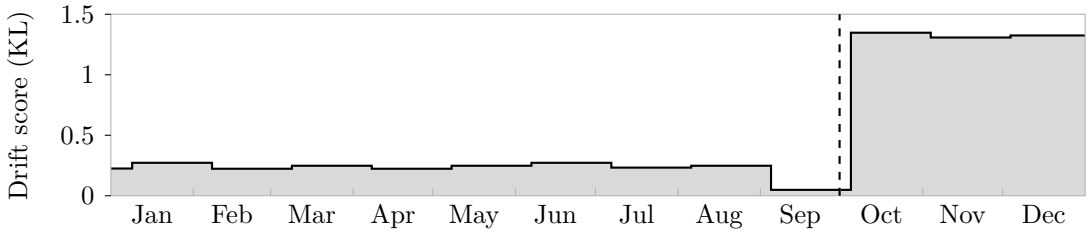
5.4.3 Inter-Case Perspective

As in the resource perspective, a signal window counts the states of its feature windows. With DBSCAN, noise is also a state (Definition 5.3.1). Thus, unusual feature windows change the state distribution also without a drift.

Example 5.4.5. We compute two drift signals of the inter-case states of L_3 (Example 5.3.4) with the KL divergence (Figure 5.15).



(a) Signal windows of one week, previous signal window as reference



(b) Signal windows of 30 days, mean of all signal windows as reference

Figure 5.15: Drift signals of the inter-case states of the running example L_3 with the KL divergence (Example 5.4.5). Each step is a signal window at its drift score, cut at the bounds of the year 2020. The dashed line marks the amount drift on September 30, 2020, 18:00. (a) The first signal window after the drift gets the drift score 14.96. The smaller steps come from feature windows that DBSCAN marks as noise. (b) The three signal windows after the drift lie far from the mean of all signal windows, so the drift shows as a step upwards.

- First, we choose signal windows of one week and the previous signal window as reference. The 54 signal windows hold 14 feature windows in the median.
- The signal window from October 1 reaches the highest drift score 14.96. In this signal window, 71.4% of the feature windows lie in state 2, which does not occur in the previous signal window.
- Ten further signal windows exceed 1. Except the last one, they contain one or two noise feature windows after a signal window without noise. The drift is still 5.9 times higher than the highest of them (2.55).
- Second, we choose signal windows of 30 days and the mean of all signal windows as reference. Before the drift, the drift scores lie between 0.22 and 0.27, and after it between 1.31 and 1.35.
- Since the amount drift lies at three quarters of the year, only three of the 13 signal windows follow the new process version. They lie far from the reference, so the drift shows as a step upwards, unlike in the intra-case perspective.

Table 5.7: Configuration of the running example L_3 per perspective. Feature groups are given by their identifiers (Definitions 5.1.4, 5.1.6 and 5.1.8). The drift signals use the KL divergence and give the length of the signal windows and the reference (Definition 5.4.1).

Step	Intra-case	Resource	Inter-case
Rows	24,600 events	366 windows of 1 day	731 windows of 12 h
Feature groups	`act_freqs`, `df_counts`, `current_act`	`res_events`, `act_res_shares`	`active_cases`, `new_arrivals`, `mean_delta_t`, `attr_means`
Columns	20	24	4
Scaling	min-max	z-score	z-score
PCA	$d = 6$	$d = 1$	none
Clustering	SOM, 3×3	k-means, $k = 2$	DBSCAN, $\varepsilon = 0.7$, $minPts = 8$
States	9	2	2 and noise
Drift signals	1 day, `recent`, $h = 7$; 7 days, `baseline`	1 day, `previous`; 30 days, `recent`, $h = 3$	7 days, `previous`; 30 days, `baseline`

The examples show how the reference and the length of the signal windows shape the drift signal. In all three perspectives, the drift creates a state that does not occur before. Therefore, with the previous signal window or the mean of the previous signal windows as reference, the drift signal reaches its highest value in the first or second signal window after the drift. Exceptions are short signal windows with only a few cases or feature windows, whose random fluctuations reach or exceed the drift. With the mean of all signal windows as reference, the drift shows as a step, and the dominant process version decides its direction. Chapter 7 evaluates which configurations signal the known drift points in Rheon logs. Table 5.7 summarizes the configuration of all steps for the three perspectives of the running example.

6 Implementation (Drift Detector)

In this chapter we describe the implementation of our framework from Chapter 5. Our implementation comprises a Python library kairo and a dashboard built on top of kairo. Both are placed in the public repository of this thesis[1]. We introduce the structure of the code and design choices of the implementation.

An LLM supports the analysis in two ways (Figure 6.1). It is not a further step of the pipeline and does not do any computations.

- *Analysis of results:* The Copilot of the dashboard is used to interpret the results of the pipelines. Its main purpose is to assist the user. The Copilot allows the user to chat with an LLM on the analysis by sharing the results of the computations of the pipelines, plots and textual abstractions (Section 6.2.2).
- *Agentic analysis:* An agent executes the pipeline itself by using kairo and tests different configurations (Section 6.1.2).

Thus, this chapter addresses the research goals RG4 and RG5.

Section 6.1 presents kairo and our skill `kairo-drift-analyst` for the agentic analysis. Section 6.2 presents our dashboard and the Kairo Copilot.

6.1 Kairo

kairo is a Python library that implements the four steps of the framework for all three perspectives. Its name comes from the Greek *kairos*, the turning point. The documentation[2] describes the approach, gives a quickstart and explains all functions. The nine notebooks[3] introduce the pipelines for each combination of perspectives and clustering algorithms. In the repository root, `uv sync` installs kairo with dependencies.

6.1.1 Running the Pipeline in Python

Our implementation follows three main design principles:

- *Modular function structure:* Each step is a function, e.g., `compute_features_inter`, `standardize`, `compute_kmeans` and `compute_drift_signal`. Their parameters are the hyperparameters of Chapter 5, e.g., the feature groups, the feature window or the divergence.

[1] `https://github.com/boraderen/thesis`
[2] `https://boraderen.github.io/thesis/`
[3] `https://github.com/boraderen/thesis/tree/main/notebooks/kairo_examples`

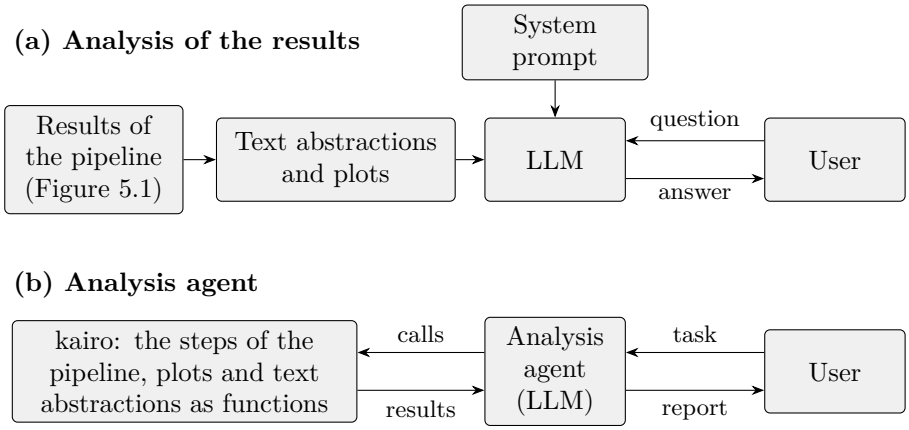


Figure 6.1: Two uses of an LLM on top of the framework. Boxes are components, and arrows show what they pass on. (a) The results of a pipeline reach the LLM as text abstractions and plots, and a system prompt instructs the LLM. The user and the LLM interact in a chat. (b) An analysis agent runs the pipeline itself by calling the functions of kairo and reads their results.

- *DataFrames:* The log, the feature matrices, the states and the drift signal are pandas DataFrames (Figure 6.2). The meta columns `case:concept:name` and `time:timestamp` travel along through all steps, but do not enter the computations. Thus, steps such as scaling and PCA can be skipped, and own features can replace the feature groups of kairo.
- *Results as plot and text abstractions*: For the results of each step, the functions `plot_*` create plots, and the functions `abstract_*` summarize them as text for an LLM (Section 6.1.2).

These design choices have two advantages and two disadvantages:

- *Flexibility:* Steps can be skipped or exchanged, e.g., by own features or by an own clustering whose states enter the drift signal like the states of kairo.
- *Traceability:* Each intermediate result is a DataFrame with a plot and a text abstraction, so the user can check each step.
- *Large hyperparameter space:* The user chooses the features, the windows, the scaling, the PCA cut, the clustering algorithm with its hyperparameters and the drift signal. This hyperparameter space is too large to test all configurations, and an unsuitable configuration can produce misleading drift signals. The Copilot and the analysis agent help the user to find a suitable configuration.
- *Computational effort:* kairo keeps the log and the feature matrices in memory. For large logs, single steps become slow, e.g., the k-distances for DBSCAN on the intra-case features, which have one row per event.

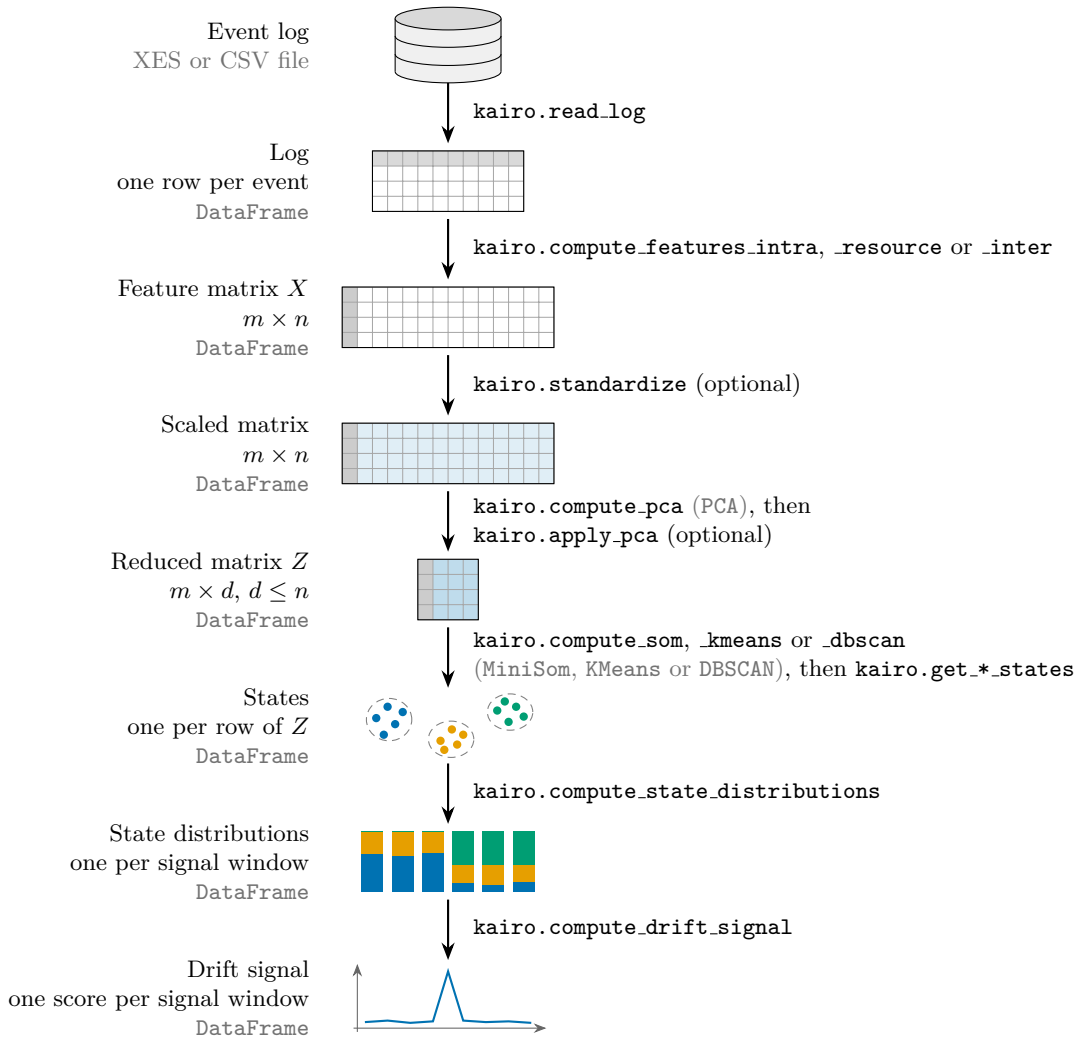


Figure 6.2: Workflow of kairo for one perspective. Each arrow is a function of kairo that takes the output above it and returns the output below it. Gray text gives the type of each output: except for the event log, every output is a pandas DataFrame, and fitting PCA or a clustering returns the fitted model, which the next function applies. The gray first column of the matrices stands for the meta columns, which travel along but do not enter the computations.

```
Drift signal: for every signal window, the KL divergence between its state
distribution and the previous signal window, the drift score of the signal window.
54 signal windows from 2019-12-26 00:00:00+00:00 to 2020-12-31 00:00:00+00:00.
Drift scores: median 0.074, mean 0.678, highest 14.958.
The signal windows with the highest drift scores:
- signal window starting 2020-10-01 00:00:00+00:00: 14.958
- signal window starting 2020-02-20 00:00:00+00:00: 2.550
- signal window starting 2020-08-27 00:00:00+00:00: 2.550
- signal window starting 2020-09-10 00:00:00+00:00: 2.550
- signal window starting 2020-12-31 00:00:00+00:00: 1.946
```

Figure 6.3: Text abstraction of the inter-case drift signal of the running example L_3 with signal windows of one week and the previous signal window as reference, as the LLM reads it. Long lines are wrapped. The abstraction names the divergence and the reference, summarizes the 54 drift scores and lists the five signal windows with the highest drift scores.

6.1.2 Agentic Analysis

Since kairo consists of small, documented functions with DataFrames as input and output, an analysis agent can run the pipeline itself (Figure 6.1). By an analysis agent, we mean an LLM with tools to read files and to run code. The agent calls the functions of kairo, combines them, tests configurations and computes own features if needed.

The agent reads the results as text abstractions. According to Berti et al. [8], a complete event log cannot be given to an LLM as a prompt due to its size. Therefore, they implement textual abstractions of process mining artifacts, e.g., of event logs and process models, in PM4Py. Similarly, each function `abstract_*` of kairo summarizes one result as text. An abstraction only names what matters for the interpretation, e.g., the signal windows with the highest drift scores. Figure 6.3 shows an example of an abstraction.

Our skill `kairo-drift-analyst`[4] gives an agent the instructions for such an analysis. The prompt names the event log and the task, e.g., to find the drift points and their perspectives. The agent reads the thesis and the documentation of kairo, runs and compares configurations on its own and reports the drifts with their perspectives, modes and what changed. For each perspective, it reports a configuration with a threshold on the drift scores.

6.2 Dashboard

Our dashboard[5] is a Streamlit app built on top of kairo. In the repository root, `uv run streamlit run dashboard/app.py` starts the dashboard locally. The sidebar lists its pages: the upload page, four pages for each perspective and the Copilot.

[4] `https://github.com/boraderen/thesis/tree/main/skills/kairo-drift-analyst`
[5] `https://thesis-kairo-dashboard.streamlit.app/`

6.2.1 Running the Pipeline in the Dashboard

On the upload page, the user uploads an event log as an XES or CSV file of up to 1 GB. By clicking on the column headers, the user assigns the columns to their roles, e.g., the case identifier, the activity and the timestamp. Then, the user picks the case attributes for the inter-case features and states whether each one is numerical or categorical.

Each perspective has four pages, one for each step: Features, PCA, States & Trajectories and Drift Signal. In the sidebar, the user chooses the hyperparameters of a step, e.g., the clustering algorithm and its hyperparameters in Figure 6.4. Each page shows the results with the plots and tables of kairo:

- *States:* The page shows the clustering measures of Section 3.4.4 as well as the colors of the states and the distances between them (Figure 6.4). It also compares the state frequencies of two date ranges, e.g., before and after a suspected drift.
- *Trajectories:* In the intra-case perspective, the page shows the trajectory of a single case and the trajectories of all cases in a date range. In the resource and inter-case perspectives, it shows the trajectory of the log with a table of its visits (Figure 6.5).
- *Drift signal:* The page shows the drift signal and the state distribution of each signal window (Figure 6.6). It also shows the mean and a chosen quantile of the drift scores and of the rows per signal window, and plots the rows of every signal window.

The user tries out configurations step by step and computes the perspectives one after another. This follows the human-in-the-loop process of Barbon Junior et al. [6], in which a human observes the emergence of drifts over time and adjusts the hyperparameters.

6.2.2 Kairo Copilot

Kairo Copilot is the page of the dashboard on which an LLM analyzes the results together with the user (Figure 6.1). We see the Copilot as an extension of our framework and not as a step of the pipeline. It computes neither states nor drift signals, but interprets the results of the pipeline. The user picks the abstractions and plots of all steps computed so far, which the LLM receives, and asks questions (Figure 6.7). The analysis is interactive: the user asks follow-up questions and requests further steps. In this way, the LLM supports the characterization of a drift (RQ3).

- *LLM connector:* The class `LLMConnector` of kairo has connectors for Claude and for OpenAI-compatible APIs and supports custom providers. The function `create_messages` brings the abstractions and the plots, which are attached as images, into the message format of the provider.
- *System prompt:* The default system prompt of kairo explains the method, the feature groups and the strengths and limits of each perspective to the LLM. It instructs the LLM to only use the shared results and never to invent numbers.

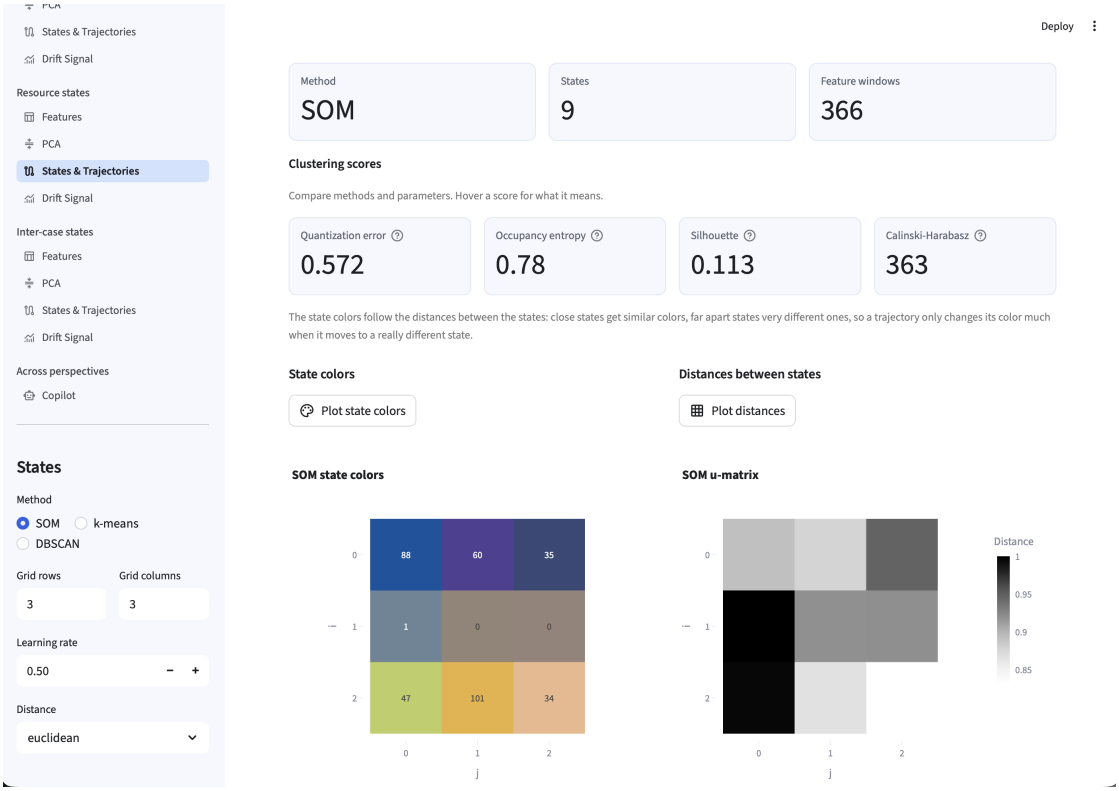


Figure 6.4: Upper part of the States & Trajectories page of the dashboard for the resource perspective of the running example L_3. The 366 feature windows of one day have the feature groups `res_events`, `res_cases` and `act_res_shares`. They are min-max scaled and reduced to 10 principal components, which explain 95.2% of the variance. In the sidebar, the user chooses a 3×3 SOM. The page shows the clustering measures, the color of each cell with its number of feature windows and the U-matrix. Cells are counted from 0. The cells $(2, 0)$ to $(2, 2)$ hold the 182 days before the reassignment drift, and the cells $(0, 0)$ to $(0, 2)$ the 183 days after it. Since each process version spreads over three close cells, the silhouette is only 0.113.

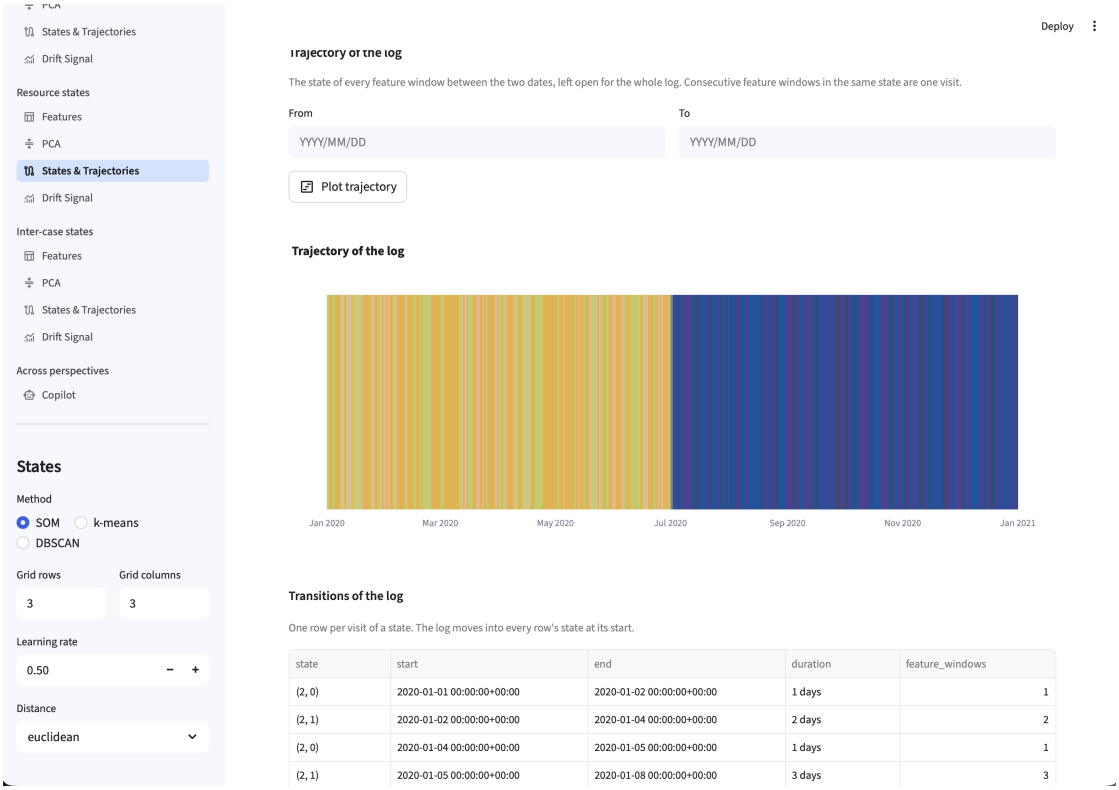


Figure 6.5: Lower part of the States & Trajectories page of Figure 6.4. Without dates, the user plots the trajectory of the whole log. Each stripe is a feature window in the color of its state. At the reassignment drift on July 1, 2020, the colors change from yellow and orange to blue and purple. The table lists each visit of a state with its start, end, duration and number of feature windows.

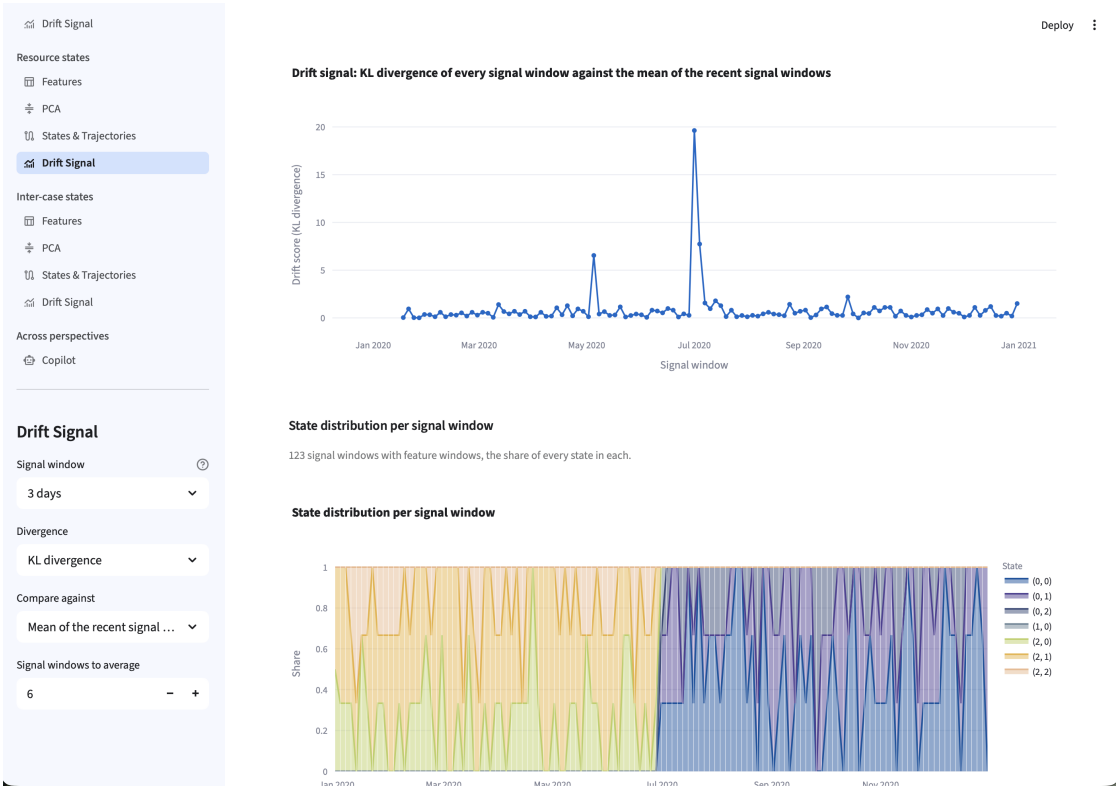


Figure 6.6: Drift Signal page of the dashboard for the resource states of Figure 6.4. In the sidebar, the user chooses signal windows of three days, the KL divergence and the mean of the six previous signal windows as reference. The drift signal reaches its highest drift score of 19.62 in the signal window from July 1, 2020, which contains the reassignment drift. The smaller spike of 6.54 in the signal window from May 5 comes from state $(2, 2)$, which does not occur in the six previous signal windows. Below, the state distributions of the 123 signal windows show the change from the cells of row 2 to the cells of row 0.

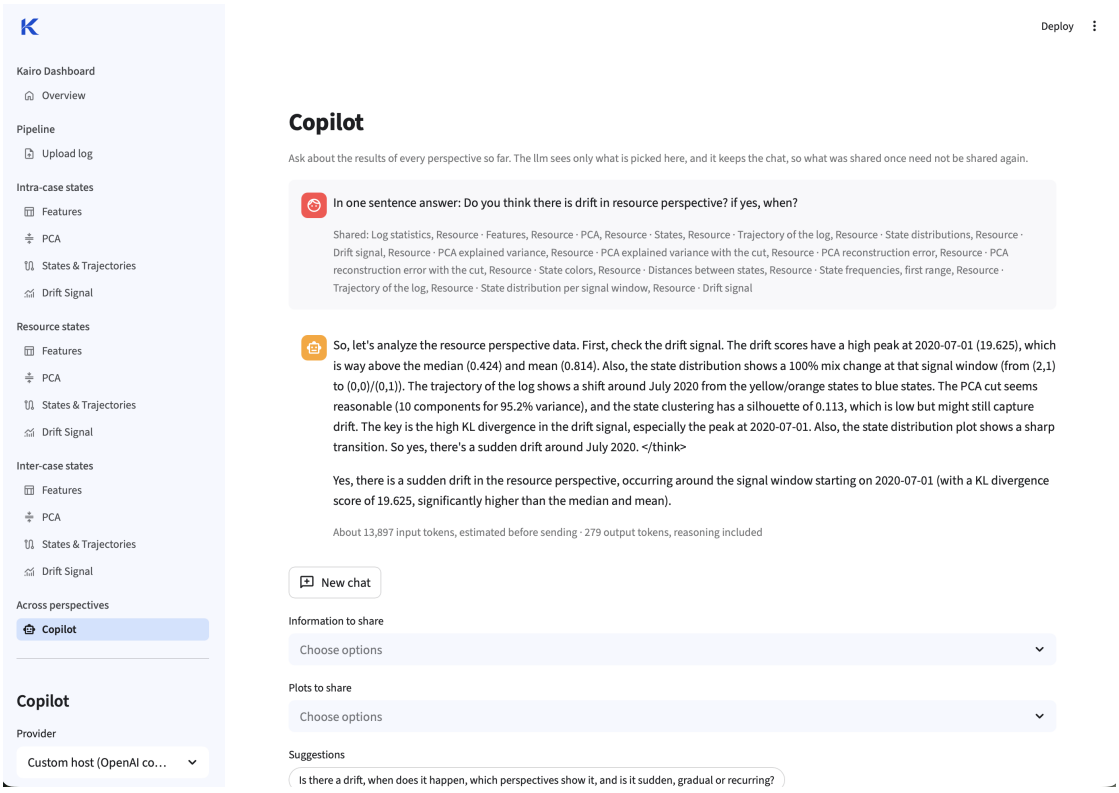


Figure 6.7: Copilot page of the dashboard for the resource perspective of the running example L_3. The user shares the log statistics as well as the abstractions and plots of all steps of the resource perspective of Figures 6.4 to 6.6 and asks whether there is a drift in the resource perspective and when. The model Qwen3-VL-30B-A3B-Thinking, a custom provider in the sidebar, answers with its reasoning, which ends with `</think>`, and then names a sudden drift in the signal window from July 1, 2020.

Further, it tells the LLM to judge the configuration before the drift signal and to stay skeptical of drift signals that contradict each other.

- *Chat history:* The Copilot keeps the chat history and sends it with each question, so the LLM remembers earlier questions and the results that the user shared.

In Figure 6.7, the model Qwen3-VL-30B-A3B-Thinking runs as a custom provider on the GPU cluster of the Chair of Process and Data Science (PADS). With the abstractions and plots of the resource perspective of Figures 6.4 to 6.6, it names the reassignment drift in the signal window from July 1, 2020. However, an LLM can hallucinate and interpret results in a misleading way. Therefore, we suggest using it as an assistant in a human-in-the-loop analysis and staying skeptical of its claims (Chapter 8).

7 Evaluation

The main handicap of our framework is its flexibility and large hyperparameter space. That has the consequence that for the same event log two different configurations of a perspective might have a very different performance in catching the drifts or not catching the non-drifts.

In this chapter we will evaluate our framework from Chapter 5. The goal is to show where the framework is strong and where it has limitations. In our evaluation we use the logs of the Padella benchmark (Section 7.1) and three sets of synthetic Rheon logs (Section 7.2).

Our framework is hard to evaluate in the classical sense:

- *Configuration:* Each log and each perspective needs a specific configuration and a threshold. A result, therefore, always assesses the framework together with the choice of configuration.
- *Metrics:* Precision, recall and F1-score are in some cases meaningless. For a log without injected drifts, recall and F1-score are undefined, and the precision is 0 if there is a detection. Real but unknown changes of a log and reactions of other perspectives count as FPs, even if the states "signal" the real change.
- *Goal:* Instead of evaluating the framework in the classical sense, we will show that it can be powerful but also misleading. That is, we compare two runs: Run 1 shows which configurations agents find using our skill (Section 6.1.2) and Run 2 reveals that appropriate configurations excel in detecting drifts. The real problem is finding those configurations.

Since the output for a log of our framework is a drift signal over signal windows per perspective, we need to align it with the log and its real drift points. In order to do our evaluation, we first formalize it and introduce an experimental methodology. Figure 7.1 shows an example of our experimental methodology and describes how it compares a drift signal and a log with drifts.

- *Runs:* A run describes for each log and evaluated perspective a configuration and a threshold τ for its drift scores of the signal windows.
- *Signal windows:* The signal windows of a drift signal (Definition 5.4.1) are the unit of our evaluation. We only evaluate scored signal windows. A signal window is not scored if it has no reference or if it is the first or the last signal window and the log covers it only partially. A thin signal window, i.e., one with fewer than a quarter of the median number of rows, is not scored either.

- *Detections:* A detection is a sequence $v_i, \ldots, v_j$ of consecutive scored signal windows with $score_l \geq \tau$ for $i \leq l \leq j$ that cannot be extended. The detection lies in its first signal window v_i. Thus, a drift that keeps the drift scores high over several signal windows gives only one detection.
- *Truth events and zones:* Each injected drift is a truth event of the perspective that is expected to catch it. If the drift point of a sudden drift lies in v_k, its zone is v_{k-1}, v_k, v_{k+1}. If a gradual drift starts in v_s and ends in v_e, its zone is $v_{s-1}, \ldots, v_{e+1}$.
- *Counts:* A truth event is a True Positive (TP) if its zone contains a detection, otherwise it is a False Negative (FN). Every detection outside of the zones is a False Positive (FP), and we ignore further detections in a zone. Every scored signal window outside of the zones without a detection is a True Negative (TN). A perspective without truth events has no zones, so each of its detections is an FP.
- *Metrics:* We sum up the counts of all logs of a run, also separately for each drift type and drift mode, and compute

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN} \quad \text{and} \quad F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}.$$

TP and FN count drifts, FP counts detections and TN counts signal windows. The zone allows a detection one signal window before or after the drift (Section 5.4). We compare two runs on the Padella benchmark and on each set of Rheon logs:

- *Run 1:* Agents with our skill `kairo-drift-analyst` create configurations and thresholds (Section 6.1.2). On the Padella benchmark, agents using the models GPT-6.1 Sol analyze 22 logs, GPT-5.6 Luna seven logs and Claude Sonnet-5.5 five logs. All Rheon logs are analyzed by agents using the model GPT-6 Luna.
- *Run 2:* All configurations and thresholds are hand-tuned. Run 2 reveals that configurations exist in which the states detect the drifts.
- *Clustering:* Figure 7.2 shows how often their configurations use each clustering algorithm.

The logs and the configurations and thresholds of all runs are available on Google Drive[1].

7.1 Padella Benchmark

The Padella benchmark [24] injects drifts into five real base logs (Table 7.1). We use their five baselines without a drift and the 29 logs with one injected drift each, but not the logs with recurring drifts:

[1] `https://drive.google.com/drive/folders/1sie4iFTUNmHuyvMVoW8tktyvtoUzbmEp`

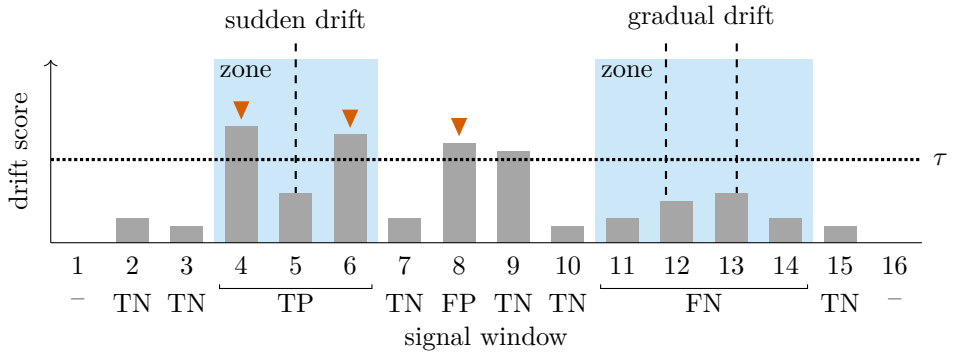


Figure 7.1: Example of the evaluation of one drift signal with the threshold τ (dotted) in a perspective that is expected to catch a sudden and a gradual drift. Each bar is the drift score of a signal window, numbered below. The first and the last signal window are not scored (–). Dashed lines mark the drift point of the sudden drift and the start and end of the gradual drift. The zones (blue) add one signal window on both sides. Triangles mark detections, i.e., the first signal window of each sequence of signal windows with a drift score of at least τ. The detection in signal window 4 makes the sudden drift a TP, and the detection in signal window 6 is ignored. The gradual drift has no detection in its zone and is an FN. The detection in signal window 8 is an FP. Signal window 9 continues its sequence and, like all other scored signal windows outside the zones, is a TN. Thus, TP = 1, FN = 1, FP = 1 and TN = 6, which gives a precision, recall and F1 of 0.5.

- *Drifts:* Each injected drift changes one activity in 35 to 70% of the cases. The 15 sudden and 14 gradual drifts have three drift types. Nine `cf` drifts delete, replace or insert an activity. Ten `time_fast` drifts multiply the duration of an activity by 0.1 to 0.5 and ten `time_slow` drifts by 2 to 6, without changing the waiting times.

- *Perspectives:* The `cf` drifts are truth events of the intra-case perspective and the time drifts of the resource perspective (Figure 7.3). We only evaluate these two perspectives.

- *Result:* Run 1 detects 19 of the 29 drifts, but has 75 FPs and only reaches an F1-score of 0.309. Run 2 detects 28 of the 29 drifts with 8 FPs and reaches an F1-score of 0.862 (Figure 7.4 and Table 7.2). This shows the main handicap of our framework: on the same logs, two configurations of a perspective perform very differently.

- *Precision of Run 1:* The low F1-score of Run 1 mainly follows from its precision of 0.202. With a recall of 0.655, Run 1 detects two thirds of the drifts, but the 75 FPs push its F1-score down. With the precision of 0.778 of Run 2, Run 1 would reach an F1-score of 0.711 at the same recall.

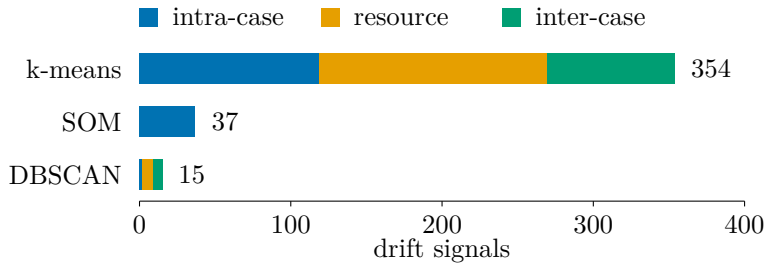


Figure 7.2: Number of drift signals of all runs per clustering algorithm and perspective.

Table 7.1: The baselines of the five Padella base logs with their number of cases and events, size, number of activities (Act.), resources (Res.) and trace variants (Var.), time span in days and average throughput time (TPT) in days of a case from the start of its first to the completion of its last event. The drifted versions of a base log have the same cases. Their `cf` drifts change the events and trace variants, and their time drifts the throughput times, which range from 7.9 to 21.2 days over all 34 logs. `time_slow` drifts extend the time span of a log up to 4.5 years.

Base log	Cases	Events	MB	Act.	Res.	Var.	Days	TPT
Bank account closure	32,429	212,721	108.3	15	654	182	646	15.5
Domestic declarations	10,500	56,437	27.7	17	2	99	889	11.5
Loan application	8,049	51,059	20.2	6	58	1,829	165	7.9
Purchase order handling	8,000	35,738	22.4	33	292	433	53	15.2
Request for payment	6,886	36,796	19.8	19	2	89	941	12.0

The FPs of Run 1 have two main causes that the benchmark does not know:

- *Real changes:* Most of the FPs of Run 1 are not mistakes of the states. Real processes are not static even without an injected drift (first-order dynamics), and the baselines alone already get 13 FPs in Run 1. In the logs of Domestic declarations and Request for payment, the pre-approver also stops approving after December 21, 2017, and from January 2018 the administration and the budget owner approve instead. Run 1 detects this real change, but the benchmark does not know it, so each detection counts as an FP. 24 of the 30 FPs of Run 1 in these 14 logs lie at this turn of the year, whereas Run 2 has only one there (Figure 7.5).

- *Case arrivals:* The logs of Bank account closure also change without an injected drift (Figure 7.6). Until August 2018, almost no case ends, and the few cases that start until then run for 99 days in the median. From September 2018 to January 2019, however, 4,600 to 6,700 cases start and end per month, which run for six days in the median. The log probably only contains the cases that end in a certain period, so that the number of cases jumps in September 2018. The benchmark does not know this change. 7 of the 14 FPs of Run 1 in the six logs of Bank

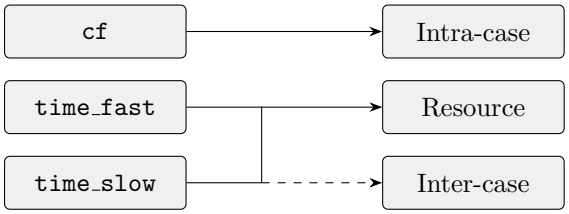


Figure 7.3: Perspectives that are expected to catch the drift types of the Padella benchmark (solid arrows). The inter-case perspective also responds to time drifts (dashed), but we do not evaluate it on the Padella logs.

	drift	no drift
drift	19	10
no drift	75	1,284

(a) Run 1

	drift	no drift
drift	28	1
no drift	8	1,522

(b) Run 2

Figure 7.4: Confusion matrices over all 34 Padella logs and both evaluated perspectives for both runs, with TP and FN in the first row and FP and TN in the second row. Rows: actual, columns: detected. TP and FN count injected drifts, FP counts detections and TN counts signal windows.

account closure lie in the signal window from September 6, 2018, whereas Run 2 has none there.

- *Implication:* With suitable configurations, the states separate the injected drifts almost completely from the natural behavior of the real logs. However, they also show real changes that the benchmark does not know, and each such detection lowers the precision.

Per drift type and drift mode, the runs show the following:

- *Control-flow drifts (`cf`):* We expect the intra-case states to show the `cf` drifts directly, since these drifts change the activities of the cases (Figure 5.3). Both runs confirm this with a recall of 0.889 and 1.0, but Run 1 has 26 FPs and Run 2 only two. For example, in the Bank account closure log the activity `Pending Request for Reservation Closure` disappears on November 26, 2018. Run 1 only uses the current activity as feature, so that each of its 15 states stands for one activity. The share of the state of this activity drops from 13.6% in the signal window from November 1 to 0.2% in the signal window from January 24, 2019. Thus, the intra-case states reliably catch deleted, inserted and replaced activities, and with one state per activity they directly show which activity changes.

- *Fast time drifts (`time_fast`):* We expect the resource states to show the `time_fast` drifts through the mean duration of the events of each resource, but not the

Table 7.2: Counts, precision, recall and F1 of both runs on the Padella benchmark, over all logs and per drift type and drift mode. Each group pools the counts of both perspectives. The baselines have no drift, so that their precision, recall and F1 are undefined (–).

Group	Run	TP	FP	FN	TN	Precision	Recall	F1
All	1	19	75	10	1,284	0.202	0.655	0.309
	2	28	8	1	1,522	0.778	0.966	0.862
`cf`	1	8	26	1	318	0.235	0.889	0.372
	2	9	2	0	386	0.818	1.000	0.900
`time_fast`	1	5	22	5	363	0.185	0.500	0.270
	2	10	2	0	423	0.833	1.000	0.909
`time_slow`	1	6	14	4	376	0.300	0.600	0.400
	2	9	4	1	465	0.692	0.900	0.783
Baselines	1	0	13	0	227	–	–	–
	2	0	0	0	248	–	–	–
Sudden	1	9	36	6	563	0.200	0.600	0.300
	2	14	5	1	636	0.737	0.933	0.824
Gradual	1	10	26	4	494	0.278	0.714	0.400
	2	14	3	0	638	0.824	1.000	0.903

intra-case states, since their features do not encode time. Run 1 only detects 5 of the 10 drifts with 22 FPs (F1-score 0.270), whereas Run 2 detects all ten with two FPs (F1-score 0.909). For example, in the Loan application log a sudden drift multiplies the duration of the activity `W_Completeren aanvraag` by 0.1. The resource perspective of Run 1 only clusters the number of events per resource, which does not encode durations, and misses the drift. Run 2 uses the durations per resource and detects it. Thus, a drift that no feature of the configuration captures stays hidden in the states (Section 5.1).

- *Slow time drifts (`time_slow`):* Since the `time_slow` drifts multiply a duration by 2 to 6, we expect them to be easier to detect than the `time_fast` drifts. This holds for Run 1 with a recall of 0.6 against 0.5. In Run 2, however, `time_slow` is the weakest drift type with an F1-score of 0.783. Run 2 only misses the sudden `time_slow` drift in the Domestic declarations log, which multiplies the duration of an activity by 4, and has four FPs on these logs. For example, in the Request for payment log a sudden drift on May 8, 2018, multiplies the duration of an activity by 5. Both perspectives of Run 1 detect the real change at the turn of the year 2017/18 instead. Since a run has only one configuration and one threshold per perspective, a stronger real change can hide an injected drift of the same perspective.

- *Sudden drifts:* A sudden drift changes the state distribution from one signal window to the next (Section 5.4). Therefore, we expect sudden drifts to be easier to detect. However, Run 1 only detects 9 of the 15 sudden drifts (F1-score 0.300) and Run 2 detects 14 (F1-score 0.824). Both runs detect all sudden `cf` drifts, but

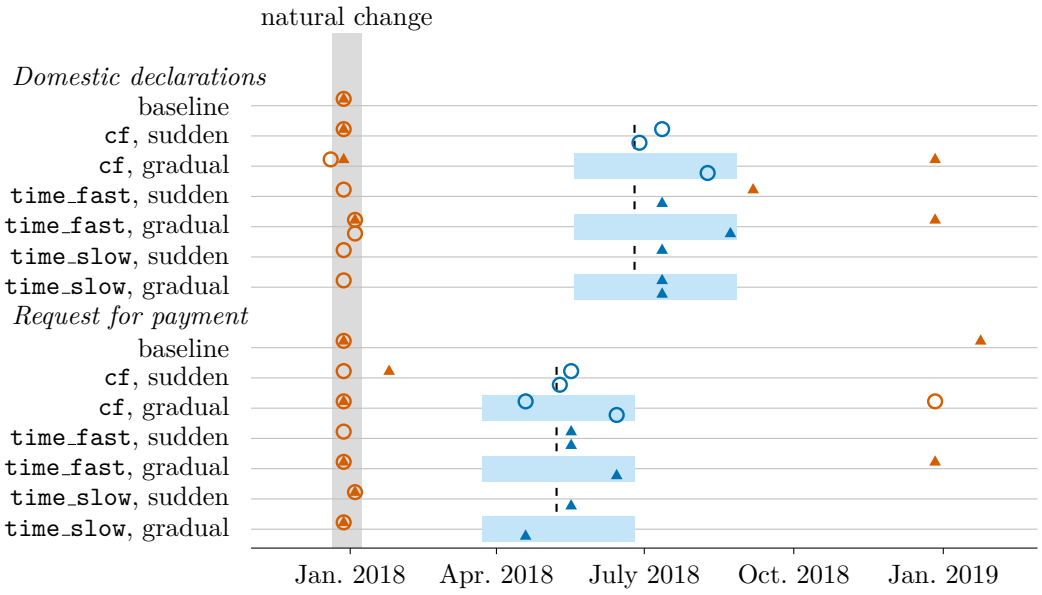


Figure 7.5: Detections in the 14 logs of the base logs Domestic declarations and Request for payment, with Run 1 in the upper and Run 2 in the lower lane of each row. Circles are intra-case and triangles resource detections at the start of their signal window, blue in the zone of the injected drift and orange as FP. Dashed lines mark sudden drift points and light blue areas gradual drift windows. The gray band marks the natural change from December 21, 2017, to January 8, 2018, at which 24 of the 30 FPs of Run 1 lie.

Run 1 misses six sudden time drifts. Moreover, the zone of a sudden drift only covers three signal windows, whereas the zone of a gradual drift covers 5.3 signal windows on average in Run 1. Thus, the longer zones favor the gradual drifts.

- *Gradual drifts:* Run 1 detects 10 of the 14 gradual drifts (F1-score 0.400) and Run 2 all 14 (F1-score 0.903). Contrary to our expectation, gradual drifts therefore perform better than sudden drifts. For example, Run 1 misses the gradual `cf` drift in the Domestic declarations log with signal windows of 30 days and the previous signal window as reference. Run 2 detects it with signal windows of 56 days and the mean of the three previous signal windows as reference. In this example, longer signal windows with a reference over several signal windows show the gradual change better.

7.2 Rheon Benchmark

The Rheon logs contain drifts from all three perspectives (Chapter 4). Therefore, we evaluate all three perspectives on three sets of 15 logs each:

- *Single-drift logs:* Each log has one drift (Section 7.2.1).

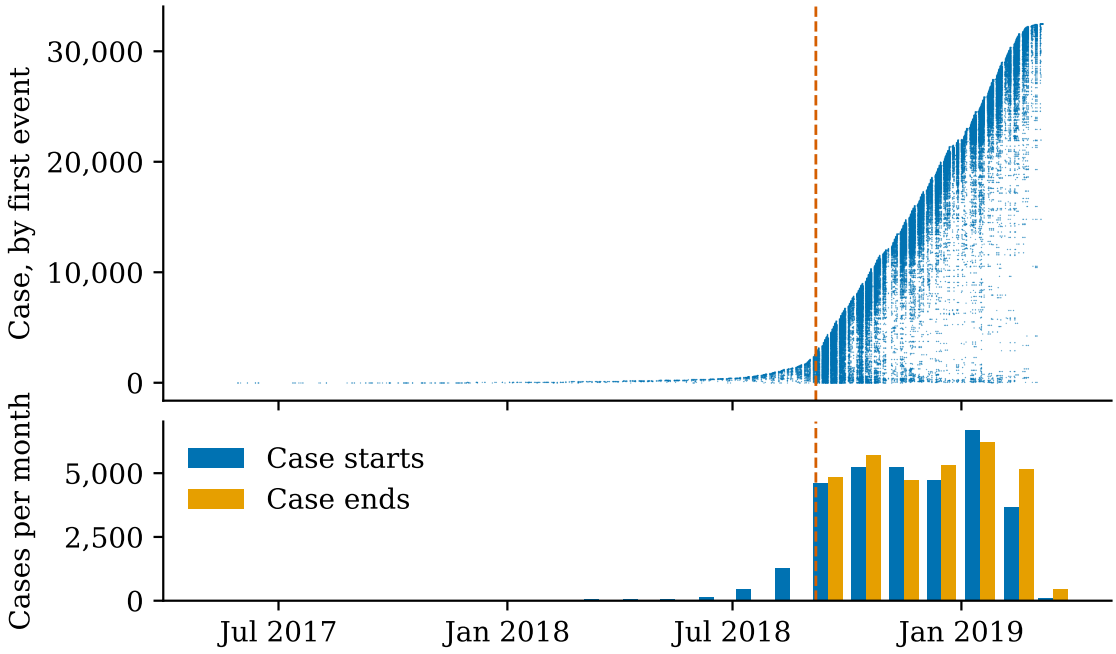


Figure 7.6: Dotted chart of the Padella baseline of Bank account closure with 32,429 cases and 212,721 events. Top: every event as a dot, with the cases ordered from bottom to top by their first event. Bottom: the cases that start (blue) and end (orange) in every month. Until August 2018, almost no case ends, and the few cases that start run long. From September 2018 to January 2019, 4,600 to 6,700 cases start and end per month. The dashed line marks the signal window from September 6, 2018, in which 7 of the 14 FPs of Run 1 in the six Bank account closure logs lie.

- *Multi-drift logs:* Each log has two to four drifts from the same perspective (Section 7.2.2).
- *Mixed-drift logs:* Each log has two to five drifts from different perspectives (Section 7.2.3).

The following holds for all three sets:

- *Expected perspective:* The expected perspective of a drift is the perspective of its drift type (Table 4.4). Apart from the injected drifts, Rheon samples the logs from fixed distributions (Section 4.2).
- *Reactions of other perspectives:* Some drift types also reach the features of other perspectives (Section 5.1), and such reactions count as FPs. 29 of the 38 FPs of Run 1 in all three sets lie at most two signal windows next to a drift of another perspective, and most of the others lie at the start or the end of a log. Thus,

	Single-drift logs		Multi-drift logs		Mixed-drift logs	
Run 1	14	1	25	19	36	8
	12	1,447	13	1,178	13	1,092
Run 2	14	1	39	5	40	4
	1	1,458	3	1,270	4	1,056

Figure 7.7: Confusion matrices of both runs on the three sets of Rheon logs over all three perspectives, with TP and FN in the first row and FP and TN in the second row of each matrix. Rows: actual, columns: detected.

a drift often shows in several perspectives, and its perspective only follows from comparing the states of all three perspectives.

- *Run 2:* Run 2 avoids most of these reactions: only one of its eight FPs lies next to a drift of another perspective, and seven lie at the start or the end of a log (Figure 7.7).

For each set, the following sections first present the results of both runs over all logs, then per drift perspective and drift mode.

7.2.1 Single-Drift Logs

The first set consists of 15 logs with one drift each, five per perspective. Eight drifts are sudden and seven gradual. The logs have 3,254 to 124,021 cases, time spans from one month to five years, 4 to 22 activities and 3 to 60 resources.

- *Result:* Run 1 detects 14 of the 15 drifts with 12 FPs and reaches an F1-score of 0.683 (Table 7.3). Nine of the 12 FPs are reactions of other perspectives to the drift of the log, and the other three lie at the end of a log. Here, too, the precision of 0.538 limits the F1-score of Run 1, whose recall is 0.933. Run 2 also detects 14 drifts, but with only one FP, and reaches an F1-score of 0.933. Thus, the states almost always detect a single drift in its expected perspective.
- *Intra-case drifts:* Run 1 detects four of the five `control_flow` drifts. Only the resource perspective reacts to the missed gradual drift, since its features include the shares of the activities and the handovers of the resources. All seven FPs of these logs come from the resource and the inter-case perspectives, which also react to the `control_flow` drifts (Figures 5.4 and 5.5). Run 2 also detects four of them, but misses a sudden drift instead of the gradual one, and has no FP in these logs. Thus, a detection in the resource or the inter-case perspective does not always indicate a drift of that perspective.
- *Resource drifts:* Both runs detect all five resource drifts. In Run 1, the inter-case perspective also reacts to the `workload` drift with an FP, and in both runs the

resource perspective reacts to the `arrival_rate` drift. Both drift types change the counts in the same way, so that no feature distinguishes them (Sections 5.1.2 and 5.1.3).

- *Inter-case drifts:* Both runs detect all five inter-case drifts, also the gradual `waiting_time` drift, which doubles the mean waiting time over almost six months. In the logs of the three attribute drifts (`amount` and `region`), no perspective of Run 1 has an FP except one at the end of a log, and Run 2 has none. Since attribute drifts only reach the attribute features, the states detect them clearly.
- *Sudden drifts:* Run 1 detects all eight sudden drifts with seven FPs (F1-score 0.696). Six of these FPs are reactions of other perspectives to the drift of their log. Run 2 misses one sudden `control_flow` drift but has no FP (F1-score 0.933). Thus, the states detect almost every single sudden drift, and most FPs of Run 1 come from the other perspectives.
- *Gradual drifts:* Run 1 detects six of the seven gradual drifts with five FPs (F1-score 0.667) and only misses one `control_flow` drift. Run 2 detects all seven with one FP (F1-score 0.933). Thus, with a single drift per log, the F1-score hardly depends on the drift mode.

Table 7.3: Counts, precision, recall and F1 of both runs on the 15 single-drift Rheon logs, over all logs, per drift perspective and per drift mode. Each group pools all drift signals of the logs whose drift belongs to this perspective or has this drift mode.

Group	Run	TP	FP	FN	TN	Precision	Recall	F1
All	1	14	12	1	1,447	0.538	0.933	0.683
	2	14	1	1	1,458	0.933	0.933	0.933
Intra-case	1	4	7	1	539	0.364	0.800	0.500
	2	4	0	1	546	1.000	0.800	0.889
Resource	1	5	1	0	369	0.833	1.000	0.909
	2	5	0	0	370	1.000	1.000	1.000
Inter-case	1	5	4	0	539	0.556	1.000	0.714
	2	5	1	0	542	0.833	1.000	0.909
Sudden	1	8	7	0	637	0.533	1.000	0.696
	2	7	0	1	644	1.000	0.875	0.933
Gradual	1	6	5	1	810	0.545	0.857	0.667
	2	7	1	0	814	0.875	1.000	0.933

7.2.2 Multi-Drift Logs

The second set consists of 15 logs with two to four drifts from the same perspective, five logs per perspective, with 30 sudden and 14 gradual drifts in total. The logs have 3,132 to 57,197 cases, time spans from six weeks to five years, 5 to 22 activities and 3 to 35 resources.

- *Result:* We expect several drifts of one perspective to be harder to detect than a single drift. Run 1 detects 25 of the 44 drifts with 13 FPs and reaches an F1-score of 0.610 (Table 7.4). Compared to the single-drift logs, its recall drops from 0.933 to 0.568. 11 of the 13 FPs are reactions of perspectives without a drift of their own to a drift in the log. Run 2, however, detects 39 of the 44 drifts with three FPs (F1-score 0.907). With suitable configurations, several drifts of one perspective are thus hardly harder to detect, with a recall of 0.886 against 0.933.

- *Intra-case drifts:* In Run 1, the `control_flow` logs only reach a recall of 0.429. Run 1 detects six of the nine sudden but none of the five gradual `control_flow` drifts. In four of the five logs, the resource perspective also reacts to a `control_flow` drift, and in three of them also the inter-case perspective. Each of these reactions counts as an FP. Run 2 detects all 14 `control_flow` drifts without any FP. Thus, several control-flow drifts in one log need a suitable configuration, but then the intra-case states detect all of them.

- *Resource drifts:* Run 1 fully detects the logs with only `pool_size` or only `reassignment` drifts. However, it misses one gradual `reassignment` drift, all three `workload` drifts and both `duration` drifts. The two `duration` drifts only affect two of the 10 and two of the 20 resources. Run 2 detects all three `workload` drifts, all four `reassignment` drifts and one of the two `duration` drifts, but misses a sudden `pool_size` drift. Its three FPs lie at the start or the end of a log. Thus, the resource states show `workload` drifts and `duration` drifts of few resources only with suitable configurations.

- *Inter-case drifts:* Run 1 detects all five `region` drifts. It misses five drifts in two logs, among them three of the five `waiting_time` drifts. In one log, the resource perspective reacts to an `arrival_rate` and a `waiting_time` drift, since its features include the counts and the mean waiting time until the resource (Figure 5.4). Run 2 detects all drifts except three of the five `waiting_time` drifts and has no FP in these logs. Thus, the inter-case states only partly separate several `waiting_time` drifts in one log.

- *Sudden drifts:* In Run 1, the logs with only sudden drifts reach an F1-score of 0.759. Both FNs come from one small log with four `control_flow` drifts in three months. In the other logs with only sudden drifts, Run 1 detects all drifts. Run 2 detects all four drifts of the small log and reaches an F1-score of 0.857, with one missed `pool_size` drift and three FPs.

- *Gradual drifts:* In the only log with only gradual drifts, Run 1 misses both `control_flow` drifts (F1-score 0), and the logs with both drift modes reach an F1-score of 0.560. Overall, Run 1 only detects four of the 14 gradual drifts, whereas Run 2 detects 12 of them and reaches F1-scores of 1.0 and 0.926. Thus, gradual drifts between other drifts of the same perspective are detectable, but only with a suitable configuration.

Table 7.4: Counts, precision, recall and F1 of both runs on the 15 multi-drift Rheon logs, over all logs, per drift perspective and per drift mode. Each group pools all drift signals of the logs whose drifts belong to this perspective or have exactly this set of drift modes.

Group	Run	TP	FP	FN	TN	Precision	Recall	F1
All	1	25	13	19	1,178	0.658	0.568	0.610
	2	39	3	5	1,270	0.929	0.886	0.907
Intra-case	1	6	8	8	384	0.429	0.429	0.429
	2	14	0	0	589	1.000	1.000	1.000
Resource	1	8	2	6	411	0.800	0.571	0.667
	2	12	3	2	342	0.800	0.857	0.828
Inter-case	1	11	3	5	383	0.786	0.688	0.733
	2	13	0	3	339	1.000	0.812	0.897
Sudden	1	11	5	2	534	0.688	0.846	0.759
	2	12	3	1	646	0.800	0.923	0.857
Gradual	1	0	1	2	83	0.000	0.000	0.000
	2	2	0	0	84	1.000	1.000	1.000
Sudden, gradual	1	14	7	15	561	0.667	0.483	0.560
	2	25	0	4	540	1.000	0.862	0.926

7.2.3 Mixed-Drift Logs

The third set consists of 15 logs with two to five drifts from different perspectives, with 12 intra-case, 15 resource and 17 inter-case drifts in total, 29 of them sudden and 15 gradual. In five logs, drifts of different perspectives share their drift point or drift window. The logs have 2,860 to 42,192 cases, time spans from one month to five years, 4 to 25 activities and 3 to 45 resources.

- *Result:* We expect simultaneous drifts of different perspectives to hide each other. The results do not confirm this: Run 1 detects all twelve drifts of the five logs with simultaneous drifts, and Run 2 eleven. Overall, Run 1 detects 36 of the 44 drifts with 13 FPs and, with an F1-score of 0.774, performs better than on the other two sets (Table 7.5). Run 2 detects 40 drifts with four FPs (F1-score 0.909). Since each perspective has its own states, our framework separates simultaneous drifts of different perspectives.

- *Intra-case drifts:* In Run 1, the intra-case perspective has no FP in any of the 15 logs, although the logs contain 32 drifts of other perspectives. Run 1 only misses one of the twelve `control_flow` drifts. Run 2 detects all twelve with one FP at the end of a log. Thus, the intra-case states are specific: a detection here indicates a control-flow drift.

- *Resource drifts:* Run 1 detects all six `reassignment` drifts, also next to a simultaneous `control_flow` drift. However, it misses two `duration`, two `workload` and one `pool_size` drift. Moreover, the resource perspective has 10 of the 13 FPs of

Run 1, six of them next to a `control_flow` drift and three at the start or the end of a log. Run 2 detects all `workload` and `pool_size` drifts and only misses one `duration` and one gradual `reassignment` drift. Its three resource FPs all lie at the start or the end of a log. Thus, in logs with several drifts the resource states only show `duration` and `workload` drifts and avoid reactions to control-flow drifts with suitable configurations.

- *Inter-case drifts:* Both runs detect all `region`, `waiting_time` and `arrival_rate` drifts and only miss two `amount` drifts each. In two logs, Run 1 detects the `arrival_rate` drift but not the `workload` drift, whereas Run 2 detects both. Thus, a change of the volume shows in the inter-case states in both runs, but in the resource states only with a suitable configuration.
- *Sudden drifts:* The logs with only sudden drifts reach F1-scores of 0.815 in Run 1 and 1.0 in Run 2. Three of these logs contain simultaneous sudden drifts, which do not hide each other.
- *Gradual drifts:* Both logs with only gradual drifts contain simultaneous drifts of several perspectives. Run 1 detects all of them (F1-score 0.833). Run 2 misses the gradual `amount` drift of one log and has two FPs at its end (F1-score 0.727). Thus, simultaneous gradual drifts mostly do not hide each other either.

Table 7.5: Counts, precision (Prec.), recall (Rec.) and F1 of both runs on the 15 mixed-drift Rheon logs, over all logs, per drift type and per drift mode. Each group pools all drift signals of the logs with exactly this set of drift types or drift modes.

Group	Run	TP	FP	FN	TN	Prec.	Rec.	F1
All	1	36	13	8	1,092	0.735	0.818	0.774
	2	40	4	4	1,056	0.909	0.909	0.909
amount, control_flow, duration	1	4	4	2	286	0.500	0.667	0.571
	2	4	3	2	292	0.571	0.667	0.615
amount, control_flow, pool_size	1	3	0	0	21	1.000	1.000	1.000
	2	3	0	0	22	1.000	1.000	1.000
amount, control_flow, pool_size, reassignment, region	1	5	1	0	109	0.833	1.000	0.909
	2	5	1	0	109	0.833	1.000	0.909
amount, control_flow, reassignment	1	2	1	1	50	0.667	0.667	0.667
	2	3	0	0	53	1.000	1.000	1.000
amount, pool_size, reassignment, waiting_time	1	3	0	1	125	1.000	0.750	0.857
	2	4	0	0	125	1.000	1.000	1.000
arrival_rate, control_flow, reassignment	1	3	1	0	135	0.750	1.000	0.857
	2	2	0	1	136	1.000	0.667	0.800
arrival_rate, control_flow, workload	1	2	1	2	20	0.667	0.500	0.571
	2	4	0	0	18	1.000	1.000	1.000
arrival_rate, workload	1	1	0	1	65	1.000	0.500	0.667
	2	2	0	0	12	1.000	1.000	1.000
control_flow, duration, region, waiting_time	1	3	1	1	39	0.750	0.750	0.750
	2	3	0	1	40	1.000	0.750	0.857
control_flow, reassignment	1	2	1	0	82	0.667	1.000	0.800
	2	2	0	0	86	1.000	1.000	1.000
control_flow, region	1	2	1	0	45	0.667	1.000	0.800
	2	2	0	0	46	1.000	1.000	1.000
control_flow, waiting_time	1	2	2	0	26	0.500	1.000	0.667
	2	2	0	0	28	1.000	1.000	1.000
reassignment, region	1	2	0	0	71	1.000	1.000	1.000
	2	2	0	0	71	1.000	1.000	1.000
waiting_time, workload	1	2	0	0	18	1.000	1.000	1.000
	2	2	0	0	18	1.000	1.000	1.000
Sudden	1	11	4	1	197	0.733	0.917	0.815
	2	12	0	0	207	1.000	1.000	1.000
Gradual	1	5	2	0	210	0.714	1.000	0.833
	2	4	2	1	215	0.667	0.800	0.727
Sudden, gradual	1	20	7	7	685	0.741	0.741	0.741
	2	24	2	3	634	0.923	0.889	0.906

8 Discussion

In this chapter we discuss the results and design choices of our framework. Section 8.1 answers our research questions from Section 1.3. Section 8.2 explains why the framework might be strong but also misleading and hard to evaluate. Section 8.3 describes useful hints and heuristics for choosing better configurations for event logs, Section 8.4 discusses the LLM assistance and Section 8.5 discusses runtime complexity and open questions.

8.1 Research Questions

Based on our evaluation on three sets of Rheon logs and on the Padella benchmark (Chapter 7), we answer the research questions from Section 1.3.

- RQ1 *States:* Feature extraction, PCA and clustering yield states in all three perspectives. The states are interpretable because their features show what changes. E.g., in the Padella log Bank account closure, each intra-case state of Run 1 corresponds to one activity such that the deletion of an activity is directly observable (Section 7.1).
- RQ2 *Detection:* The changes of the states signal drift points. In Run 2 on the Rheon logs, the framework detects 14 of the 15 drifts in the single-drift logs, 39 of the 44 drifts in the multi-drift logs and 40 of the 44 drifts in the mixed-drift logs. In Run 2 on the Padella logs, our framework detects 28 of 29 drifts. With the configurations of Run 1 on the Rheon logs, it is obviously harder to detect multiple drifts of the same perspective (recall 0.568).
- RQ3 *Characterization:* Since each perspective has its own states, the framework separates simultaneous drifts of different perspectives. However, some drift types also reach the features of other perspectives such that the perspective of a drift is better analyzed by looking at the drift signals of all perspectives.
- RQ4 *Applicability:* The framework is strong if the chosen features capture the drift and misleading otherwise. In logs with multiple drifts, Run 1 fails to detect some `duration` drifts with less number of resources and `workload` drifts whereas Run 2 is able to detect most of them.

8.2 Flexibility and Evaluation

The large hyperparameter space is the biggest disadvantage of the framework but the flexibility behind it is its advantage. With appropriate features the framework detects

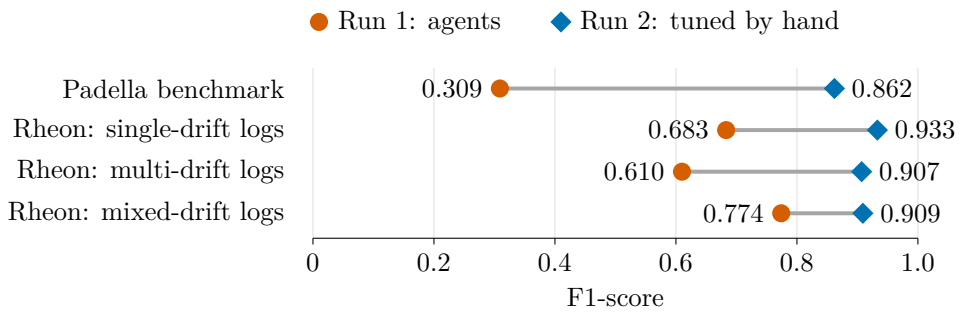


Figure 8.1: F1-scores of both runs on the Padella benchmark (34 logs, 29 drifts) and on the three sets of 15 Rheon logs each with a single drift, multiple drifts of one perspective and mixed drifts (15, 44 and 44 drifts). The gap between the configurations of the agents (circles) and those tuned by hand (diamonds) shows how strongly the results depend on the configuration.

drifts in all three perspectives and shows what changed. With inappropriate features, a drift stays undetected. E.g., if the resource perspective only counts events per resource, it misses the `time_fast` drift in the Padella log Loan application (Section 7.1). On the same Padella logs, Run 1 reaches an F1-score of 0.309 and Run 2 of 0.862. On the Rheon logs with multiple drifts of the same perspective, the F1-score rises from 0.610 in Run 1 to 0.907 in Run 2. That means, good results mainly show that suitable configurations exist, but not that every configuration works (Figure 8.1).

This makes the framework hard to evaluate:

- *Configuration per log:* A run reports its own configuration and threshold for each log and perspective. Thus the metrics always evaluate the framework together with the choice of its configuration.
- *Reactions of other perspectives:* Some drift types reach also the features of other perspectives. Such reactions count as FP, while the states show a "real" change. 29 of the 38 FPs in Run 1 on Rheon logs lie at most two signal windows nearby of a drift of another perspective. Run 2 avoids these reactions better (Section 7.2).
- *Real logs:* Real processes are not static even without an injected drift (first-order dynamics). They may contain changes that a benchmark does not know. 24 of the 30 FPs of Run 1 in the 14 logs of Domestic declarations and Request for payment lie at the turn of the year 2017/18, at which the approval steps really change (Section 7.1). Therefore, the precision on the Rheon logs is higher, in Run 2 from 0.909 to 0.933 against 0.778 on the Padella logs. Apart from the injected drifts, Rheon samples the logs from fixed distributions, so that no unknown real changes count as FPs.
- *Zones:* A detection only counts in the zone of a drift. The zone of a gradual drift covers more signal windows than the zone of a sudden drift and thus favors gradual

drifts (Section 7.1).

Thus, a precision, recall and F1-score "evaluate" the framework only partially.

8.3 Choosing a Configuration

For the clustering, a SOM with a large grid is useful in helping to choose k for k-means. It shows how many groups of frequently used cells lie far apart from each other (Section 5.3). DBSCAN is harder to tune. In our running example, the sorted k-dist graph has no clear knee for the choice of ε. Its k-distances also become slow for many rows (Section 6.1.1). This is another reason to use DBSCAN in the resource and inter-case perspectives, in which the rows are feature windows, and not in the intra-case perspective, in which they are events.

The choice of the window lengths and thresholds is still manual work. Similar to the heuristics for ε and *minPts* of DBSCAN (Section 3.4.3), we suggest three rules of thumb as a starting point. Let $span_L$ be the time span of the log L in days and $|E|$ its number of events:

- *Signal window:* $\Delta_s \approx \sqrt{span_L}$ days, rounded to 1, 3, 7, 14, 28 or 56 days. E.g., for a log over one year, this gives 14 days. Thus, both the length and the number of the signal windows grow with $\sqrt{span_L}$, which balances fluctuations against granularity in time (Section 5.4).
- *Feature window:* $\Delta_f \approx 300 \cdot span_L/|E|$ days, rounded to one hour, one day or one week. Thus, a feature window holds about 300 events on average.
- *Threshold:* The median of the drift scores of a drift signal describes its fluctuations without a drift. Therefore, an upper quantile of the drift scores, e.g., the 90% or 95% quantile, is a starting point for the threshold τ. However, a quantile always marks a fixed share of the signal windows, also without a drift.

For 86% of the 406 drift signals of our evaluation, the chosen Δ_s differs at most by a factor of 2 from the first rule. The second rule exactly matches 69% of the 248 chosen feature window lengths, among them all four windows of one hour and 18 of the 42 windows of one week. For 400 of the 406 drift signals, the chosen τ lies above the median of the drift scores, and for 82% at least at the 90% quantile. Thus, the rules summarize the configurations we chose, but they do not guarantee a good configuration.

8.4 LLM Support

Using our skill `kairo-drift-analyst`, an AI agent can execute the pipeline itself, compare configurations and deliver configurations and thresholds per perspective of a log (Section 6.1.2). The Kairo Copilot assists users in the dashboard (Section 6.2.2). Both help in searching the large hyperparameter space of our pipelines, but do not avoid the examination of a human. In the Padella benchmark, the Run 1 configurations by the

agents alone receive an F1-score of 0.309, whereas the Run 2 configurations yield 0.862. On the three Rheon sets of logs, Run 1 configurations from the agents receive F1-scores from 0.610 to 0.774, whereas Run 2 configurations receive F1-scores from 0.907 to 0.933. Besides, the results of LLMs should be treated skeptically and are very model-dependent. Therefore, we see our agent skill and Copilot as assistants of a human-in-the-loop-analysis and not as independent detectors.

8.5 Runtime and Open Issues

The runtime of the pipeline grows with the number of rows. In the intra-case perspective, each event becomes a row whereas in the resource and inter-case perspectives only each feature window becomes a row. Feature extraction, PCA, k-means and drift signals grow linearly with the number of rows, a SOM additionally with the number of its cells. The feature extraction in kairo uses pandas, which computes its operations mostly on a single core. Polars[1] distributes DataFrame operations on multiple cores, and cuDF[2] provides a pandas-like interface on the GPU. Both might accelerate the computations in especially the intra-case perspective. kairo trains a SOM in only one epoch, in which each row is used to update the neuron prototypes once in random order. One of the limitations of our implementation kairo is that it currently has no multi epoch support. Multi epoch support and GPU-based training are reasonable extension directions of our implementation.

Another topic we did not cover in our thesis is the evaluation of recurring drifts.

[1] `https://pola.rs`
[2] `https://docs.rapids.ai/api/cudf/stable/`

9 Conclusion

Real processes change while event logs capture them. Such a concept drift is not only about the order of activities, but also about the resources and the situation of all running cases. Existing detectors, however, mainly focus on control-flow or follow single KPIs. Also, existing public test data contain neither resource drifts nor arrival rate drifts (Section 2.6). Therefore, this thesis investigates whether a process can be monitored as a combination of intra-case, resource and inter-case states and whether this combination is a useful indicator for a concept drift (Section 1.3).

Hence, we first separate the terms drift type (what changes), drift perspective (which perspective a drift type belongs to) and drift mode (how the change affects the process over time). Our state-based framework adapts the execution states of Berti et al. [7] from objects to cases and extends them to resource and inter-case states of feature windows. Each perspective passes separately through four steps: feature extraction, scaling and PCA, clustering into states with a SOM, k-means or DBSCAN as well as drift signals (Chapter 5). The drift signal compares the state distribution of each signal window with a reference using a divergence. Since the drift signal is a time series, it is also suitable as input to change point detection algorithms like PELT [3] or ADWIN [5]. Rheon generates synthetic event logs with nine drift types from all three perspectives, each sudden or gradual, and lists the injected drifts in a metadata file (Chapter 4). Our library kairo implements the framework, and in our dashboard, users interpret the results with the LLM-based Kairo Copilot (Chapter 6). Thus, we achieve RG1 to RG5.

For the research goals RG6 and RG7, we evaluate our framework on the 34 logs of the Padella benchmark and on three sets of 15 Rheon logs each (Chapter 7). The Padella benchmark contains 29 control-flow and duration drifts in five real base logs, whereas the Rheon logs contain 103 drifts from all three perspectives. On each set, we compare two runs: in Run 1, agents created the configurations and thresholds, and in Run 2, we tuned them by hand. The results answer our research questions as follows (Section 8.1):

- RQ1 *States:* The states show what changed. E.g., in the Padella log Bank account closure, each intra-case state of Run 1 corresponds to one activity, so that the deletion of an activity is directly visible.
- RQ2 *Detection:* With the configurations of Run 2, our framework detects 28 of the 29 Padella drifts and 93 of the 103 Rheon drifts. Its F1-scores lie between 0.862 and 0.933 on all four sets (Figure 8.1).
- RQ3 *Characterization:* Since each perspective has its own states, simultaneous drifts of different perspectives do not hide each other. However, some drift types also reach the features of other perspectives. Thus, the perspective of a drift

only follows from comparing all three drift signals. A sudden drift shows as a spike against the previous signal window, whereas a gradual drift shows better in longer signal windows with a reference over several signal windows (Section 7.1). Section 5.1 shows to which drift types the features of each perspective respond, e.g., time drifts need the durations per resource (Section 7.1).

- RQ4 *Applicability:* On the same logs, the configurations of the agents only reach F1-scores from 0.309 to 0.774. A drift that no chosen feature captures stays hidden. Moreover, real logs contain changes that a benchmark does not know, e.g., at the turn of the year 2017/18 in two Padella base logs.

Therefore, we answer our main research question with yes. A process can be monitored as a combination of intra-case, resource and inter-case states, and their changes are a useful and explainable indicator for concept drift. However, this only holds if the chosen features capture the drift and the configuration fits the log. Our agent skill and the Copilot help to find such configurations, but do not replace a human (Section 8.4).

Future Work. Based on our results, we see five directions for future work:

- *Automatic configuration:* Since the configuration determines the results the most, choosing it automatically is the most important next step. The rules of thumb in Section 8.3 are a starting point for that.
- *Online drift detection:* On an event stream, scaling, PCA and clustering learn their models on a warm-up period. Afterwards, the framework only assigns each new row to a state. Without the mean of all signal windows as reference (Section 5.4) and without features that know the future of a case, such as the case progress, our dashboard then reports drifts live. After a drift, e.g., with new activities, the framework learns the models again.
- *Object-centric drift detection:* Berti et al. [7] compute execution states for objects in object-centric event logs (OCELs). Applied to OCELs, our framework computes states and a drift signal per object type. Adams et al. [3] already detect drifts in OCELs, but in single time series instead of states. For the evaluation, we plan to extend Rheon to object-centric logs with drifts (Section 4.5).
- *Rheon as training data:* Kraus and van der Aa [19] train the supervised detector CV4CDD-4D on 40,000 logs with control-flow drifts generated by CDLG and plan to include the time, resource and data perspectives. Rheon logs provide such training data from all three perspectives, and their metadata files serve as labels.
- *State distances:* The divergences only compare the shares of the states (Section 5.4). Thus, a transition into a similar state counts as much as a transition into a very different state. As for the trace distributions of Brockhoff et al. [11], the Earth Mover's Distance additionally weighs each shift by the distance between the states.

Bibliography

[1] B. R. Abarca Zúñiga, A. Yeshchenko, and H. van der Aa. "Version Clustering: A Top-Down Approach for Process Concept Drift Detection." In: *Advanced Information Systems Engineering - 38th International Conference, CAiSE 2026, Verona, Italy, June 8-12, 2026, Proceedings, Part I*. Ed. by L. Fuentes, P. Plebani, C. Combi, and H. Reijers. Lecture Notes in Computer Science. Springer, 2026, pp. 61–77. DOI: 10.1007/978-3-032-28110-4_4.

[2] J. N. Adams, C. Pitsch, T. Brockhoff, and W. M. P. van der Aalst. "An Experimental Evaluation of Process Concept Drift Detection." In: *Proceedings of the VLDB Endowment* 16.8 (2023), pp. 1856–1869. DOI: 10.14778/3594512.3594517.

[3] J. N. Adams, S. J. van Zelst, T. Rose, and W. M. P. van der Aalst. "Explainable Concept Drift in Process Mining." In: *Information Systems* 114 (2023), p. 102177. DOI: 10.1016/j.is.2023.102177.

[4] E. Alpaydin. *Introduction to Machine Learning*. 2nd ed. Adaptive Computation and Machine Learning. MIT Press, 2010. ISBN: 978-0-262-01243-0.

[5] L. Baier, J. Reimold, and N. Kühl. "Handling Concept Drift for Predictions in Business Process Mining." In: *22nd IEEE Conference on Business Informatics, CBI 2020, Antwerp, Belgium, June 22-24, 2020*. IEEE, 2020, pp. 76–83. DOI: 10.1109/CBI49978.2020.00016.

[6] S. Barbon Junior, G. M. Tavares, V. G. Turrisi da Costa, P. Ceravolo, and E. Damiani. "A Framework for Human-in-the-Loop Monitoring of Concept-Drift Detection in Event Log Stream." In: *Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon, France, April 23-27, 2018*. ACM, 2018, pp. 319–326. DOI: 10.1145/3184558.3186343.

[7] A. Berti, D. Kretzschmann, and W. M. P. van der Aalst. *Identifying Execution States and Boundary Conditions in Object-Centric Event Logs: A Framework and Case Study*. TechRxiv preprint. 2025. DOI: 10.36227/techrxiv.176522567.74931712/v1.

[8] A. Berti, D. Schuster, and W. M. P. van der Aalst. "Abstractions, Scenarios, and Prompt Definitions for Process Mining with LLMs: A Case Study." In: *Business Process Management Workshops - BPM 2023 International Workshops, Utrecht, The Netherlands, September 11-15, 2023, Revised Selected Papers*. Ed. by J. De Weerdt and L. Pufahl. Vol. 492. Lecture Notes in Business Information Processing. Springer, 2024, pp. 427–439. DOI: 10.1007/978-3-031-50974-2_32.

[9] A. Berti, S. J. van Zelst, and W. M. P. van der Aalst. "Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science." In: *Proceedings of the ICPM Demo Track 2019, Co-Located with 1st International Conference on Process Mining (ICPM 2019), Aachen, Germany, June 24-26, 2019*. Ed. by A. Burattin, A. Polyvyanyy, and S. J. van Zelst. Vol. 2374. CEUR Workshop Proceedings. CEUR-WS.org, 2019, pp. 13–16. URL: https://ceur-ws.org/Vol-2374/paper4.pdf.

[10] R. P. J. C. Bose, W. M. P. van der Aalst, I. Žliobaitė, and M. Pechenizkiy. "Handling Concept Drift in Process Mining." In: *Advanced Information Systems Engineering - 23rd International Conference, CAiSE 2011, London, UK, June 20-24, 2011. Proceedings*. Ed. by H. Mouratidis and C. Rolland. Vol. 6741. Lecture Notes in Computer Science. Springer, 2011, pp. 391–405. DOI: 10.1007/978-3-642-21640-4_30.

[11] T. Brockhoff, M. S. Uysal, and W. M. P. van der Aalst. "Time-Aware Concept Drift Detection Using the Earth Mover's Distance." In: *2nd International Conference on Process Mining, ICPM 2020, Padua, Italy, October 5-8, 2020*. IEEE, 2020, pp. 33–40. DOI: 10.1109/ICPM49681.2020.00016.

[12] T. Caliński and J. Harabasz. "A Dendrite Method for Cluster Analysis." In: *Communications in Statistics* 3.1 (1974), pp. 1–27. DOI: 10.1080/03610927408827101.

[13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise." In: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), Portland, Oregon, USA*. Ed. by E. Simoudis, J. Han, and U. M. Fayyad. AAAI Press, 1996, pp. 226–231. URL: https://www.aaai.org/Library/KDD/1996/kdd96-037.php.

[14] J. Grimm, A. Kraus, and H. van der Aa. "CDLG: A Tool for the Generation of Event Logs with Concept Drifts." In: *Proceedings of the Best Dissertation Award, Doctoral Consortium, and Demonstration & Resources Track at BPM 2022, Münster, Germany, September 11-16, 2022*. Ed. by C. Janiesch, C. Di Francescomarino, T. Grisold, A. Kumar, J. Mendling, B. Pentland, H. Reijers, M. Weske, and R. Winter. Vol. 3216. CEUR Workshop Proceedings. CEUR-WS.org, 2022, pp. 92–96. URL: https://ceur-ws.org/Vol-3216/paper_241.pdf.

[15] S. Haykin. *Neural Networks and Learning Machines*. 3rd ed. Pearson Education, 2009. ISBN: 978-0-13-147139-9.

[16] S. Khraiwesh, D. Dagtekin, and L. Pufahl. "CV-Driven Window Selection for Multi-Perspective Concept Drift Detection in Event Logs." In: *Business Process Management Forum - BPM 2026 Forum, Toronto, ON, Canada, September 27 - October 2, 2026, Proceedings*. Ed. by F. M. Maggi, H. Leopold, F. Santoro, and A. Gal. Vol. 599. Lecture Notes in Business Information Processing. Springer, 2027, pp. 185–202. DOI: 10.1007/978-3-032-37877-4_11.

[17] E. L. Klijn, F. Mannhardt, and D. Fahland. "Multi-Perspective Concept Drift Detection: Including the Actor Perspective." In: *Advanced Information Systems Engineering - 36th International Conference, CAiSE 2024, Limassol, Cyprus, June 3-7, 2024, Proceedings*. Ed. by G. Guizzardi, F. Santoro, H. Mouratidis, and P. Soffer. Vol. 14663. Lecture Notes in Computer Science. Springer, 2024, pp. 141–157. DOI: 10.1007/978-3-031-61057-8_9.

[18] A. Kraus, K. Amiri Elyasi, A. Rebmann, S. Hadian, and H. van der Aa. "A Framework for Steady-State Detection in Process Mining." In: *Information Systems* 140 (2026), p. 102740. DOI: 10.1016/j.is.2026.102740.

[19] A. Kraus and H. van der Aa. "Machine Learning-Based Detection of Concept Drift in Business Processes." In: *Process Science* 2.1 (2025), p. 5. DOI: 10.1007/s44311-025-00012-w.

[20] A. Kraus and H. van der Aa. "Comprehensive Characterization of Concept Drifts in Process Mining." In: *Information Systems* 135 (2026), p. 102584. DOI: 10.1016/j.is.2025.102584.

[21] J. Lin. "Divergence Measures Based on the Shannon Entropy." In: *IEEE Transactions on Information Theory* 37.1 (1991), pp. 145–151. DOI: 10.1109/18.61115.

[22] A. Maaradji, M. Dumas, M. La Rosa, and A. Ostovar. "Detecting Sudden and Gradual Drifts in Business Processes from Execution Traces." In: *IEEE Transactions on Knowledge and Data Engineering* 29.10 (2017), pp. 2140–2154. DOI: 10.1109/TKDE.2017.2720601.

[23] D. Moulavi, P. A. Jaskowiak, R. J. G. B. Campello, A. Zimek, and J. Sander. "Density-Based Clustering Validation." In: *Proceedings of the 2014 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics, 2014, pp. 839–847. DOI: 10.1137/1.9781611973440.96.

[24] A. Padella and F. Vinci. *Everything Real but the Drift: A Concept-Drift Benchmark Event Logs for Process Mining*. Zenodo dataset, version 1. 2026. DOI: 10.5281/zenodo.21005862.

[25] P. J. Rousseeuw. "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis." In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65. DOI: 10.1016/0377-0427(87)90125-7.

[26] D. M. V. Sato, S. C. De Freitas, J. P. Barddal, and E. E. Scalabrin. "A Survey on Concept Drift in Process Mining." In: *ACM Computing Surveys* 54.9 (2022), 189:1–189:38. DOI: 10.1145/3472752.

[27] A. Senderovich, C. Di Francescomarino, C. Ghidini, K. Jorbina, and F. M. Maggi. "Intra and Inter-Case Features in Predictive Process Monitoring: A Tale of Two Dimensions." In: *Business Process Management - 15th International Conference, BPM 2017, Barcelona, Spain, September 10-15, 2017, Proceedings*. Ed. by J. Carmona, G. Engels, and A. Kumar. Vol. 10445. Lecture Notes in Computer Science. Springer, 2017, pp. 306–323. DOI: 10.1007/978-3-319-65000-5_18.

[28] W. M. P. van der Aalst. *Process Mining: Data Science in Action*. 2nd ed. Springer, 2016. DOI: 10.1007/978-3-662-49851-4.

[29] C. Zheng, L. Wen, and J. Wang. "Detecting Process Concept Drifts from Event Logs." In: *On the Move to Meaningful Internet Systems. OTM 2017 Conferences - Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23-27, 2017, Proceedings, Part I*. Ed. by H. Panetto, C. Debruyne, W. Gaaloul, M. Papazoglou, A. Paschke, C. A. Ardagna, and R. Meersman. Vol. 10573. Lecture Notes in Computer Science. Springer, 2017, pp. 524–542. DOI: 10.1007/978-3-319-69462-7_33.